

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ

Український державний університет науки і технологій

ННІ «Український державний хіміко-технологічний університет»

Кафедра комп'ютерно-інтегрованих технологій та автоматизації

В авторській редакції

ОБЧИСЛЮВАЛЬНА МАТЕМАТИКА ТА ПРОГРАМУВАННЯ

Навчально-методичні рекомендації до лабораторних занять

Електронне видання

Дніпро

2025

Упорядники: *А. Р. Шейкус, В. Р. Бакеркін*

Електронне видання

Схвалено групою забезпечення якості освітньої програми

«Хімічні технології та інженерія»

Протокол № 2 від 06.05.2025

Обчислювальна математика та програмування: навчально-методичні рекомендації до лабораторних занять / упоряд. А. Р. Шейкус, В. Р. Бакеркін; Укр. держ. ун-т науки і технологій, ННІ «Український державний хіміко-технологічний університет». Електрон. вид. – Дніпро: УДУНТ, ННІ УДХТУ, 2025. – 173 с.

Навчально-методичні рекомендації до лабораторних занять з дисципліни «Обчислювальна математика та програмування» для студентів освітньої програми «Хімічні технології та інженерія» спеціальності 161 (G1) «Хімічні технології та інженерія» першого (бакалаврського) рівня вищої освіти всіх форм навчання.

Навчально-методичні рекомендації містять основні теоретичні відомості, завдання і інструкції до виконання лабораторних робіт, питання для самоконтролю, перелік посилань.

Іл. 177. Табл. 30. Бібліогр.: 23 назви.

ЗМІСТ

Передмова.....	4
Вимоги з охорони праці під час виконання лабораторних робіт	6
Лабораторна робота № 1. Програмування мовою <i>Python</i> . Інтерпретатор <i>Python</i> як калькулятор. Розгалужені і циклічні програми.....	8
Лабораторна робота № 2. Оцінка похибок обчислень. Базові операції над матрицями.....	29
Лабораторна робота № 3. Побудова лінійних графіків функцій. Розв’язання алгебраїчних і трансцендентних рівнянь.....	51
Лабораторна робота № 4. Розв’язання систем лінійних і нелінійних рівнянь.....	77
Лабораторна робота № 5. Апроксимація табличних функцій.....	93
Лабораторна робота № 6. Диференціювання і інтегрування функцій.....	120
Лабораторна робота № 7. Розв’язання звичайних диференціальних рівнянь та їх систем.....	141
Лабораторна робота № 8. Розв’язання оптимізаційних задач.....	162
Список використаної літератури.....	171

ПЕРЕДМОВА

Дисципліна «Обчислювальна математика та програмування» є важливим компонентом підготовки фахівців за спеціальністю «Хімічні технології та інженерія». Сучасна хімічна технологія ґрунтується на високоточних обчисленнях, комп'ютерних моделях процесів і апаратів, аналізі великих обсягів даних. Це вимагає від фахівця глибокого розуміння числових методів розрахунків та вміння застосовувати програмні засоби для розв'язання наукових і професійних задач.

Метою дисципліни «Обчислювальна математика та програмування» є забезпечення студентів необхідними знаннями та навичками з програмної реалізації методів числового аналізу для вирішення математичних задач.

Вивчення даної дисципліни сприяє розвитку навичок автоматизації рутинних обчислень, оптимізації виробничих процесів та розробки нових технологій. Числові методи, наприклад, використовуються при моделюванні хімічних реакцій, аналізі експериментальних даних, прогнозуванні результатів досліджень тощо.

Виконання лабораторних робіт з дисципліни «Обчислювальна математика та програмування» допомагає студентам закріпити теоретичні знання, розвинути алгоритмічне мислення і отримати практичні навички роботи з програмними інструментами.

Для виконання лабораторних робіт пропонується мова програмування *Python*, яка є зручною та ефективною при вирішенні обчислювальних задач. *Python* має ряд переваг, таких як простий синтаксис і легкість вивчення, велика кількість бібліотек для числових розрахунків, підтримка та можливість інтеграції з іншими інструментами тощо. Ці переваги дозволяють студентам швидко опанувати основи програмування та зосередитися на вирішенні конкретних прикладних задач.

Функції бібліотеки *Math* дозволяють виконати базові математичні обчислення. Для роботи з матрицями та вирішення задач лінійної алгебри використовуються функції бібліотеки *NumPy*. Для реалізації числових методів вирішення науково-технічних задач призначена бібліотека *SciPy*. Виконувати аналітичні розрахунки і символічні перетворення виразів дозволяють функції бібліотеки *SymPy*. Бібліотека *Matplotlib* призначена для візуалізації даних та побудови графіків функцій.

Виконувати лабораторні роботи пропонується в *JupyterLab*. Дане середовище розробки дозволяє інтерактивно працювати з кодом, візуалізувати результати та документувати процес виконання завдань. Це сприяє кращому розумінню обчислень і засвоєнню матеріалу, розвитку навичок самостійної роботи.

В процесі виконання лабораторних робіт студенти набувають навичок програмування мовою *Python*, застосування числових методів, аналізу та візуалізації даних, розв'язання алгебраїчних і диференціальних рівнянь, а також оптимізації технологічних процесів.

Метою даних навчально-методичних інструкцій є забезпечення студентів чіткими рекомендаціями щодо виконання лабораторних робіт, що допомагає засвоїти теоретичний матеріал і розвинути практичні навички. Інструкції сприяють ефективній організації навчального процесу, спрощують підготовку до виконання робіт та аналіз отриманих результатів.

Навчально-методичні інструкції сприяють досягненню результатів навчання завдяки структурованому викладу матеріалу, доступності теоретичних і практичних аспектів, а також застосуванню сучасного програмного забезпечення для розв'язання задач. Вони допомагають студентам сформулювати цілісне уявлення про дисципліну «Обчислювальна математика та програмування» і підвищити рівень професійної підготовки майбутніх інженерів-технологів.

В методичних вказівках розглянуто основи програмування мовою *Python*, автоматизацію арифметичних операцій, реалізацію розгалужених і циклічних конструкцій, оцінку похибок розрахунків, матричні обчислення, представлення результатів розрахунків в графічній формі, графічне і числове розв'язання алгебраїчних і трансцендентних рівнянь, систем лінійних і нелінійних рівнянь, апроксимацію експериментальних даних, розрахунки похідних функцій і визначених інтегралів, розв'язання звичайних диференціальних рівнянь та їх систем, визначення екстремумів функції однієї змінної.

Вимоги з охорони праці під час виконання лабораторних робіт

1. Загальні положення.

1.1. Дані правила регламентують порядок перебування та роботи студентів у комп'ютерних аудиторіях під час навчального процесу.

1.2. Метою правил є забезпечення безпечних умов праці та збереження комп'ютерної техніки.

1.3. Відповідальність за дотримання правил покладається на студентів, викладачів та технічний персонал.

2. Правила поведінки в аудиторії.

2.1. Вхід до лабораторії можливий лише за дозволом викладача.

2.2. До виконання лабораторної роботи допускаються студенти, які ознайомились з порядком виконання роботи та пройшли первинний інструктаж з охорони праці на робочому місці.

2.3. Розпочинати виконання роботи дозволяється лише після перевірки справності обладнання.

2.4. Під час роботи необхідно дотримуватись тиші та не відволікати інших студентів.

2.5. В лабораторії забороняється приймати їжу, курити, розміщувати сторонні предмети на робочому місці і обладнанні.

2.6. Забороняється встановлювати або видаляти програмне забезпечення без дозволу викладача; змінювати налаштування системи (роздільну здатність екрану, параметри мережі тощо).

2.7. При виявленні несправностей обладнання слід негайно повідомити викладача та припинити виконання роботи до їх усунення.

2.8. Робоче місце повинно підтримуватись у чистоті.

3. Ергономічні вимоги.

3.1. Висота сидіння має забезпечувати кут у колінах 90° та опору стоп на підлозі. Під час роботи необхідно дотримуватися правильної постави.

3.2. Відстань від очей до екрану монітора має становити 50–70 см; верхній край екрану повинен знаходитись на рівні очей або трохи нижче.

3.3. Рівень освітлення робочої зони має становити 300–500 лк; джерела світла повинні бути розташовані так, щоб уникати відблисків на екрані.

3.4. Після кожних 45–50 хвилин роботи необхідно робити перерву 5–10 хвилин для відпочинку очей та виконання простих фізичних вправ.

3.5. Температура в аудиторії має становити 18–22 °С. При цьому має бути забезпечено постійний доступ свіжого повітря.

4. Електробезпека.

4.1. Перед початком роботи необхідно перевірити цілісність проводів, роз'ємів та справність розеток.

4.2. Забороняється:

- працювати з приладами, які перебувають під струмом, за відсутності заземлення або при порушенні ізоляції;
- без дозволу викладача вмикати електрообладнання;
- вмикати/вимикати електрообладнання вологими руками;
- самовільно підключати подовжувачі, розгалужувачі чи інші електропристрої без дозволу викладача.

4.3. У разі перегріву обладнання, припинення подачі електроенергії, появи запаху горіння, диму, вогню необхідно негайно припинити роботу та повідомити викладача.

5. Пожежна безпека.

5.1. Заборонено блокувати проходи, коридори та виходи з аудиторії.

5.2. Необхідно знати план евакуації, розташування аварійних виходів, вогнегасників та пожежних кранів.

5.3. У разі виявлення пожежі слід негайно повідомити викладача та дотримуватись його інструкцій щодо евакуації.

6. Домедична допомога.

6.1. У разі ураження електричним струмом:

- необхідно негайно звільнити постраждалого від дії струму – вимкнути електроживлення на щиті керування;
- якщо постраждалий при свідомості (навіть якщо раніше втрачав свідомість), його потрібно зручно укласти та забезпечити спокій;
- при відсутності свідомості, але наявному диханні та пульсі – розстебнути одяг, забезпечити доступ повітря;
- при відсутності ознак життя необхідно негайно розпочати штучне дихання та масаж серця.

6.2. При порізах – промити рану, обробити антисептиком і забинтувати.

6.3. При забитих місцях або розтягненнях — забезпечити спокій і прикласти холод.

6.4. При вивихах і переломах – забезпечити нерухомість ушкодженої частини, накласти шину; при відкритому переломі – обробити шкіру навколо рани йодом і накласти стерильну пов'язку.

6.5. Про кожний нещасний випадок свідок або сам потерпілий повинні повідомити викладача.

Лабораторна робота № 1

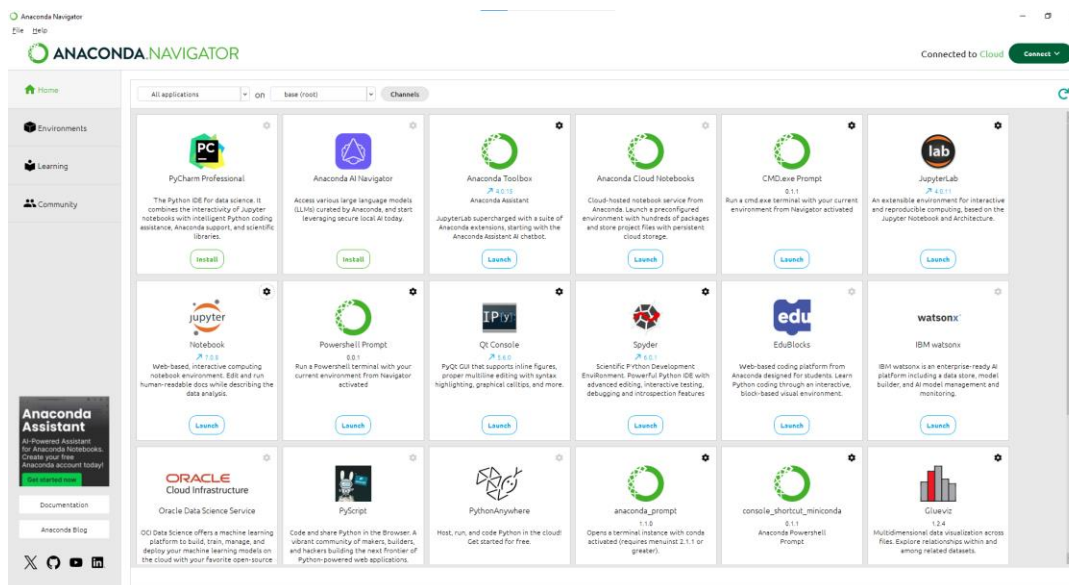
ПРОГРАМУВАННЯ МОВОЮ *PYTHON*. ІНТЕРПРЕТАТОР *PYTHON* ЯК КАЛЬКУЛЯТОР. РОЗГАЛУЖЕНІ І ЦИКЛІЧНІ ПРОГРАМИ

Мета роботи: ознайомлення з інтерфейсом і можливостями інтегрованого середовища розробки *JupyterLab*, з основними числовими типами даних мови *Python*. Одержання практичних навичок використання умовних операторів, організації циклів і написання програм для обчислення математичних виразів.

1.1 Інсталяція, запуск і початок роботи в *JupyterLab*

В рамках навчального курсу для зручності і наочності пропонується використання середовища розробки *JupyterLab*, що є програмою-блоком для запису і запуску програмного коду в *Python*. Для роботи рекомендується попередньо встановити графічний інтерфейс *Anaconda Navigator* [1].

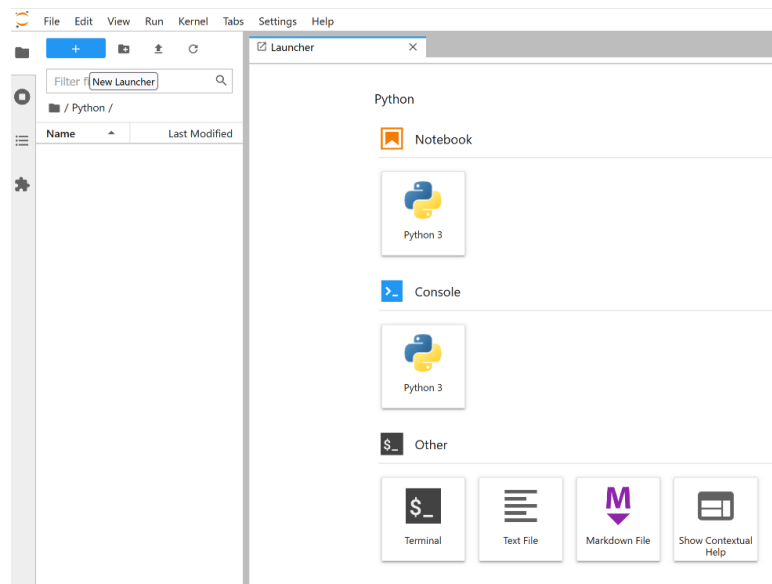
Запуск *JupyterLab* здійснюється натисненням кнопки «Launch» у відповідній комірці інтерфейсу:



Відкриття середовища відбувається у вкладці інтернет-браузера.

Також можлива робота в онлайн-сервісах, наприклад, в *JupyterLite* [2].

Після запуску середовища розробки необхідно створити новий проект, для чого у вкладці «Launcher» в полі «Notebook» натискається кнопка «Python». Проект створюється в новій вкладці. Вкладка «Launcher» відкривається натисненням на кнопку «New Launcher» синього кольору з позначкою «+»:



Новий проект за замовчуванням має назву «Untitled». Змінити назву можливо натисненням правої клавіші миші на проект в файловому менеджері ліворуч. В контекстному меню, що відкрилось, обирається команда «Rename» і вводиться нова назва.

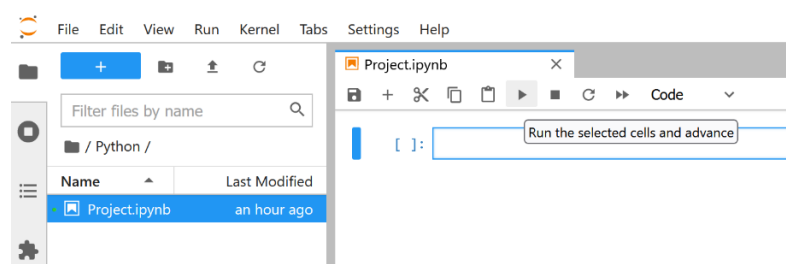
Створення нової папки здійснюється за допомогою кнопки «New folder». Папку можливо перейменувати так само, як і проект.

Відкриття проекту здійснюється подвійним натисненням лівої клавіші миші на його назві в файловому менеджері ліворуч. Якщо проект розташовується не в робочій директорії, то його відкриття можливе з використанням кнопки «Upload Files» і обранням відповідного файлу.

Зберегти зміни в проекті можливо натисненням на кнопку «Save and create checkpoint» з позначкою дискети.

Додавання комірок в проект здійснюється кнопкою «Insert a cell below» з позначкою «+». Додавання і видалення комірок, очищення результатів розрахунків також можливе через контекстне меню, що викликається правою клавішею миші.

Виконання операцій в комірці і одержання результатів здійснюється її виділенням і натисненням на кнопку «Run the selected cells and advance»:



1.2 Основи математичних розрахунків у Python

Перш ніж використовувати можливості, які надає *Python* для вирішення задач обчислювальної математики, необхідно розглянути основи його синтаксису.

Символьні (текстові) рядки в *Python* вводяться з використанням лапок ('...' або "..."). Символьний рядок може містити будь-які символи, включаючи літери, цифри та пробіли. В *JupyterLab* рядкові літерали мають червоний колір:

```
' Текстовий рядок '
```

Змінна – це іменована область пам'яті, яка зберігає певне значення. Динамічно змінювати зв'язки змінних з їх значеннями дозволяє операція присвоєння. Присвоєння здійснюється з використанням оператора `=`.

Ім'я змінної повинно починатись з літери алфавіту або підкреслення. Може містити літери, цифри та підкреслення. При цьому великі та малі літери розрізняються:

```
x = 15
_ror = 'ror'
X = 23
```

В якості імен змінних в *Python* не можна використовувати зарезервовані (ключові) слова, наприклад: *as*, *break*, *continue*, *def*, *elif*, *else*, *for*, *from*, *if*, *import*, *in*, *return* тощо [3].

Вивести на екран текстовий рядок або значення змінної можливо з використання функції *print()*:

```
x = 15
print( ' x ' )
print( x )
```

```
x
15
```

Однією командою *print()* можливо вивести на екран декілька об'єктів. Для цього їх необхідно перерахувати в якості параметрів функції через кому:

```
x = 15
X = 23
print( 'a =', x + X )
```

```
a = 38
```

В *Python* кінець рядка є кінцем інструкції. Для того, щоб записати декілька інструкцій в одному рядку, їх необхідно розділити крапкою з комою:

```
a = 10; print( a )
```

```
10
```

В *JupyterLab* можливе виведення значення змінної без використання команди `print()`. Для цього в останній інструкції комірки необхідно вказати ім'я цієї змінної:

```
x = 15  
x
```

15

Для пояснення коду та його документації використовуються коментарі. В *Python* однорядковий коментар починається символом `#` і закінчується кінцем рядка. Коментарі ігноруються при компіляції або інтерпретації програми.

В *JupyterLab* однорядкові коментарі мають зелений колір:

```
x = 15 # Це коментар
```

В *Python* існують три основні числові типи даних:

- ціле число (*int*): 1, -5, 42;
- дійсне число (*float*): -0.001, 23.273;
- комплексне число (*complex*): $2 + 3j$, де j – це уявна одиниця.

Дізнатись тип змінної можливо використанням команди `type()`:

```
x = 15  
y = 2.32  
a = 2 + 4j  
print( type( x ) )  
print( type( y ) )  
print( type( a ) )  
  
<class 'int'>  
<class 'float'>  
<class 'complex'>
```

Для представлення дійсних чисел можливо використовувати експоненційну форму запису:

```
x = 1.5e6 # = 1,5 · 10 ** 6  
print(x)
```

1500000.0

Для розділення цілої та дробової частин дійсного числа використовується крапка, а не кома.

Під час математичних розрахунків у *Python* всі операції виконуються в певному порядку. Чим вище в табл. 1.1 розташована операція, тим вище її пріоритет. Тобто така операція виконується раніше, ніж операція, що розташована в таблиці нижче за неї. Математичні операції з однаковим пріоритетом виконуються послідовно, зліва направо, відповідно до їх розташування в виразі, що розраховується.

Таблиця 1.1 – Пріоритет виконання математичних операцій в *Python*

(...)	Операції в дужках
$f(\text{аргумент}, \dots)$	Виклик функції
**	Піднесення до степеня
*, /, //, %	Множення і ділення
+, -	Додавання і віднімання
< <= > >= !=, <> ==	Порівняння: менше менше або дорівнює більше більше або дорівнює не дорівнює дорівнює
<i>Not</i>	Логічне НЕ
<i>And</i>	Логічне І
<i>Or</i>	Логічне АБО

Для знаходження абсолютного значення числа використовується функція *abs()*. Якщо число є цілим або дійсним, функція *abs()* повертає його модуль, тобто невід'ємне значення. Модуль комплексного числа розраховується як квадратний корінь із суми квадратів його дійсної та уявної частин:

```
x = -12.5
y = 4 + 3j
print('|-12.5| =', abs(x))
print('|4 + 3j| =', abs(y))

|-12.5| = 12.5
|4 + 3j| = 5.0
```

Для округлення чисел і виведення результату на екран використовується команда *round()*:

```
round(number, ndigits)
```

де *number* – число, яке округлюється, а *ndigits* (необов'язковий параметр) – кількість значущих цифр після крапки, до яких здійснюється округлення. За замовчуванням *ndigits* дорівнює нулю:

```
print( round( 1.732 ) )
print( round( 1.732, 2 ) )

2
1.73
```

Список в *Python* – це впорядкована колекція (індексована послідовність) об'єктів довільних типів. Елементи списку перераховуються через кому і

записуються в квадратних дужках. Список може містити довільну кількість елементів довільних типів.

Для того, щоб звернутися до відповідного елемента списку, необхідно вказати номер цього елемента в квадратних дужках. Перший елемент списку має нульовий індекс:

```
list = [ 1, 2, 3, 4, 5 ]
print( list[ 0 ] )
print( list[ 2 ] )
print( list[ 4 ] )
```

1
3
5

Вирішення задач обчислювальної математики суттєво спрощуються використанням можливостей модулів *Python*, які призначені для математичних і наукових розрахунків.

Функції модуля *Math* (табл. 1.2) дозволяють виконати базові математичні обчислення [4]. Для роботи з матрицями та вирішення задач лінійної алгебри використовуються функції модуля *NumPy* [5]. Для реалізації числових методів вирішення науково-технічних задач призначений модуль *SciPy* [6]. Виконувати аналітичні розрахунки і символічні перетворення виразів дозволяють функції модуля *SymPy* [7]. Модуль *Matplotlib* призначений для візуалізації даних та побудови графіків функцій [8].

Таблиця 1.2 – Функції і сталі модулів *Math*, *NumPy* і *SymPy* для реалізації основних математичних операцій

Опис	Функція або стала модуля <i>Math</i>	Функція або стала модуля <i>NumPy</i>	Функція або стала модуля <i>SymPy</i>
Число $\pi = 3,141592\dots$	<i>pi</i>	<i>pi</i>	<i>pi</i>
Основа натурального логарифма $e = 2,718281\dots$	<i>e</i>	<i>e</i>	- <i>exp(1)</i>
Розрахунок $\sqrt{x}$	<i>sqrt(x)</i>	<i>sqrt(x)</i>	<i>sqrt(x)</i>
Розрахунок e^x	<i>exp(x)</i>	<i>exp(x)</i>	<i>exp(x)</i>
Розрахунок натурального логарифма $\ln(x) = \log_e(x)$	<i>log(x)</i>	<i>log(x)</i>	<i>log(x)</i> , <i>ln(x)</i>

Продовження табл. 1.2

Розрахунок логарифма $\log_a(x)$	$\log(x, a)$	$\frac{\log(x)}{\log(a)}$	$\log(x, a)$
Розрахунок двійкового логарифма $\log_2(x)$	$\log_2(x)$	$\log_2(x)$	$\log(x, 2)$
Розрахунок десятькового логарифма $\lg(x) = \log_{10}(x)$	$\log_{10}(x)$	$\log_{10}(x)$	$\log(x, 10)$
Розрахунок $\cos(x)$	$\cos(x)$	$\cos(x)$	$\cos(x)$
Розрахунок $\sin(x)$	$\sin(x)$	$\sin(x)$	$\sin(x)$
Розрахунок $\operatorname{tg}(x)$	$\tan(x)$	$\tan(x)$	$\tan(x)$
Розрахунок $\operatorname{ctg}(x)$	$\frac{1}{\tan(x)},$ $\frac{\cos(x)}{\sin(x)}$	$\frac{1}{\tan(x)},$ $\frac{\cos(x)}{\sin(x)}$	$\cot(x)$
Розрахунок $\arccos(x)$	$\operatorname{acos}(x)$	$\arccos(x)$	$\operatorname{acos}(x)$
Розрахунок $\arcsin(x)$	$\operatorname{asin}(x)$	$\arcsin(x)$	$\operatorname{asin}(x)$
Розрахунок $\operatorname{arctg}(x)$	$\operatorname{atan}(x)$	$\arctan(x)$	$\operatorname{atan}(x)$
Розрахунок $\operatorname{arcctg}(x)$	$\frac{\pi}{2} - \operatorname{atan}(x)$	$\frac{\pi}{2} - \arctan(x)$	$\operatorname{acot}(x);$ для від'ємних значень x : $\frac{\pi}{2} - \operatorname{atan}(x)$
Розрахунок $\sqrt{x^2 + y^2}$	$\operatorname{hypot}(x, y)$	$\operatorname{hypot}(x, y)$	$\sqrt{x^2 + y^2}$
Розрахунок $n!$	$\operatorname{factorial}(n)$	$\operatorname{math.factorial}(n)$	$\operatorname{factorial}(n)$

Імпорт модуля до програми здійснюється за допомогою ключового слова *import*. Для імені модуля можливо створити псевдонім за допомогою ключового слова *as*:

```
import math
import numpy as np
import sympy as smp
import scipy as scp
import matplotlib.pyplot as plt
```

Для використання в програмі функції імпортованого модуля необхідно вказати назву цієї функції і додати до неї префікс. Префікс складається з псевдоніма відповідного модуля (або імені модуля, якщо псевдонім не використовується) і крапки:

```
import math as m
print( m.sqrt( 16 ) )
print( m.pi )

4.0
3.141592653589793
```

Для імпорту до проекту не всіх функцій модуля разом, а лише певної функції, використовується ключове слово *from*. В цьому випадку для доступу до імпортованої функції використовується її назва без префіксу:

```
from math import pi

3.141592653589793
```

1.3 Умовні переходи

Умовний перехід – це конструкція мови програмування, яка забезпечує виконання (або не виконання) частини коду програми в залежності від значення певної логічної умови [9].

Логічна умова – це вираз, який може набувати одного з двох значень: *True* (істина) або *False* (хибність):

```
x = 10
y = 5
print( x > y )
print( x == y )

True
False
```

В *Python* умовні переходи реалізуються за допомогою умовного оператора, який має наступний загальний синтаксис:

```
if condition_1:
    state_1
elif condition_2:
    state_2
...
elif condition_N:
    state_N
else:
    state_else
```

де *condition_1*, ..., *condition_N* – логічні умови, *state_1*, ..., *state_N*, *state_else* – альтернативні блоки інструкцій.

Інструкції блоків *state_1*, ..., *state_N*, *state_else* є вкладеними інструкціями умовного оператора. Вкладені інструкції записуються з відступом в 4 пробіли.

Інтерпретатор *Python* виконує інструкції умовного оператора наступним чином [9]. Спочатку розраховується логічна умова *condition_1*. Якщо вона приймає значення *True*, то виконується блок інструкцій *state_1* і відбувається перехід до наступної після умовного оператора інструкції. Якщо *condition_1* приймає значення *False*, то розраховується значення умови *condition_2* і дії повторюються. Якщо жодна з умов *condition_1*, ..., *condition_N* не приймає значення *True*, то виконується блок інструкцій *state_else*:

```
x = 10
y = 10
if x > y:
    print( 'x > y' )
elif x < y:
    print( 'x < y' )
else:
    print( 'x == y' )

x == y
```

Наведена програма порівнює значення змінних *x* і *y*. Якщо значення *x* більше за значення *y*, командою *print()* на екран виводиться текстовий рядок '*x > y*'. Якщо значення *x* менше за значення *y*, то виводиться рядок '*x < y*'. У випадку, коли значення *x* дорівнює значенню *y*, на екран виводиться рядок '*x == y*'.

В умовному операторі блоки *elif* і *else* не є обов'язковими.

1.4 Циклічні програми

Цикл – структура мови програмування, що здійснює повторення певної послідовності інструкцій:

```
for iterator in collection:
    process_iteration
```

де *iterator* – це ітератор, який послідовно приймає значення всіх елементів колекції *collection*; *process_iteration* – тіло циклу, що виконується для кожного поточного значення змінної *iterator* з колекції.

Колекція – це структура, що містить скінченний набір елементів, до яких реалізовано доступ. Колекціями, наприклад, можуть бути списки (*list*), кортежі (*tuple*), множини (*set*), словники (*dict*) [10].

Приклад циклічної програми:

```
num = [ 1, 2, 3, 4, 5 ]
for x in num:
    print( x )
```

1
2
3
4
5

В цьому прикладі команда *print()* виводить на екран кожне значення, яке приймає змінна *x* зі списку *num*.

Для створення числових колекцій можливо використовувати функції модуля *NumPy*:

- *arange()* – створює в заданому інтервалі послідовність чисел з визначеним кроком;

- *linspace()* – створює в заданому інтервалі послідовність визначеної кількості рівномірно розподілених чисел.

Синтаксис функції *arange()*:

numpy.arange (start = 0, stop, step = 1)

де *start* – початок інтервалу. Колекція включає це значення. Значення за замовчуванням дорівнює 0;

stop – кінець інтервалу. Є обов'язковим параметром функції *arange()*. Значення *stop* не включається до колекції.

step – крок між значеннями колекції. Розмір кроку за замовчуванням дорівнює 1.

Синтаксис функції *linspace()*:

numpy.linspace (start, stop, num = 50)

де *start* – початок інтервалу. Обов'язковий параметр. Колекція включає це значення;

stop – кінець інтервалу. Обов'язковий параметр. Колекція включає це значення;

num – це кількість значень, які створюються в інтервалі між значеннями *start* та *stop*. За замовчуванням дорівнює 50. Має бути невід'ємним цілим числом.

Приклади створення колекцій з використанням функцій *linspace()* та *arange()*:

```
import numpy as np
x = np.arange(1, 5, 1)
y = np.linspace(1, 10, 5)
print('Колекція, яка створена з використанням команди arange:', x)
print('Колекція, яка створена з використанням команди linspace:', y)
```

Колекція, яка створена з використанням команди *arange*: [1 2 3 4]

Колекція, яка створена з використанням команди *linspace*: [1. 3.25 5.5 7.75 10.]

В цьому прикладі функція *arange()* створює колекцію цілих чисел від 1 до 5 (верхня межа інтервалу не входить до колекції) з кроком 1. Функція *linspace()* створює колекцію з п'яти рівномірно розподілених чисел між 1 і 10 включно.

Для переривання виконання циклу або ігнорування частини тіла циклу в *Python* існують команди *break* та *continue*.

Оператор *break* перериває виконання циклу і здійснює перехід до наступної після циклу інструкції. Оператор *continue* перериває виконання тіла циклу, після чого ітератор приймає наступне значення з колекції і цикл продовжує роботу:

```
for num in range( 1, 10 ):
    if num == 6:
        break
    if num < 3:
        continue
    print( num )
```

3
4
5

Змінна *num* послідовно приймає значення з колекції цілих чисел від 1 до 9. Якщо поточне значення *num* менше за 3, то за командою *continue* відбувається перехід до наступної ітерації циклу і команда *print()* не виконується. Робота циклу і всієї програми припиняється командою *break*, коли *num* приймає значення 6. Значення *num*, які належать інтервалу від 3 до 5 включно, виводяться на екран командою *print()*.

1.5 Завдання до лабораторної роботи

Завдання 1.1

Розрахуйте значення невідомих величин:

1	$a = \frac{2 \cos\left(x - \frac{\pi}{6}\right) b}{1/2 + \sin^2(y)};$	$m = 1 + \frac{z^2}{3 + y^2/5};$
	якщо $x = 1,45; y = -1,22; z = 3,5; b = 3,07$.	
2	$s = 1 + y + \frac{x^2}{2!} + \frac{x^3}{3!} + \frac{y^4}{5!};$	$z = \frac{\sin(x^3) + \cos^2(y)}{\lg(\sqrt{x^2 + y})};$
	якщо $x = 1,2; y = -0,8$.	
3	$y = e^{-bt} \sin(at + b) - \sqrt{bt + a};$	$s = b \cdot \sin(at^2 \cos(2t)) - 1;$
	якщо $a = -0,5; b = 1,7; t = 0,44$.	
4	$P = \sqrt{x^2 + b} - b^2 \sin^3(x + a) / (x \cdot y);$	
	якщо $y = \cos^2(x^3) - \frac{x}{\sqrt{a^2 + b^2}}; a = 1,5; b = 15,5; x = -2,9$.	
5	$S = \frac{x^3 \operatorname{tg}^2(x + b)^2 + a}{\sqrt{x + c}} \cdot c;$	
	якщо $c = \frac{bx^2 - a}{e^{a \cdot x} - 1}; a = 16,5; b = 3,4; x = 0,61$.	
6	$T = \left x^{y/x} - \sqrt[3]{y/x} \right ;$	$f = (x - y) \frac{y - e^{z/(y-x)}}{1 + (y - x)^2};$
	якщо $x = 1,82; y = 18,5; z = -3,4$.	
7	$R = x^2(x + 1) / b - \sin^2(x - a);$	$S = \sqrt{xb/a} + \left \cos(x + b)^3 \right ;$
	якщо $a = 0,7; b = 0,05; x = 0,43$.	
8	$a = \frac{\sqrt{ x-1 } - \sqrt{y}}{1 + x^2/2! + y^2/4!};$	$b = x(\operatorname{arctg}(z + e^{x+3}));$
	якщо $x = 2,21; y = 0,49; z = 0,09$.	
9	$p = a / b;$	
	якщо $a = (1 - y) \frac{(x + y)^2 / (x + 4)^3}{e^{-(x-2)} + (x^3 + 4)}; b = \frac{1 + \cos(y - 2)}{x^{4/3} + \sin^2(y - 2)};$ $x = 1,25; y = 0,93$.	

10	$P = \left \frac{\sin^3(ax^3 + by^2 - ab)}{\sqrt[3]{(ax^3 + by^2 - a)^2 + \pi}} \right + \operatorname{tg}(ax^3 + by^2 - ab);$	
	якщо $x = 0,25; y = 1,31; a = 3,5; b = 0,9.$	
11	$R = x^3(x+1)/b^2 - \sin^2(x(x+a));$	$S = \sqrt{xb/a} + \cos^2(x+b)^3;$
	якщо $a = 0,7; b = 0,05; x = 0,5.$	
12	$y = \sin^3(x^2 + a)^2 - \sqrt[3]{x/b};$	$z = x^2/a + \cos^2(x+b)^3;$
	якщо $a = 1,1; b = 0,004; x = 0,2.$	
13	$\varphi = \sqrt{m \cdot \operatorname{tg}(x) + c \cdot \sin(x) };$	$\xi = m \cdot \cos(bx + \sin(x)) + c^3;$
	якщо $m = 2; c = -1; x = 1,2; b = 0,7.$	
14	$P = \frac{1 + \sin^2(x+1)}{2 + \left x - 2x^3 / (1 + x^2 y^3) \right } + x^4;$	$Q = \cos^2 \left(\arctg \left(\frac{1}{z} \right) \right);$
	якщо $x = 0,25; y = 0,79; z = 0,81.$	
15	$Y = b^3 \operatorname{tg}^2(x) - \frac{a}{\sin^2(x/a)};$	$Z = ae^{-\sqrt{3}} \cos(bx/a);$
	якщо $a = 3,2; b = 0,5; x = -4,8.$	
16	$K = \ln(a + x^3) + \sin^2(x/b);$	$M = e^{-c \cdot x} \cdot \frac{x + \sqrt[3]{x+a}}{x - \sqrt{ x-b }};$
	якщо $a = 10,2; b = 9,3; x = 2,4; c = 0,5.$	
17	$Y = \frac{a^{2x} + b^{-x} \cdot \cos((a+b)x)}{ x-1 };$	$R = \sqrt{x^2 + b} - b^2 \sin^3((x+a)/x);$
	якщо $a = 0,3; b = 0,9; x = 0,53.$	
18	$\lambda = \sqrt{ ax^2 \sin(2x) + e^{-2x}(x+b) };$	$\omega = 1 / \cos^2(x^3) - x / \sqrt[3]{a^2 + b^2};$
	якщо $a = 0,5; b = 3,1; x = 1,4.$	
19	$U = \frac{a^3 + e^{-x} \cos(bx)}{bx - e^{-x} \sin(bx+1)};$	$F = e^{2x} \ln(a+x) - b^{3x} \ln x-b ;$
	якщо $a = 0,5; b = 2,9; x = 0,3.$	
20	$\gamma = \frac{a^x + b^{-x} \sin(a-b)}{\sqrt{ a-b }};$	$\beta = a \cdot e^{-\sqrt{a}} \cos(bx/a);$
	якщо $a = 1,78; b = 3,06; x = 0,55.$	
21	$Z = \frac{\sin(x)}{\sqrt{1 + m^3 \sin^2(x)}} - m \cdot x \cdot \ln(m \cdot x);$	$S = e^{-ax} \sqrt{x+1} + e^{-bx} \sqrt{ x-3 };$
	якщо $m = 0,7; x = 1,77; a = 0,5; b = 1,08.$	

22	$P = \sqrt[3]{m \cdot \operatorname{tg}(t) + c \cdot \sin(t) };$	$K = \cos^2(t) - m / \sqrt[3]{ m^2 - t^2 };$
	якщо $m = 2,1; t = 1,02; c = -1.$	
23	$a = y + \frac{x}{y^3 + \left x^2 / \left(y + \sqrt[3]{x^2} \right) \right };$	$b = (1 + \operatorname{tg}^2(x/2)) \cdot \ln(x+1);$
	якщо $x = 1,23; y = 0,79; z = 0,9.$	
24	$P = \frac{3 + e^{y-1}}{1 + x^2 y - \operatorname{tg}(z) };$	$S = 1 + y - \frac{(y-x)^2}{2} + \frac{ y-x }{3};$
	якщо $x = 10,3; y = 4,93; z = 0,4.$	
25	$A = \frac{z \cdot \cos(x - \pi/6)}{1/2 + \sin^3(y)};$	$B = 1 + \frac{z^3 + y}{3 + z^2/5};$
	якщо $x = 1,2; y = 0,9; z = 3,5.$	
26	$A = \ln \left y - \sqrt{ x } \right \cdot \left(x - \frac{y}{z + x^2/4} \right);$	$B = x - \frac{y^2}{2!} + \frac{z^3}{3!};$
	якщо $x = -5,3; y = 2,8; z = 0,39.$	
27	$P = \left \frac{\sin^3 cx^2 + by^2 }{\sqrt[3]{ cx^3 + by^2 }} \right + \operatorname{tg}(cx^3 + by^2);$	$S = \cos^2(x^3) - x \sqrt{(x-b)^2};$
	якщо $c = 0,5; b = -0,5; x = 0,61; y = 1,2.$	
28	$a = \frac{2 \cos \left(x - \frac{\pi}{3} \right) b + 7,2}{5/2 + \sin^3(y)};$	$b = 2,1 + \frac{(z+y)^2}{3 + \sqrt[3]{y^4/5}};$
	якщо $x = 2,23; y = -0,94; z = 1,8; b = -2,67.$	
29	$s = 1 + y + \frac{x^3}{2!} + \frac{x^2}{4!} + \frac{y^x}{6!};$	$z = \frac{\sin(xy^3) + \cos^2(y^2)}{3y+1};$
	якщо $x = 5,31; y = -1,04.$	
30	$y = e^{-bt} \sin(a\sqrt{t} + b) + \sqrt{ bt + a };$	$s = b + \sin(at^2 \operatorname{tg}(2t^3)) / 1,5;$
	якщо $a = -3,63; b = -0,72; t = 1,28.$	

Завдання 1.2

Розрахуйте значення функції при зміні значень аргументу в заданому діапазоні і з заданим кроком h_x .

1	$y = \begin{cases} ax + b \cdot \cos(x), & x < 0,5, \\ bx^2 + c \cdot \sin(2x), & 0,5 \leq x \leq 1; \end{cases}$
	якщо $x \in [0; 1]; h_x = 0,1; a = 0,75; b = 1,19; c = -2,5$.
2	$z = \begin{cases} a^x + b^y, & x + y \leq 1, \\ ax^2 + \ln(b \cdot x \cdot y), & x + y > 1; \end{cases}$
	якщо $x \in [0; 2]; h_x = 0,1; a = 1; y = 0,5; b = 2$.
3	$y = \begin{cases} \sin ax + b , & x < a/b, \\ \cos ax - b , & x \geq a/b; \end{cases}$
	якщо $x \in [-2,5; 2]; h_x = 0,5; a = 5; b = 3$.
4	$y = \begin{cases} \frac{\sin(x + 0,4)^2}{4x^2}, & x \leq 3, \\ x^4 + 2x^2 \cdot \cos(x), & x > 3; \end{cases}$
	якщо $x \in [-5; 5]; h_x = 1$.
5	$z = \begin{cases} \frac{x^2}{15}, & x^2 + y^2 \leq a^2, \\ x^2 + e^x, & x^2 + y^2 > a^2; \end{cases}$
	якщо $x \in [-3; 3]; h_x = 0,5; a = 5,3; y = 4,85$.
6	$y = \begin{cases} \sin ax + b^a , & x < b, \\ \cos ax - b^a , & x \geq b; \end{cases}$
	якщо $x \in [0,5; 3,3]; h_x = 0,2; a = -1,1; b = 1,5$.
7	$y = \begin{cases} a^{b-x} + c, & x < 2, \\ b^{c-x} + a, & 2 \leq x \leq 4, \\ c^{a-x} + b, & x > 4; \end{cases}$
	якщо $x \in [0; 5]; h_x = 0,25; a = 2; b = 3; c = 7$.

8	$y = \begin{cases} \sin \left \frac{a^b}{x} + \sqrt{b} \right , & x < b, \\ \cos \left \frac{a^b}{x} - \sqrt{b} \right , & x \geq b; \end{cases}$
	якщо $x \in [1; 3]; h_x = 0,1; a = 1,5; b = 2.$
9	$z = \begin{cases} \frac{x^2}{(x-5)^3}, & x > y, \\ \frac{(x-2)^3}{y(x-5)^4}, & x \leq y; \end{cases}$
	якщо $x \in [1; 10]; h_x = 1; y = 5,7.$
10	$z = \begin{cases} \ln \sqrt{ax^2 + by^3}, & xy > 0, \\ a^x + b^y, & xy \leq 0; \end{cases}$
	якщо $x \in [-1; 1]; h_x = 0,1; a = 1,7; b = 2,3; y = 0,4.$
11	$y = \begin{cases} \cos(ax + 2), & x > 2, \\ \operatorname{tg} x - 2a , & x \leq 2; \end{cases}$
	якщо $x \in [0,5; 3,1]; h_x = 0,2;$ a має початкове значення 0,1 і змінюється одночасно з x з кроком $h_a = 0,3.$
12	$y = \begin{cases} a^2 + x, & a > x, \\ a^2, & a = x, \\ a^2 - x, & a < x; \end{cases}$
	якщо $x \in [1; 8]; h_x = 0,5; a = 5.$
13	$y = \begin{cases} \frac{\lg(x^2 + 0,75)}{4x^2 + x^2}, & x \leq 5, \\ x^4 + 2x^2 \sin(x), & x > 5; \end{cases}$
	якщо $x \in [-10; 10]; h_x = 2.$
14	$z = \begin{cases} x^2 + y^2, & x^2 + y^2 \leq 1, \\ x^2 + y, & x^2 + y^2 > 1 \text{ i } y \geq x, \\ \lg(x^2 + y^2), & x^2 + y^2 > 1 \text{ i } y < x; \end{cases}$

	якщо $x \in [0,1; 1,3]; h_x = 0,2; y = 0,9$.
15	$y = \begin{cases} 1 + a\sqrt{1+x^2}, & x < 0, \\ 0, & x = 0, \\ 1 - a\sqrt{1+x^2}, & x > 0; \end{cases}$
	якщо $x \in [-10; 10]; h_x = 2; a = 4$.
16	$y = \begin{cases} \lg(A \cdot B \cdot C), & A < 0, \\ A \cdot B \cdot C, & 0 < A < 2, \\ 1, & \text{в інших випадках;} \end{cases}$
	якщо $A \in [-4; 3]; h_A = 0,25; B = -2; C = 4$.
17	$y = \begin{cases} \frac{a \ln(x)}{x} + d \cdot \cos(x), & x \leq 3,5, \\ \sqrt[3]{2a^5 c + \sin(x)} / x, & x > 3,5; \end{cases}$
	якщо $x \in [2; 4,5]; h_x = 0,5; a = 1,42; d = 5,4; c = 3,15$.
18	$y = \begin{cases} \sqrt{a + bx^2}, & x \leq 0, \\ a \cdot \ln(x^4) / x + d \cdot \cos(x), & x > 0; \end{cases}$
	якщо $x \in [-2; 2]; h_x = 0,25; a = 1; b = 2; d = -1$.
19	$y = \begin{cases} \sin\left(\frac{a}{x} + 4\right), & x > 2, \\ \operatorname{tg} x + 2a , & x \leq 2; \end{cases}$
	якщо $x \in [0,1; 2,7]; h_x = 0,2;$ a має початкове значення 0,4 і змінюється одночасно з x з кроком $h_a = 0,2$.
20	$y = \begin{cases} bx + c \cdot \cos(x), & x < 0,5 \text{ і } x > 1, \\ cx^3 + a \cdot \sin(2x), & 0,5 \leq x \leq 1; \end{cases}$
	якщо $x \in [0; 2]; h_x = 0,2; a = 0,55; b = 1,28; c = -2,2$.
21	$z = \begin{cases} b^x + a^y, & x + y \leq 1, \\ ax^2 + \log_2(b \cdot x \cdot y), & x + y > 1; \end{cases}$
	якщо $x \in [0; 2]; h_x = 0,1; a = 1,25; b = 2; y = 0,6$.

22	$z = \begin{cases} x^2 + y^2, & x^2 + y^2 \leq 1, \\ x + y^2, & x^2 + y^2 > 1 \text{ i } y < x, \\ \ln(x^2 + y^3), & x^2 + y^2 > 1 \text{ i } y \geq x; \end{cases}$
	якщо $x \in [0,1; 1,5]; h_x = 0,2; y = 0,8.$
23	$z = \begin{cases} \frac{x^2}{x+8}, & x^2 + y^2 \leq a^2, \\ x^2 + e^{2x}, & x^2 + y^2 > a^2; \end{cases}$
	якщо $x \in [-2; 4]; h_x = 0,5; a = 5,2; y = 4,75.$
24	$y = \begin{cases} \sin \left \frac{a^b}{x} + \sqrt[3]{b} \right , & x < \sqrt{b}, \\ \cos \left \frac{a^b}{x} - \sqrt[3]{b} \right , & x \geq \sqrt{b}; \end{cases}$
	якщо $x \in [1; 3]; h_x = 0,1; a = 2,4; b = 2.$
25	$z = \begin{cases} x^2 / (x-6)^2, & x > y, \\ (x-3)^3 / (y + (x-2,2)^4), & x \leq y; \end{cases}$
	якщо $x \in [1; 10]; h_x = 1; y = 5,7.$
26	$z = \begin{cases} 2 \ln \sqrt{a / x^2 + by^3}, & xy > 0, \\ -a^y + b^x, & xy \leq 0; \end{cases}$
	якщо $x \in [-1; 1]; h_x = 0,1; a = 2,6; b = 3,9; y = 0,4.$
27	$y = \begin{cases} 3 \sin^2(ax + 2), & x > 2, \\ -0,5 \cdot \operatorname{tg} x + 2a , & x \leq 2; \end{cases}$
	якщо $x \in [0,5; 3,1]; h_x = 0,2;$ a має початкове значення 0,2 і змінюється одночасно з x з кроком $h_a = 0,3.$
28	$y = \begin{cases} 2a^2 + \sqrt{x}, & a > x, \\ 2a, & a = x, \\ 2a^3 - \sqrt{x}, & a < x; \end{cases}$
	якщо $x \in [1; 9]; h_x = 0,5; a = 5.$

29	$y = \begin{cases} \frac{\lg(x^2 + 0,25)}{2x^2 - x^2}, & x \leq 5, \\ x^3 + 3x^2 \sin(x), & x > 5; \end{cases}$
	якщо $x \in [-10; 10]; h_x = 2$.
30	$y = \begin{cases} \log_2(A + B + C), & A > 0, \\ A + B + C, & -3 < A < 0, \\ -1, & \text{в інших випадках;} \end{cases}$
	якщо $A \in [-5; 2]; h_A = 0,5; B = -2,4; C = 3$.

1.6 Приклади розв'язання завдань

Завдання 1.1

Розрахуйте значення невідомих величин:

$$A = \frac{\cos(x^3 - \pi / 2)}{1 + \log_2(y - 2)} + \sqrt[3]{y - 2};$$

$$B = \frac{a^{2x} + y^{-x} \sin(a + x) \cdot x}{e^{|x-1|}};$$

якщо $x = 0,92; y = 5,3; a = 0,25$.

Розв'язання

```
import math
x = 0.92
y = 5.3
a = 0.25
A = math.cos(x ** 3 - math.pi / 2) / (1 + math.log(y - 2, 2)) + (y - 2) ** (1 / 3)
print('A =', round(A, 3))
B = (a ** (2 * x) + y ** (-x) * math.sin(a + x) * x) / math.exp(abs(x - 1))
print('B =', round(B, 3))
```

A = 1.747

B = 0.241

Відповідь: $A = 1,747; B = 0,241$.

Завдання 1.2

Розрахуйте значення функції при зміні значень аргументу в заданому діапазоні і з заданим кроком h_x .

$$y(x) = \begin{cases} m^{p-x} + n & x < 2; \\ n^{m-x} + p, & 2 \leq x \leq 4, \\ p^{m-x+n} + n, & x > 4; \end{cases}$$

якщо $x \in [-1,2; 6,8]$; $h_x = 0,8$; $m = 1,5$; $n = 2,5$; $p = 7,2$.

Розв'язання

```
import numpy as np
a = -1.2
b = 6.8
m = 1.5
n = 2.5
p = 7.2
hx = 0.8
for x in np.arange(a, b + hx, hx):
    if x < 2:
        y = m ** (p - x) + n
    elif x <= 4:
        y = n ** (m - x) + p
    else:
        y = p ** (m - x + n) + n
    print('y( ', round(x, 2), ' ) = ', round(y, 3))

y( -1.2 ) = 32.642
y( -0.4 ) = 24.292
y( 0.4 ) = 18.255
y( 1.2 ) = 13.891
y( 2.0 ) = 7.832
y( 2.8 ) = 7.504
y( 3.6 ) = 7.346
y( 4.4 ) = 2.954
y( 5.2 ) = 2.594
y( 6.0 ) = 2.519
y( 6.8 ) = 2.504
```

Відповідь:

x	-1,2	-0,4	0,4	1,2	2,0	2,8	3,6	4,4	5,2	6,0	6,8
y	32,642	24,292	18,255	13,891	7,832	7,504	7,346	2,954	2,594	2,519	2,504

Контрольні питання

1. Для чого використовується операція присвоєння в *Python*?
2. Яким чином в *JupyterLab* можливо вивести на екран значення змінної?
3. Які існують числові типи даних в *Python*?
4. Які є вимоги до імен змінних в *Python*?
5. Яке призначення мають цикли при вирішенні обчислювальних задач у *Python*?
6. Яким чином можливо імпортувати до проекту окрему функцію модуля *Python*?
7. Яким символом розпочинається однорядковий коментар в *Python*?
8. Яку мінімальну кількість логічних умов і альтернатив може містити умовний оператор *Python*?
9. Яким чином можливо розрахувати в *Python* абсолютне значення числа?
10. Який індекс має перший елемент списку *Python*?
11. Який колір в *JupyterLab* мають текстові рядки?
12. Яким чином можливо записати декілька інструкцій *Python* в одному рядку?

Лабораторна робота № 2

ОЦІНКА ПОХИБОК ОБЧИСЛЕНЬ.

БАЗОВІ ОПЕРАЦІЇ НАД МАТРИЦЯМИ

Мета роботи: одержання практичних навичок оцінки похибок обчислень і виконання основних операцій над матрицями.

2.1 Правила округлення чисел

Округленням числа a є заміна його числом b із меншою кількістю значущих цифр [11].

При округленні числа до n -ї значущої цифри відкидають усі його цифри праворуч, починаючи з $(n+1)$ -ї. Зазначене відкидання цифр здійснюється за наступними правилами округлення:

- 1) якщо $(n+1)$ -а цифра більша за 5, то до n -ї цифри додається одиниця;
- 2) якщо $(n+1)$ -а цифра менша за 5, то всі цифри, що залишилися, зберігаються без зміни;
- 3) якщо $(n+1)$ -а цифра дорівнює 5 і серед інших цифр, що відкидаються, є ненульові, то до n -ї цифри додається одиниця;
- 4) якщо $(n+1)$ -а цифра дорівнює 5, а всі інші цифри, що відкидаються, дорівнюють нулю, то n -на зберігається без зміни, якщо вона парна, і до n -ї додається одиниця, якщо вона непарна.

2.2 Класифікація похибок обчислень

Особливістю використання числових методів для вирішення математичної задачі є те, що знайдений числовий розв'язок є наближеним і містить деяку похибку [12].

Припустимо, що a – наближене значення деякої точної величини A .

Абсолютною похибкою Δ числа є модуль різниці між його точним і наближеним значеннями:

$$\Delta_a = |A - a|. \quad (2.1)$$

Тоді

$$a - \Delta_a \leq A \leq a + \Delta_a; \quad (2.2)$$

$$A = a \pm \Delta_a. \quad (2.3)$$

Відносною похибкою δ наближеного числа є відношення абсолютної похибки Δ цього числа до його модуля ($a \neq 0$):

$$\delta_{a,\%} = \frac{\Delta_a}{|a|} \cdot 100. \quad (2.4)$$

Задача 2.1

Визначте, яка рівність точніше: $\frac{9}{11} = 0,818$ або $\sqrt{18} = 4,24$.

Розв'язання

Позначимо наближені значення першої і другої рівностей a і b відповідно: $a = 0,818$; $b = 4,24$.

Обчислимо точні значення заданих виразів. Для цього залишимо в результатах розрахунків більшу кількість значущих цифр:

$$A = \frac{9}{11} = 0,81818,$$

$$B = \sqrt{18} = 4,2426.$$

Абсолютні похибки округлення:

$$\Delta_a = |A - a| = |0,81818 - 0,818| = 0,00018,$$

$$\Delta_b = |B - b| = |4,2426 - 4,24| = 0,0026.$$

Відносні похибки округлення:

$$\delta_{a,\%} = \frac{\Delta_a}{|a|} \cdot 100 = \frac{0,00018}{0,818} \cdot 100 = 0,00022 \cdot 100 = 0,022\%,$$

$$\delta_{b,\%} = \frac{\Delta_b}{|b|} \cdot 100 = \frac{0,0026}{4,24} \cdot 100 = 0,00061 \cdot 100 = 0,061\%.$$

Відповідь: оскільки $0,022\% < 0,061\%$, то рівність $\frac{9}{11} = 0,818$ точніше.

2.3 Розрахунок похибок функцій від декількох наближених аргументів

Припустимо, що відома функція $y = f(x_1, x_2, \dots, x_n)$ від n аргументів, а значення кожного з аргументів x_i визначено з деякими абсолютними похибками Δ_{xi} , $i = 1, 2, \dots, n$.

Тоді абсолютна похибка функції у:

$$\Delta_y = \sum_{i=1}^n \left| \frac{df}{dx_i} \right| \cdot \Delta_{x_i}. \quad (2.5)$$

Відносна похибка функції у:

$$\delta_y = \sum_{i=1}^n \left| \frac{d}{dx_i} \ln(y) \right| \cdot \Delta_{x_i}. \quad (2.6)$$

За (2.5) і (2.6) одержано формули для розрахунків відносних похибок деяких найпростіших функцій наближених аргументів (табл. 2.1).

Таблиця 2.1 – Відносні похибки функцій наближених аргументів

Функція	Відносна похибка
$y = A \pm B$	$\delta_y = \frac{\Delta A + \Delta B}{A \pm B}$
$y = A \cdot B$	$\delta_y = \delta_A + \delta_B = \frac{\Delta A}{A} + \frac{\Delta B}{B}$
$y = \frac{A}{B}$	$\delta_y = \delta_A + \delta_B = \frac{\Delta A}{A} + \frac{\Delta B}{B}$
$y = A^n$	$\delta_y = n \cdot \delta_A = \frac{n \cdot \Delta A}{A}$
$y = \sqrt[n]{A}$	$\delta_y = \frac{1}{n} \cdot \delta_A = \frac{\Delta A}{n \cdot A}$
$y = \frac{1}{A} \pm \frac{1}{B}$	$\delta_y = \frac{\frac{\Delta A}{A^2} + \frac{\Delta B}{B^2}}{\frac{1}{A} \pm \frac{1}{B}}$

Задача 2.2

Обчисліть значення виразу та похибку результату:

$$x = \frac{m^2 n^3}{\sqrt{k}},$$

де $m = 2,3 \pm 0,002$, $n = 1,45 \pm 0,001$, $k = 0,678 \pm 0,0003$.

Розв'язання

Наближене значення x :

$$x = \frac{2,3^2 \cdot 1,45^3}{\sqrt{0,678}} = 19,586.$$

Відносні похибки чисел m , n , k :

$$\delta_{m,\%} = \frac{0,002}{2,3} \cdot 100 = 0,087\%,$$

$$\delta_{n,\%} = \frac{0,001}{1,45} \cdot 100 = 0,069\%,$$

$$\delta_{k,\%} = \frac{0,0003}{0,678} \cdot 100 = 0,044\%.$$

Відносна похибка добутку та частки наближених чисел дорівнює сумі відносних похибок даних чисел (табл. 2.1). Тоді відносна похибка результату:

$$\delta_{x,\%} = 2\delta_{m,\%} + 3\delta_{n,\%} + 0,5\delta_{k,\%};$$

$$\delta_{x,\%} = 2 \cdot 0,087 + 3 \cdot 0,069 + 0,5 \cdot 0,044 = 0,403\%.$$

Абсолютна похибка результату:

$$\Delta_x = |x| \cdot \delta_{x,\%} / 100 = 19,59 \cdot 0,403 / 100 = 0,079.$$

Відповідь: Значення виразу: $x = 19,586 \pm 0,079$.

Відносна похибка результату: $\delta_{x,\%} = 0,403\%$.

Задача 2.3

Обчисліть значення виразу та похибку результату:

$$x = \frac{(n-1)(m+n)}{(m-n)^2},$$

де $m = 5,72 \pm 0,02$; $n = 3,0567 \pm 0,0001$.

Розв'язання

Абсолютна похибка суми та різниці наближених чисел дорівнює сумі абсолютних похибок даних чисел (табл. 2.1):

$$n - 1 = (3,0567 - 1) \pm 0,0001 = 2,0567 \pm 0,0001;$$

$$m + n = (5,72 + 3,0567) \pm (0,02 + 0,0001) = 8,7767 \pm 0,0201;$$

$$m - n = (5,72 - 3,0567) \pm (0,02 + 0,0001) = 2,6633 \pm 0,0201.$$

Наближене значення x :

$$x = \frac{2,0567 \cdot 8,7767}{2,6633^2} = 2,545.$$

Відносна похибка результату:

$$\delta_{x,\%} = \delta_{n-1,\%} + \delta_{m+n,\%} + 2\delta_{m-n,\%};$$

$$\delta_{x,\%} = \left(\frac{0,0001}{2,0567} + \frac{0,0201}{8,7767} + 2 \cdot \frac{0,0201}{2,6633} \right) \cdot 100 = 1,74\%.$$

Абсолютна похибка результату:

$$\Delta_x = |x| \cdot \delta_{x,\%} / 100 = 2,545 \cdot 1,74 / 100 = 0,044.$$

Відповідь: Значення виразу: $x = 2,545 \pm 0,044$.

Відносна похибка результату: $\delta_x = 1,74\%$.

2.4 Основи матричних розрахунків у *Python*

Матрицею розміру m на n є таблиця, яка складається з m рядків і n стовпців:

$$A = \begin{bmatrix} a_{11} & a_{12} & \dots & a_{1n} \\ a_{21} & a_{22} & \dots & a_{2n} \\ \dots & \dots & \dots & \dots \\ a_{m1} & a_{m2} & \dots & a_{mn} \end{bmatrix} = [a_{ij}]. \quad (2.7)$$

Перший індекс i елемента матриці вказує номер рядка, в якому він розташований, другий j – номер стовпця.

Матриця є квадратною, якщо число її рядків співпадає з числом стовпців, $m = n$.

Елементами матриці можуть бути як числа (числова матриця), так і функції (функціональна матриця) [13].

Матричні розрахунки в *Python* зручно виконувати з використанням функцій модуля *NumPy* [5]. Так, для створення матриці можливо скористатись функцією *array()*.

При створенні матриці, яка складається з одного рядка, параметром функції `array()` є числовий список:

```
import numpy as np
A = np.array( [1, 2, 3] )
print(A)
```

[1 2 3]

При створенні багатовимірної матриці (в тому числі матриці, що складається з одного стовпця) параметром функції `array()` є вкладений список, в якому кожний внутрішній список задає відповідний рядок матриці:

```
import numpy as np
A = np.array( [[1, 2, 3], [4, 5, 6], [7, 8, 9]] )
print(A)
```

[[1 2 3]
[4 5 6]
[7 8 9]]

Якщо в комірках матриці, яка створюється, ввести математичні вирази, то відповідні елементи матриці будуть розраховані за цими виразами:

```
import numpy as np
A = np.array( [[1 + 2, np.sin(np.pi/2)], [5 ** 2, 3 * 4]] )
print(A)
```

[[3. 1.]
[25. 12.]]

Транспонованою матрицею A^T для матриці A розміру m на n є матриця розміру n х m , яка одержується з матриці A заміною її рядків стовпцями:

$$A^T = [a_{ji}] = \begin{bmatrix} a_{11} & a_{21} & \dots & a_{m1} \\ a_{12} & a_{22} & \dots & a_{m2} \\ \dots & \dots & \dots & \dots \\ a_{1n} & a_{2n} & \dots & a_{mn} \end{bmatrix}. \quad (2.8)$$

Транспоновану матрицю можливо знайти з використанням функції `transpose()` модуля *NumPy* [5].

Визначник (детермінант) квадратної матриці – скалярна величина, яка характеризує деякі властивості матриці, такі як оберненість, лінійну залежність рядків або стовпців, зміну об'єму при лінійному перетворенні.

Визначник квадратної матриці A позначається як $\det(A)$ або $|A|$.

Визначник дорівнює нулю, якщо:

- хоча б один із рядків (або стовпців) матриці містить тільки нульові елементи;
- хоча б два рядки (або стовпці) матриці пропорційні або однакові;
- хоча б один рядок (або стовець) визначається як сума або різниця інших рядків (стовпців).

Визначник можливо розрахувати з використанням функції *det()* підмодуля *linalg* модуля *NumPy* [5].

Операції додавання і віднімання матриць, додавання числа до матриці, множення матриці на число, а також поелементні множення і ділення матриць здійснюються аналогічно до відповідних операцій над скалярами з використанням тих самих операторів.

Добуток матриці A розміру m_1 на n_1 на матрицю B розміру m_2 на n_2 визначений, якщо $n_1 = m_2$, тобто коли число стовпців матриці A дорівнює кількості рядків матриці B . Результатом є матриця C розміру m_1 на n_2 , елементи якої визначаються за формулою

$$c_{ij} = \sum_{k=1}^{n_1=m_2} a_{ik} b_{kj} \quad (i=1,2,\dots,m_1; j=1,2,\dots,n_2). \quad (2.9)$$

Операція множення матриць не володіє властивістю комутативності: $AB \neq BA$.

Для розрахунку добутку матриць використовується функція *dot()* модуля *NumPy*.

Для піднесення квадратної матриці до степеня використовується функція *matrix_power()* підмодуля *linalg* модуля *NumPy*.

Квадратна матриця є одиничною, якщо її елементи, які розташовані на головній діагоналі, дорівнюють одиниці, а решта елементів – нулі:

$$E = \begin{bmatrix} 1 & 0 & 0 & \dots & 0 & 0 \\ 0 & 1 & 0 & \dots & 0 & 0 \\ \dots & \dots & \dots & \dots & \dots & \dots \\ 0 & 0 & 0 & \dots & 0 & 1 \end{bmatrix}. \quad (2.10)$$

Добуток одиничної матриці на довільну квадратну матрицю A ліворуч або праворуч не змінює матрицю A : $AE = EA = A$.

Оберненою матрицею A^{-1} до квадратної матриці A є матриця того самого розміру, для якої справедливі співвідношення:

$$AA^{-1} = A^{-1}A = E. \quad (2.11)$$

Обернена матриця A^{-1} існує тоді і тільки тоді, коли визначник квадратної матриці A не дорівнює нулю.

Обернену матрицю можливо знайти з використанням функції *inv()* підмодуля *linalg* модуля *NumPy*.

Частка двох квадратних матриць A і B (за умови, що матриця B має обернену матрицю B^{-1}) розраховується наступним чином:

$$A / B = AB^{-1}. \quad (2.12)$$

Якщо у виразі, який обчислюється, аргументом є матриця, то обчислення виконуються для кожного її елемента. В результаті одержується матриця того ж розміру, що й вихідна матриця-аргумент:

```
import numpy as np
A = np.array([[4, 9], [25, 36]])
B = np.sqrt(A) + 10
print(B)

[[12. 13.]
 [15. 16.]]
```

2.5 Завдання до лабораторної роботи

Завдання 2.1

Визначте, яка рівність точніша.

1	$\sqrt{44} = 6,63;$	$\frac{19}{41} = 0,463.$	2	$\sqrt{83} = 9,11;$	$\frac{6}{11} = 0,545.$
3	$\sqrt{10,5} = 3,24;$	$\frac{4}{17} = 0,235.$	4	$\frac{17}{19} = 0,895;$	$\sqrt{52} = 7,21.$
5	$\frac{7}{15} = 0,467;$	$\sqrt{30} = 5,48.$	6	$\sqrt{47,5} = 6,89;$	$\frac{21}{29} = 0,724.$
7	$\frac{15}{7} = 2,14;$	$\sqrt{10} = 3,16.$	8	$\frac{50}{19} = 2,63;$	$\sqrt{27} = 5,19.$
9	$\frac{6}{7} = 0,857;$	$\sqrt{4,8} = 2,19.$	10	$\frac{7}{22} = 0,318;$	$\sqrt{13} = 3,6.$
11	$\frac{12}{11} = 1,091;$	$\sqrt{6,8} = 2,61.$	12	$\frac{13}{17} = 0,764;$	$\sqrt{31} = 5,56.$
13	$\frac{2}{21} = 0,095;$	$\sqrt{22} = 4,69.$	14	$\frac{17}{11} = 1,545;$	$\sqrt{18} = 4,24.$
15	$\frac{23}{15} = 1,53;$	$\sqrt{9,8} = 3,13.$	16	$\sqrt{29,7} = 5,45;$	$\frac{26}{7} = 3,714.$
17	$\sqrt{83} = 9,11;$	$\frac{6}{11} = 0,545.$	18	$\sqrt{44} = 6,63;$	$\frac{19}{41} = 0,463.$
19	$\frac{17}{19} = 0,895;$	$\sqrt{52} = 7,21.$	20	$\sqrt{10,5} = 3,24;$	$\frac{4}{17} = 0,235.$
21	$\sqrt{47,5} = 6,89;$	$\frac{21}{29} = 0,724.$	22	$\frac{7}{15} = 0,467;$	$\sqrt{30} = 5,48.$
23	$\frac{50}{19} = 2,63;$	$\sqrt{27} = 5,19.$	24	$\frac{15}{7} = 2,14;$	$\sqrt{10} = 3,16.$
25	$\frac{7}{22} = 0,318;$	$\sqrt{13} = 3,6.$	26	$\frac{6}{7} = 0,857;$	$\sqrt{4,8} = 2,19.$
27	$\frac{13}{17} = 0,764;$	$\sqrt{31} = 5,56.$	28	$\frac{12}{11} = 1,091;$	$\sqrt{6,8} = 2,61.$
29	$\frac{17}{11} = 1,545;$	$\sqrt{18} = 4,24.$	30	$\frac{2}{21} = 0,095;$	$\sqrt{22} = 4,69.$

Завдання 2.2

Обчисліть значення виразу та похибку результату.

1	$X = \frac{ab}{\sqrt[3]{c}};$ якщо $a = 3,85 \pm 0,01;$ $b = 2,0435 \pm 0,0004;$ $c = 962,6 \pm 0,1.$
2	$X = \frac{\sqrt{a} \cdot b}{c};$ якщо $a = 228,6 \pm 0,06;$ $b = 86,4 \pm 0,02;$ $c = 68,7 \pm 0,05.$
3	$X = \frac{\sqrt{ab}}{c};$ якщо $a = 3,845 \pm 0,004;$ $b = 16,2 \pm 0,05;$ $c = 10,8 \pm 0,1.$
4	$X = \frac{a^2 b}{c};$ якщо $a = 3,456 \pm 0,002;$ $b = 0,642 \pm 0,0005;$ $c = 7,12 \pm 0,004.$
5	$X = \frac{ab^3}{c};$ якщо $a = 0,643 \pm 0,0005;$ $b = 2,17 \pm 0,002;$ $c = 5,843 \pm 0,001.$
6	$X = \frac{ab}{c^2};$ якщо $a = 0,3575 \pm 0,0002;$ $b = 2,63 \pm 0,01;$ $c = 0,854 \pm 0,0005.$
7	$V = a^2 \frac{m}{d^3};$ якщо $a = 3,64 \pm 0,0002;$ $d = 8,235 \pm 0,001;$ $m = 54 \pm 0,5.$
8	$Y = \frac{m^3 \sqrt{n}}{c^2};$ якщо $m = 1,653 \pm 0,003;$ $n = 3,78 \pm 0,002;$ $c = 0,158 \pm 0,002.$
9	$X = \sqrt{\frac{cd}{b}};$ якщо $c = 0,7568 \pm 0,0002;$ $b = 21,7 \pm 0,02;$ $d = 2,65 \pm 0,01.$
10	$F = \frac{qr^2}{\sqrt{b}};$ якщо $q = 54,8 \pm 0,02;$ $r = 2,45 \pm 0,01;$ $b = 0,863 \pm 0,004.$
11	$Y = \frac{ab^2}{\sqrt[3]{c}};$ якщо $a = 4,16 \pm 0,005;$ $b = 12,163 \pm 0,002;$ $c = 55,18 \pm 0,01.$
12	$X = \frac{a \cdot \sqrt{b}}{c};$ якщо $a = 315,6 \pm 0,05;$ $b = 72,5 \pm 0,03;$ $c = 53,8 \pm 0,04.$
13	$X = \frac{\sqrt{ab}}{c};$ якщо $a = 4,632 \pm 0,003;$ $b = 23,3 \pm 0,04;$ $c = 11,3 \pm 0,06.$
14	$X = \sqrt[3]{\frac{a}{c}} \cdot b;$ якщо $a = 1,245 \pm 0,001;$ $b = 0,121 \pm 0,0002;$ $c = 2,34 \pm 0,003.$

15	$X = \left(\frac{a}{c}\right)^2 \cdot b$; якщо $a = 0,142 \pm 0,0003$; $b = 1,71 \pm 0,002$; $c = 3,727 \pm 0,001$.
16	$X = \frac{ab}{\sqrt[3]{c}}$; якщо $a = 3,85 \pm 0,01$; $b = 2,0435 \pm 0,0004$; $c = 962,6 \pm 0,1$.
17	$X = \frac{\sqrt{a} \cdot b}{c}$; якщо $a = 228,6 \pm 0,06$; $b = 86,4 \pm 0,02$; $c = 68,7 \pm 0,05$.
18	$X = \frac{\sqrt{ab}}{c}$; якщо $a = 3,845 \pm 0,004$; $b = 16,2 \pm 0,05$; $c = 10,8 \pm 0,1$.
19	$X = \frac{a^2b}{c}$; якщо $a = 3,456 \pm 0,002$; $b = 0,642 \pm 0,0005$; $c = 7,12 \pm 0,004$.
20	$X = \frac{ab^3}{c}$; якщо $a = 0,643 \pm 0,0005$; $b = 2,17 \pm 0,002$; $c = 5,843 \pm 0,001$.
21	$X = \frac{ab}{c^2}$; якщо $a = 0,3575 \pm 0,0002$; $b = 2,63 \pm 0,01$; $c = 0,854 \pm 0,0005$.
22	$V = a^2 \frac{m}{d^3}$; якщо $a = 3,64 \pm 0,0002$; $d = 8,235 \pm 0,001$; $m = 54 \pm 0,5$.
23	$Y = \frac{m^3 \sqrt{n}}{c^2}$; якщо $m = 1,653 \pm 0,003$; $n = 3,78 \pm 0,002$; $c = 0,158 \pm 0,002$.
24	$X = \sqrt{\frac{cd}{b}}$; якщо $c = 0,7568 \pm 0,0002$; $b = 21,7 \pm 0,02$; $d = 2,65 \pm 0,01$.
25	$F = \frac{qr^2}{\sqrt{b}}$; якщо $q = 54,8 \pm 0,02$; $r = 2,45 \pm 0,01$; $b = 0,863 \pm 0,004$.
26	$Y = \frac{ab^2}{\sqrt[3]{c}}$; якщо $a = 4,16 \pm 0,005$; $b = 12,163 \pm 0,002$; $c = 55,18 \pm 0,01$.
27	$X = \frac{a \cdot \sqrt{b}}{c}$; якщо $a = 315,6 \pm 0,05$; $b = 72,5 \pm 0,03$; $c = 53,8 \pm 0,04$.
28	$X = \frac{\sqrt{ab}}{c}$; якщо $a = 4,632 \pm 0,003$; $b = 23,3 \pm 0,04$; $c = 11,3 \pm 0,06$.
29	$X = \sqrt[3]{\frac{a}{c}} \cdot b$; якщо $a = 1,245 \pm 0,001$; $b = 0,121 \pm 0,0002$; $c = 2,34 \pm 0,003$.
30	$X = \left(\frac{a}{c}\right)^2 \cdot b$; якщо $a = 0,142 \pm 0,0003$; $b = 1,71 \pm 0,002$; $c = 3,727 \pm 0,001$.

Завдання 2.3

Сформуйте матриці A і B , якщо $a = 0,01 \cdot k$; k – номер варіанту.

$$A = \begin{vmatrix} 1-a & 2-a & 3-a & 4-a \\ 5-a & 6-a & 7-a & -8-a \\ 8-a & 7-a & 6-a & -5-a \\ 4-a & -3-a & -2-a & 1-a \end{vmatrix}; \quad B = \begin{vmatrix} 1-a \\ 2-a \\ 3-a \\ 4-a \end{vmatrix}.$$

Знайдіть:

- визначник $|A|$;
- обернену матрицю A^{-1} ;
виконайте перевірку правильності розрахунку оберненої матриці A^{-1} ;
- транспоновану матрицю A^T ;
- результат піднесення матриці до другого і третього степенів: A^2 і A^3 ;
- суму матриці і числа: $A + x$, де x – довільне дійсне число;
- добуток матриці і числа: $A \cdot x$;
- суму матриць: $A + B$;
- добуток матриць: $A \cdot B$.

Завдання 2.4

Сформуйте нову матрицю B , кожний елемент b_{ij} якої є результатом розрахунку функції f для відповідного елемента a_{ij} матриці A .

№	Функція $f(a)$	Матриця A
1	$f(a) = \sqrt{a} + \cos(a) + 0,2a$;	$\begin{bmatrix} 0,54 & 1,98 & 4,23 \\ 2,5 & 3 & 5,41 \end{bmatrix}$
2	$f(a) = \cos(a) + \sin(2a) + 0,4a$;	$\begin{bmatrix} 7,62 & -8 & -2,01 \\ -6,4 & 4,85 & 0,48 \end{bmatrix}$
3	$f(a) = e^{-0,64a} - \frac{\cos(1,48a)}{2,5a}$;	$\begin{bmatrix} 1,73 & 0,1 & 7,3 \\ -6 & -9,36 & 8,44 \end{bmatrix}$
4	$f(a) = -0,5\sqrt{a} + \cos(1,8a) + 1,1a$;	$\begin{bmatrix} 9,6 & 3,67 & 7 \\ 8,08 & 0,3 & 4,56 \end{bmatrix}$
5	$f(a) = a^5 e^{-a} \sin(0,8a) + 1,3a$;	$\begin{bmatrix} -5 & 9,74 & 0,19 \\ 6,64 & -4,6 & 2,81 \end{bmatrix}$

6	$f(a) = e^{-\cos(a)} + 1,5a;$	$\begin{bmatrix} 9,7 & 2,6 & -1,18 \\ -5,64 & 6 & 7,03 \end{bmatrix}$
7	$f(a) = \ln(a) + 1,1\sin(a) + 1,6a;$	$\begin{bmatrix} 0,66 & 5,1 & 2 \\ 1,45 & 8,38 & 7,3 \end{bmatrix}$
8	$f(a) = \frac{13,8\ln(a^2)}{0,8\sin(a) + 2,7a};$	$\begin{bmatrix} 3,29 & 9,4 & 5 \\ 0,98 & 1,2 & 7,05 \end{bmatrix}$
9	$f(a) = \frac{1,23}{a} + \cos(1,44a) + 2,1a;$	$\begin{bmatrix} 2 & 4,51 & -8,39 \\ -3,29 & 1,9 & 7,06 \end{bmatrix}$
10	$f(a) = \sqrt{a} + \sin(a) + 0,1a;$	$\begin{bmatrix} 0,1 & 2,96 & 5 \\ 1,51 & 3,75 & 6,46 \end{bmatrix}$
11	$f(a) = e^{-0,1a} \cos(a) + 0,8a;$	$\begin{bmatrix} -1,95 & 9,3 & -6,09 \\ 4 & 8,23 & 0,89 \end{bmatrix}$
12	$f(a) = \sin(2a) + \cos(a^2) - \frac{8}{3a}.$	$\begin{bmatrix} 5,58 & -4,12 & 9,1 \\ -6,1 & -7,23 & 8 \end{bmatrix}$
13	$f(a) = a^2 \cos(2a) + 0,9a;$	$\begin{bmatrix} 5,8 & 8,47 & 4,2 \\ -1,22 & 2 & -7,66 \end{bmatrix}$
14	$f(a) = e^{-\sin(a) - \cos(2a)} + 2,5a;$	$\begin{bmatrix} 0,34 & 8,48 & -2 \\ 4,7 & 9,83 & -3,35 \end{bmatrix}$
15	$f(a) = \ln(0,86a) \cos(3,75a) + 1,9a;$	$\begin{bmatrix} 5,3 & 6,08 & 0,4 \\ 3,69 & 4 & 8,9 \end{bmatrix}$
16	$f(a) = \sin(a) + \cos(2a) + 0,3a;$	$\begin{bmatrix} 0,88 & -3,8 & 5,1 \\ 2 & 4,2 & -6,05 \end{bmatrix}$
17	$f(a) = \frac{8,4}{a^{\cos(a)} + \sin(1,2a) + 1,6} + 2,8a;$	$\begin{bmatrix} 2,43 & 0,12 & 4,21 \\ 8,7 & 7 & 5,97 \end{bmatrix}$
18	$f(a) = e^{-\sin(a)} + 1,4a;$	$\begin{bmatrix} 3,62 & -6,29 & 5,98 \\ 8 & -1,7 & 0,97 \end{bmatrix}$
19	$f(a) = \cos(a) + 0,5\sin(2a) + 0,6a;$	$\begin{bmatrix} -7 & -4,75 & 1,4 \\ -0,43 & 5,8 & 6,42 \end{bmatrix}$

20	$f(a) = \frac{a^2 \cos(2a)}{a + 4,7} + 2,2a;$	$\begin{bmatrix} 8,7 & 1,89 & -6,73 \\ -9,91 & 4,2 & 0,44 \end{bmatrix}$
21	$f(a) = a^2 - e^{-0,2a} \cos(5,2a) - 2,4a;$	$\begin{bmatrix} 8,6 & -3,2 & -4,03 \\ 9,95 & 2 & 7,59 \end{bmatrix}$
22	$f(a) = \sin(a^2) + 0,5 \cos(2a) + \frac{0,5a}{4}.$	$\begin{bmatrix} -8,48 & 4 & 3,05 \\ 2,7 & 5,91 & -7,6 \end{bmatrix}$
23	$f(a) = \frac{2,5}{a} + \sin(2,56a) + 2a;$	$\begin{bmatrix} 3,6 & 1 & 7,95 \\ 2,48 & -5,6 & -6,06 \end{bmatrix}$
24	$f(a) = \sqrt{a^2 + 0,2a} + \cos(3a);$	$\begin{bmatrix} 3,2 & 4,01 & 5,89 \\ 9 & 6,8 & 2,1 \end{bmatrix}$
25	$f(a) = (\sin(1,2a) + 2,9a)e^{-0,18a};$	$\begin{bmatrix} 4,78 & 1,4 & -3 \\ 9,3 & -5,27 & -2,02 \end{bmatrix}$
26	$f(a) = \ln(2a) + 3,2 \cos(a) + 1,7a;$	$\begin{bmatrix} 1,3 & 3 & 6,74 \\ 9,4 & 5,7 & 2,15 \end{bmatrix}$
27	$f(a) = \sin(a) + 0,5 \cos(2a) + 0,5a;$	$\begin{bmatrix} 5,57 & -3,23 & 9 \\ -0,5 & -8,3 & 1,17 \end{bmatrix}$
28	$f(a) = \sqrt{a} + \sin(3a) + a;$	$\begin{bmatrix} 5,9 & 2,05 & 8,76 \\ 6,62 & 7,2 & 3 \end{bmatrix}$
29	$f(a) = 5,18e^{-0,3a} \sin(a) + 0,7a;$	$\begin{bmatrix} 6,9 & 2,68 & -4,02 \\ 1,86 & -0,4 & 5 \end{bmatrix}$
30	$f(a) = \ln(a) \sin(2a) + 1,8a;$	$\begin{bmatrix} 7,16 & 6,98 & 2,04 \\ 8 & 5,9 & 1,4 \end{bmatrix}$

2.6 Приклади розв'язання завдань

Завдання 2.1

Визначте, яка рівність точніше: $\frac{9}{11}=0,818$ чи $\sqrt{18} = 4,24$.

Розв'язання

```
import math
a = 0.818
b = 4.24
A = 9 / 11
print( "Точне значення першої рівності: A =", A )
B = math.sqrt( 18 )
print( "Точне значення другої рівності: B =", B )
Δa = abs( A - a ) # Абсолютна похибка першої рівності
Δb = abs( B - b ) # Абсолютна похибка другої рівності
δa = Δa / abs(a) * 100 # Відносна похибка першої рівності
print( "Відносна похибка першої рівності: δa =", round( δa, 5 ), "%" )
δb = Δb / abs(b) * 100 # Відносна похибка другої рівності
print( "Відносна похибка другої рівності: δb =", round( δb, 5 ), "%" )
if ( δa < δb ):
    print( "Перша рівність точніше" )
else:
    print( "Друга рівність точніше" )
```

```
Точне значення першої рівності: A = 0.8181818181818182
Точне значення другої рівності: B = 4.242640687119285
Відносна похибка першої рівності: δa = 0.02223 %
Відносна похибка другої рівності: δb = 0.06228 %
Перша рівність точніше
```

Відповідь: оскільки $0,022\% < 0,062\%$, то рівність $\frac{9}{11}=0,818$ точніше.

Завдання 2.2

Обчисліть значення виразу та похибку результату:

$$x = \frac{m^2 n^3}{\sqrt{k}},$$

де $m = 2,3 \pm 0,002$, $n = 1,45 \pm 0,001$, $k = 0,678 \pm 0,0003$.

Розв'язання

```
import math
m = 2.3; n = 1.45; k = 0.678 # Наближені значення чисел m, n, k
Δm = 0.002; Δn = 0.001; Δk = 0.0003 # Абсолютні похибки чисел m, n, k
x = (m ** 2) * (n ** 3) / math.sqrt(k) # Наближене значення x
δm = Δm / abs(m) * 100 # Відносна похибка числа m
δn = Δn / abs(n) * 100 # Відносна похибка числа n
δk = Δk / abs(k) * 100 # Відносна похибка числа k
δx = 2 * δm + 3 * δn + 0.5 * δk # Відносна похибка результату
print( "Відносна похибка результату:", round( δx, 3 ), "%" )
Δx = abs(x) * δx / 100 # Абсолютна похибка результату
print( "x =", round( x, 3 ), "+-", round( Δx, 3 ) )
```

Відносна похибка результату: 0.403 %

$x = 19.586 \pm 0.079$

Відповідь: Значення виразу: $x = 19,586 \pm 0,079$.

Відносна похибка результату: $\delta_x = 0,403 \%$.

Завдання 2.3

Сформууйте матриці A і B , якщо $a = 0,01 \cdot k$; $k = 40$

$$A = \begin{vmatrix} 1-a & 2-a & 3-a & 4-a \\ 5-a & 6-a & 7-a & -8-a \\ 8-a & 7-a & 6-a & -5-a \\ 4-a & -3-a & -2-a & 1-a \end{vmatrix}; \quad B = \begin{vmatrix} 1-a \\ 2-a \\ 3-a \\ 4-a \end{vmatrix}.$$

Знайдіть:

- визначник $|A|$;
- обернену матрицю A^{-1} ;
виконайте перевірку правильності розрахунку оберненої матриці A^{-1} ;
- транспоновану матрицю A^T ;
- результат піднесення матриці до другого і третього степенів: A^2 і A^3 ;
- суму матриці і числа: $A + x$, де x – довільне дійсне число;
- добуток матриці і числа: $A \cdot x$;
- суму матриць: $A + B$;
- добуток матриць: $A \cdot B$.

Розв'язання

Задаємо вихідні дані і створюємо матрицю A :

```
import numpy as np
import math
k = 40
a = 0.01 * k
A = np.array([[1-a, 2-a, 3-a, 4-a],
              [5-a, 6-a, 7-a, -8-a],
              [8-a, 7-a, 6-a, -5-a],
              [4-a, -3-a, -2-a, 1-a]])
print("A =")
print(A)

A =
[[ 0.6  1.6  2.6  3.6]
 [ 4.6  5.6  6.6 -8.4]
 [ 7.6  6.6  5.6 -5.4]
 [ 3.6 -3.4 -2.4  0.6]]
```

Створюємо матрицю B :

```
B = np.array([[1-a],
              [2-a],
              [3-a],
              [4-a]])
print("B =")
print(B)

B =
[[0.6]
 [1.6]
 [2.6]
 [3.6]]
```

Розраховуємо визначник $|A|$:

```
print("|A| =", np.round(np.linalg.det(A), 1))

|A| = 729.6
```

Розраховуємо обернену матрицю A^{-1} і виконуємо перевірку розрахунку:

```
print("A ** (-1) =")
print(np.linalg.inv(A))
print()
print("Перевірка правильності розрахунку оберненої матриці:")
print("A * A ** (-1) =")
print(np.dot(A, np.linalg.inv(A)))

A ** (-1) =
[[ 1.97368421e-02 -4.60526316e-02  9.86842105e-02  1.25000000e-01]
 [-1.64473684e-01 -2.82894737e-01  3.02631579e-01 -2.50000000e-01]
 [ 2.96052632e-01  3.09210526e-01 -2.69736842e-01  1.25000000e-01]
 [ 1.33771930e-01 -8.99122807e-02  4.38596491e-02 -3.71291691e-18]]

Перевірка правильності розрахунку оберненої матриці:
A * A ** (-1) =
[[ 1.00000000e+00  1.66533454e-16 -2.49800181e-16  4.21446503e-17]
 [ 0.00000000e+00  1.00000000e+00  2.22044605e-16  3.11885021e-17]
 [ 0.00000000e+00 -1.66533454e-16  1.00000000e+00  2.00497513e-17]
 [-1.94289029e-16 -1.04083409e-16 -4.30211422e-16  1.00000000e+00]]
```

В результаті множення заданої матриці A на її обернену матрицю A^{-1} одержали одиничну матрицю, що підтверджує правильність розрахунку. Елементи головної діагоналі одиничної матриці дорівнюють одиниці. Значення інших елементів або дорівнюють нулю, або наближаються до нього.

Розраховуємо транспоновану матрицю A^T :

```
print("AT =")
print(np.transpose(A))

AT =
[[ 0.6  4.6  7.6  3.6]
 [ 1.6  5.6  6.6 -3.4]
 [ 2.6  6.6  5.6 -2.4]
 [ 3.6 -8.4 -5.4  0.6]]
```

Розраховуємо A^2 і A^3 :

```
print("A ** 2 =")
print(np.linalg.matrix_power(A, 2))
print()
print("A ** 3 =")
print(np.linalg.matrix_power(A, 3))

A ** 2 =
[[ 40.44  14.84  18.04 -23.16]
 [ 48.44 110.84 106.04 -71.16]
 [ 58.04 104.44 107.64 -61.56]
 [-29.56 -31.16 -27.96  54.84]]

A ** 3 =
[[ 146.256  345.616  359.696 -90.384]
 [ 1088.656 1640.016 1622.096 -1371.984]
 [ 1111.696 1597.456 1590.736 -1286.544]
 [-176.144 -592.784 -570.704  339.216]]
```

Розраховуємо суму матриці і числа $A + x$, де x – довільне дійсне число:

```
x = 2.5 # Довільне число
print("A + x =")
print(A + x)

A + x =
[[ 3.1  4.1  5.1  6.1]
 [ 7.1  8.1  9.1 -5.9]
 [10.1  9.1  8.1 -2.9]
 [ 6.1 -0.9  0.1  3.1]]
```

Розраховуємо добуток матриці і числа $A \cdot x$:

```
print("A * x =")
print(A * x)

A * x =
[[ 1.5  4.  6.5  9. ]
 [ 11.5 14. 16.5 -21. ]
 [ 19.  16.5 14. -13.5]
 [ 9.  -8.5 -6.  1.5]]
```

Розраховуємо суму матриць $A + B$:

```
print("A + B =")  
print(A + B)
```

```
A + B =  
[[ 1.2  2.2  3.2  4.2]  
 [ 6.2  7.2  8.2 -6.8]  
 [10.2  9.2  8.2 -2.8]  
 [ 7.2  0.2  1.2  4.2]]
```

Розраховуємо добуток матриць $A \cdot B$:

```
print("A * B =")  
print(np.dot(A, B))
```

```
A * B =  
[[22.64]  
 [-1.36]  
 [10.24]  
 [-7.36]]
```

Завдання 2.4

Сформууйте нову матрицю B , кожний елемент b_{ij} якої є результатом розрахунку функції f для відповідного елемента a_{ij} матриці A :

$$f(a) = e^{-\sin(0,25a)} \cos(6a) - \sqrt{2a} \cdot \ln\left(\frac{a}{1,6}\right);$$

$$A = \begin{bmatrix} 0,1 & 6,23 & 2,7 \\ 1,64 & 4,8 & 0,52 \end{bmatrix}.$$

Розв'язання

```
import numpy as np
# Створюємо матрицю A:
A = np.array([[0.1, 6.23, 2.7],
              [1.64, 4.8, 0.52]])
# Розраховуємо елементи матриці B:
B = np.exp(-np.sin(0.25 * A)) * np.cos(6 * A) - np.sqrt(2 * A) * np.log(A / 1.6)
print("B =")
print(np.round(B, 2))
```

```
B =
[[ 2.04 -4.45 -1.69]
 [-0.66 -3.74  0.27]]
```

Відповідь: $B = \begin{bmatrix} 2,04 & -4,45 & -1,69 \\ -0,66 & -3,74 & 0,27 \end{bmatrix}.$

Контрольні питання

1. В чому полягає операція округлення числа?
2. Яке число є наближеним?
3. Якими похибками характеризується наближене число?
4. Яким чином розраховується абсолютна похибка наближеного числа?
5. В яких одиницях вимірюється відносна похибка наближеного числа?
6. Яким чином визначаються похибки функції, якщо її аргументи є наближеними числами?
7. В чому полягає операція транспонування матриці?
8. Яким чином розраховується визначник матриці?
9. Скільки обернених матриць має квадратна матриця, якщо її визначник не дорівнює нулю?
10. В чому відмінність числової матриці від функціональної?
11. Яка матриця є квадратною?
12. Як за допомогою функції `array()` модуля *NumPy* створити матрицю, яка складається з одного стовпця?

Лабораторна робота № 3

ПОБУДОВА ЛІНІЙНИХ ГРАФІКІВ ФУНКЦІЙ. РОЗВ'ЯЗАННЯ АЛГЕБРАЇЧНИХ І ТРАНСЦЕНДЕНТНИХ РІВНЯНЬ

Мета роботи: одержання практичних навичок розв'язання алгебраїчних, трансцендентних рівнянь і побудови графіків функцій.

3.1 Графічне представлення функцій, рівнянь та нерівностей у *Python*

Числовою функцією є залежність між двома множинами чисел, яка кожному набору чисел з першої множини (область визначення функції) ставить у відповідність єдине число з другої множини (область значень функції).

Кожний елемент області визначення функції має назву незалежної змінної або аргументу функції. Значення функції приймає залежна змінна.

Залежність, яка встановлює взаємозв'язок між значеннями аргументів і функції, може бути задана аналітично за допомогою формул. Іншим способом представлення функції однієї або двох змінних є її графічне зображення.

Графіком функції однієї змінної є крива на площині. В прямокутній (декартовій) системі координат абсциса довільної точки цієї кривої відповідає значенню аргументу функції, а ордината – значенню функції для цього аргументу.

В *Python* для візуалізації даних у вигляді графіків використовують функції модуля *Matplotlib*, зокрема його підмодуля *Pypplot* [8]. Так, побудувати лінійний графік функції однієї змінної можливо за допомогою функції *plot()*:

```
matplotlib.pyplot.plot(x, y, color = 'auto', ls = '-', marker = None, lw = 1.5,  
                        label = None)
```

де x – колекція (список, *NumPy*-матриця тощо), яка містить значення аргументу в точках, за якими будується графік;

y – колекція (список, *NumPy*-матриця тощо), яка містить значення функції в точках, за якими будується графік.

Кількість елементів в колекції x повинна відповідати кількості елементів в колекції y . Якщо параметр x не задано, то в якості абсцис точок приймаються індекси елементів колекції y : $0, 1, 2, \dots, n - 1$, де n – кількість елементів в y ;

$color, c$ – колір кривої (табл. 3.1). Якщо значення даного параметру не задано, то кольори кривим призначаються по черзі відповідно до переліку, який наведено в табл. 3.1. Позначення для чорного кольору – 'k';

ls – стиль кривої (табл. 3.2);

$marker$ – форма точок, за якими побудовано криву (табл. 3.3). Колір маркерів визначає параметр *markerfacecolor*, розмір – *markersize*;

lw – товщина кривої;

label – назва кривої. Якщо до графіка додано легенду, то в ній буде відображатись дана назва.

Таблиця 3.1 – Значення параметру *color* функції *matplotlib.pyplot.plot()* та відповідні кольори кривих

Значення параметру <i>color</i>	Колір кривої
'b'	Синій
'orange'	Помаранчевий
'g'	Зелений
'r'	Червоний
'purple'	Фіолетовий
'brown'	Коричневий
'pink'	Рожевий
'gray'	Сірий
'y'	Жовто-зелений
'c'	Бірюзовий

Таблиця 3.2 – Значення параметру *ls* функції *matplotlib.pyplot.plot()* та відповідні стилі кривих

Значення параметру <i>ls</i>	Стиль кривої
'-'	суцільна лінія
'--'	штрихова лінія
'-.'	штрихпунктирна лінія
'::'	пунктирна лінія

Таблиця 3.3 – Значення параметру *marker* функції *matplotlib.pyplot.plot()* та відповідні форми маркерів

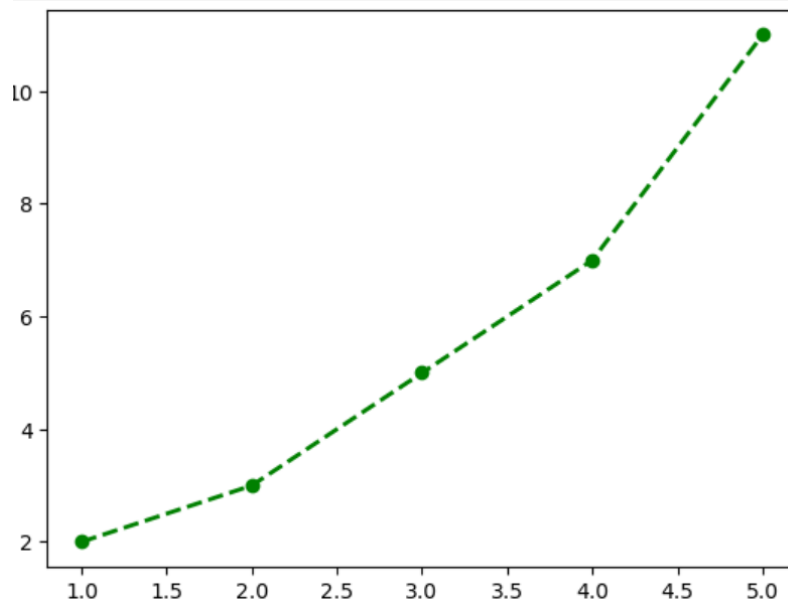
Значення параметру <i>marker</i>	Форма маркера	Значення параметру <i>marker</i>	Форма маркера
'.'	точка	'p'	п'ятикутний маркер
'o'	зафарбоване коло	'*'	зірка

Продовження табл. 3.3

'v'	трикутний маркер	'h'	шестикутний маркер
'g'	восьмикутний маркер	'+'	плюс
's'	квадрат	'x'	хрест

Приклад побудови лінійного графіка функції однієї змінної:

```
import matplotlib.pyplot as plt
x = [1, 2, 3, 4, 5]
y = [2, 3, 5, 7, 11]
plt.plot(x, y, c = 'g', ls = '--', marker = 'o', lw = 2)
```



Додати назву графіка можливо з використанням функції *title()*.

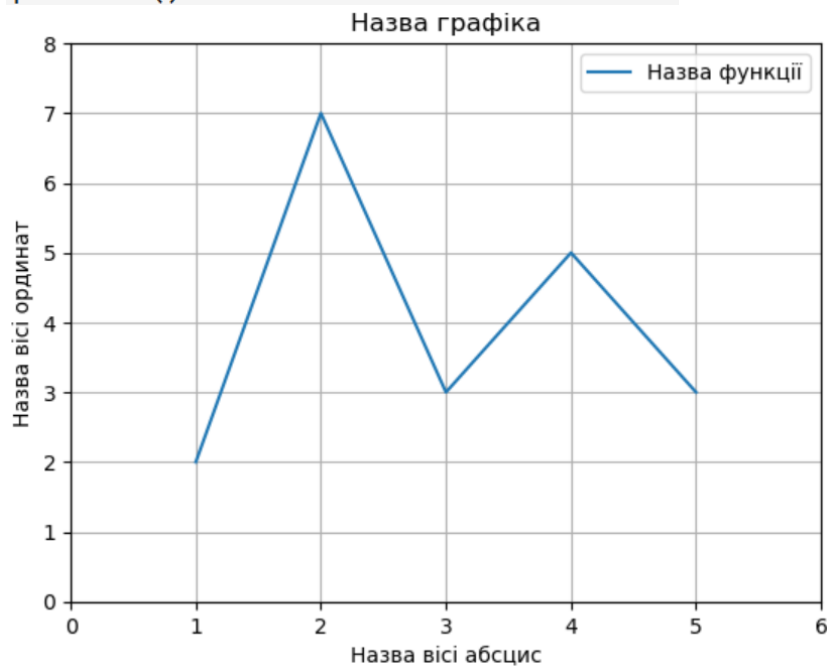
Для підпису осей графіка використовуються функції *xlabel()* і *ylabel()* відповідно.

Сітка додається на графік функцією *grid()* з параметром *True*, легенда – функцією *legend()*.

Діапазони побудови графіка за осями абсцис і ординат задаються функціями *xlim()* і *ylim()* відповідно.

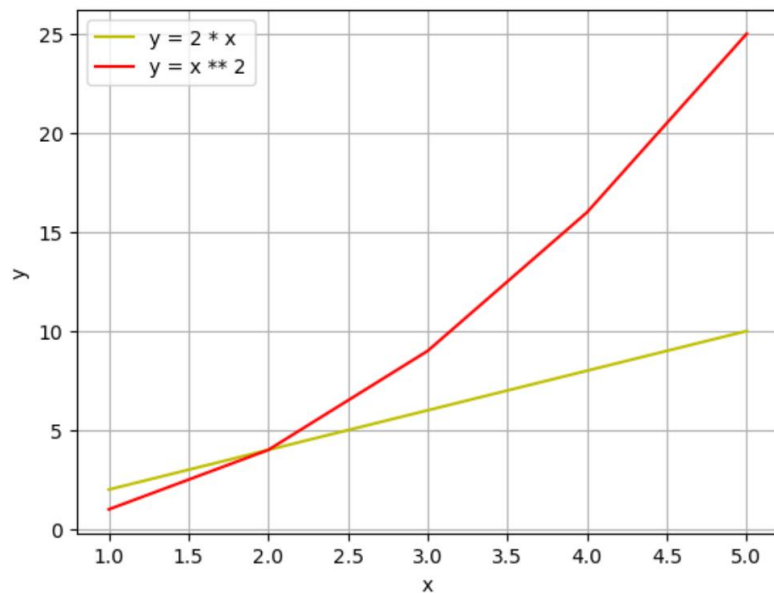
В *JupyterLab* графіки відображаються автоматично після використання функції *plot()*. Для ручного відображення графіка на екрані в потрібний момент використовується функція *show()*:

```
import matplotlib.pyplot as plt
x = [1, 2, 3, 4, 5]
y = [2, 7, 3, 5, 3]
plt.plot(x, y, label = 'Назва функції')
plt.title('Назва графіка')
plt.xlabel('Назва осі абсцис')
plt.ylabel('Назва осі ординат')
plt.grid(True)
plt.legend()
plt.xlim(0, 6)
plt.ylim(0, 8)
plt.show()
```



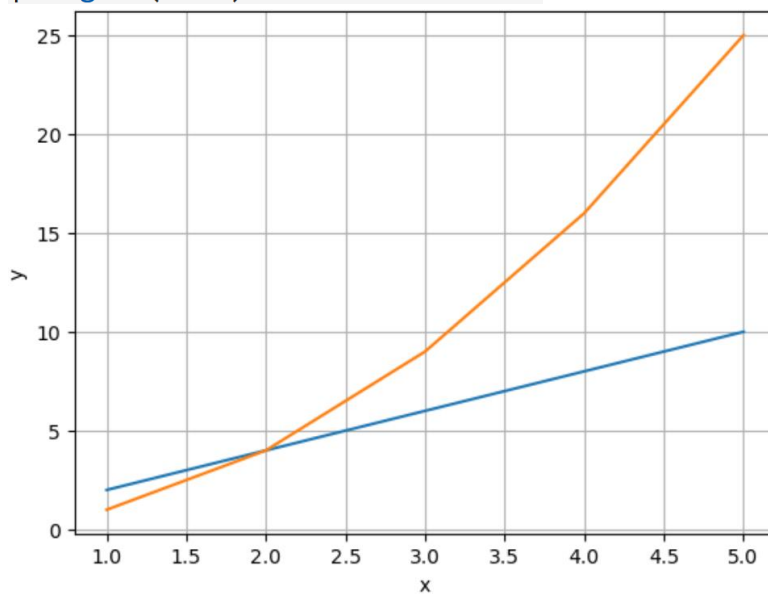
Для побудови на одному полі декількох кривих функцію *plot()* викликають для кожного набору даних. Це дозволяє налаштувати відображення кожної кривої окремо:

```
import numpy as np
import matplotlib.pyplot as plt
x = np.array([1, 2, 3, 4, 5])
y1 = 2 * x
y2 = x ** 2
plt.plot(x, y1, c = 'y', label = 'y = 2 * x')
plt.plot(x, y2, c = 'r', label = 'y = x ** 2')
plt.xlabel('x'); plt.ylabel('y')
plt.grid(True)
plt.legend()
plt.show()
```



Також можливо використати один виклик функції `plot()`. У цьому випадку відповідні пари колекцій даних вказуються як параметри цієї функції послідовно:

```
import numpy as np
import matplotlib.pyplot as plt
x = np.array([1, 2, 3, 4, 5])
y1 = 2 * x
y2 = x ** 2
plt.plot(x, y1, x, y2)
plt.xlabel('x'); plt.ylabel('y')
plt.grid(True)
```



Іншим способом побудови графіка функції є введення її аналітичного виразу в символьному вигляді та застосування функцій модуля *SymPy*.

Символьні змінні, які використовуються для створення символьних виразів, задаються за допомогою функції *symbols()*:

```
import sympy as smp
x, y = smp.symbols('a b')
x
```

a

де *x* і *y* – імена символьних змінних, *a* і *b* – відображення цих символьних змінних в символьному виразі.

Символьні змінні при створенні не мають конкретних значень, що дозволяє працювати з математичними виразами в загальному вигляді.

Після створення символьних змінних необхідно задати вираз функції, графік якої буде створено. Для візуалізації функції однієї змінної можливо скористатися функцією *plot()* модуля *SymPy*:

```
sympy.plot(f, (x, xmin, xmax), title = None, xlabel = 'x', ylabel = 'f(x)',
axis_center = 'auto', label = None, line_color = 'b', legend = False,
markers = None, show = True)
```

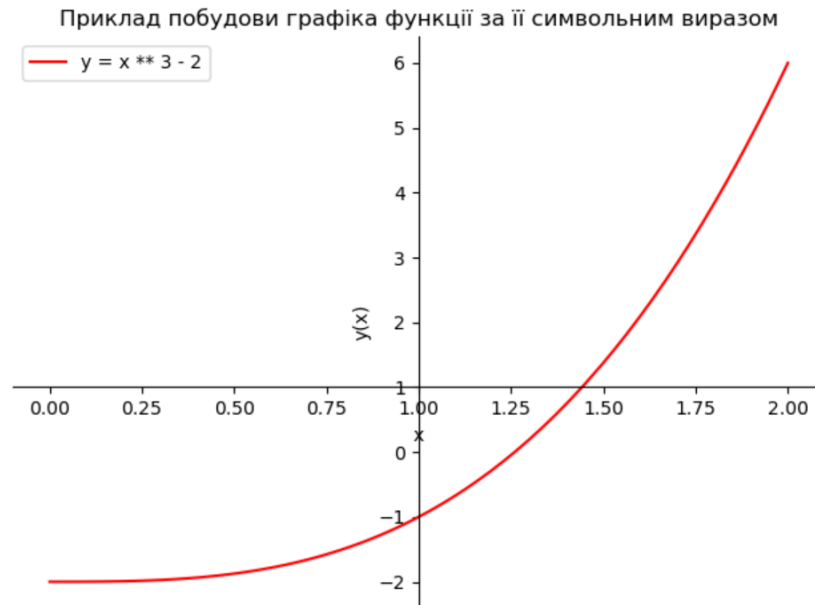
де *f* – символьний вираз функції; *x* – символьна змінна, що представляє аргумент функції; *xmin*, *xmax* – межі діапазону побудови графіка за віссю абсцис; *title* – назва графіка; *xlabel* – підпис осі абсцис; *ylabel* – підпис осі ординат; *axis_center* – положення початку координат; *label* – підпис кривої на легенді; *line_color* – колір кривої; *legend* – відображення легенди, приймає значення *True* або *False*; *markers* – налаштування маркерів для окремих точок графіка; *show* – відображення графіка після виклику функції *plot()*, приймає значення *True* або *False*.

Приклад побудови графіка функції з використанням функції *plot()* модуля *SymPy*:

```
# Символьний вираз функції, графік якої будується
import sympy as smp
x = smp.symbols('x')
y = x ** 3 - 2
y
```

$x^3 - 2$

```
# Побудова графіка функції
smp.plot(y, (x, 0, 2), xlabel = 'x', ylabel = 'y(x)', axis_center = [1, 1],
title = 'Приклад побудови графіка функції за її символьним виразом',
label = 'y = x ** 3 - 2', line_color = 'r', legend = True)
```



Функція `plot()` модуля `SymPy` автоматично обирає кількість точок для побудови графіка.

Не кожне рівняння задає функцію. Функція описує залежність однієї змінної від інших, причому кожному набору значень аргументів відповідає єдине значення функції. Рівняння ж може встановлювати взаємозв'язок між кількома змінними, і ця залежність не завжди є однозначною [14].

Для побудови графіків таких рівнянь використовується функція `plot_implicit()` модуля `SymPy`. З її допомогою можливо будувати графіки неявних рівнянь двох змінних, в яких одна змінна не виражена через іншу, а також графіки нерівностей.

Графіком рівняння є множина всіх точок на площині або у просторі, координати яких є розв'язками даного рівняння.

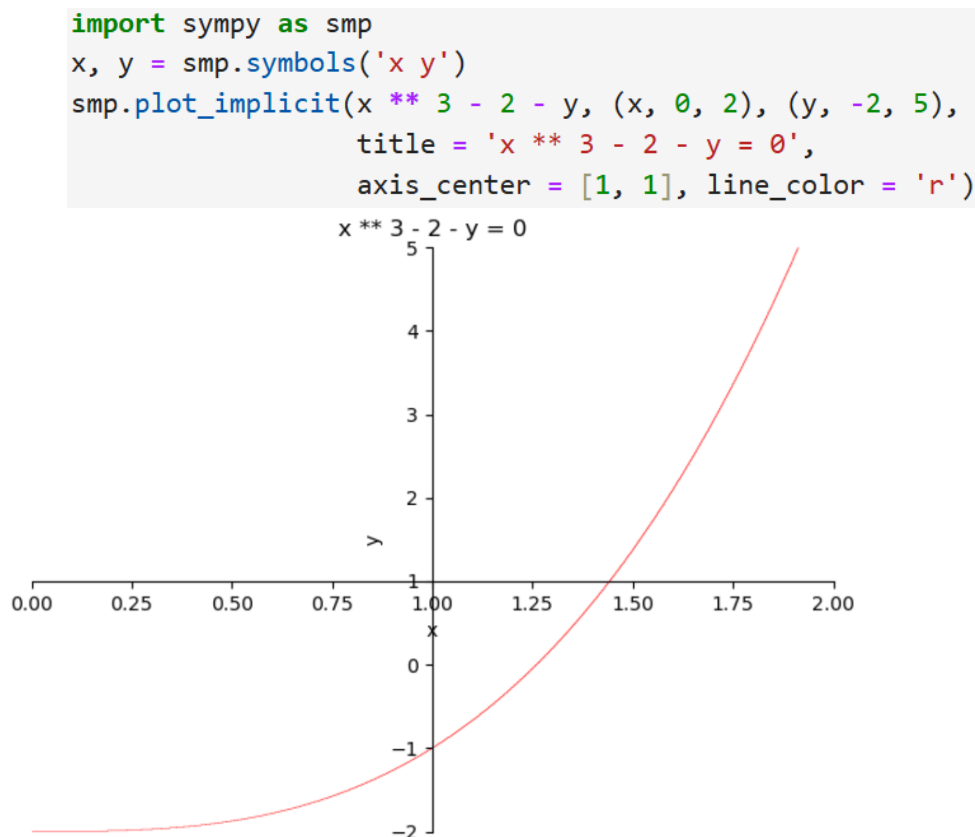
Функція `plot_implicit()` має наступний синтаксис:

```
sympy.plot_implicit(expr, (x, xmin, xmax), (y, ymin, ymax), line_color = 'b',
                    xlabel = 'x', ylabel = 'y', title = None, axis_center = True, adaptive = True,
                    show = True)
```

де `expr` – символьний вираз $F(x, y)$ неявного рівняння $F(x, y) = 0$, символьний вираз нерівності або символьне рівняння, яке створене з використанням функції `Eq()`; `x` – символьна змінна, яка представляє аргумент уздовж осі абсцис; `xmin`, `xmax` – межі діапазону побудови графіка за віссю абсцис; `y` – символьна змінна, яка представляє аргумент уздовж осі ординат; `ymin`, `ymax` – межі діапазону побудови графіка за віссю ординат; `line_color` – колір кривої; `xlabel` – підпис осі абсцис; `ylabel` – підпис осі ординат; `title` – назва графіка; `axis_center` – положення початку координат; `adaptive` – адаптивний алгоритм побудови графіка: графік будується з постійним кроком між сусідніми точками (`False`)

або крок змінюється автоматично (*True*); *show* – відображення графіка після виклику функції *plot()*, приймає значення *True* або *False*.

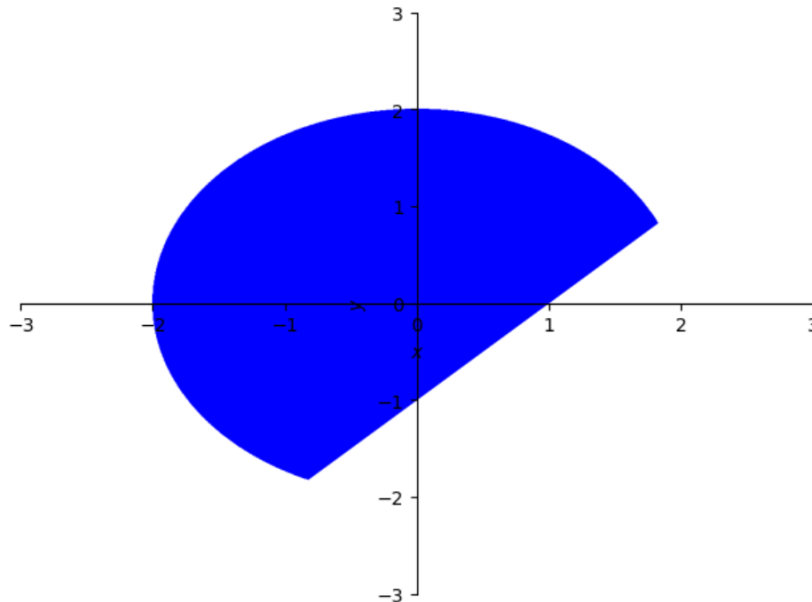
Приклад побудови графіка неявного рівняння $x^3 - 2 - y = 0$ з використанням функції *plot_implicit()*:



При побудові за допомогою функції *plot_implicit()* фігур, які задаються нерівностями, точка площини належить фігурі, якщо після підстановки координат цієї точки до нерівності вона приймає значення *True*.

Для побудови фігури, яка визначається одночасно кількома нерівностями, можливо використовувати функції *And()* та *Or()* модуля *SymPy*. Результатом виконання функції *And()* є *True*, якщо всі вказані логічні умови одночасно приймають значення *True*. Результатом виконання функції *Or()* є *True*, якщо хоча б одна логічна умова приймає значення *True*:

```
import sympy as smp
x, y = smp.symbols('x y')
smp.plot_implicit(smp.And(x ** 2 + y ** 2 <= 4, y >= x - 1),
                  (x, -3, 3), (y, -3, 3))
```



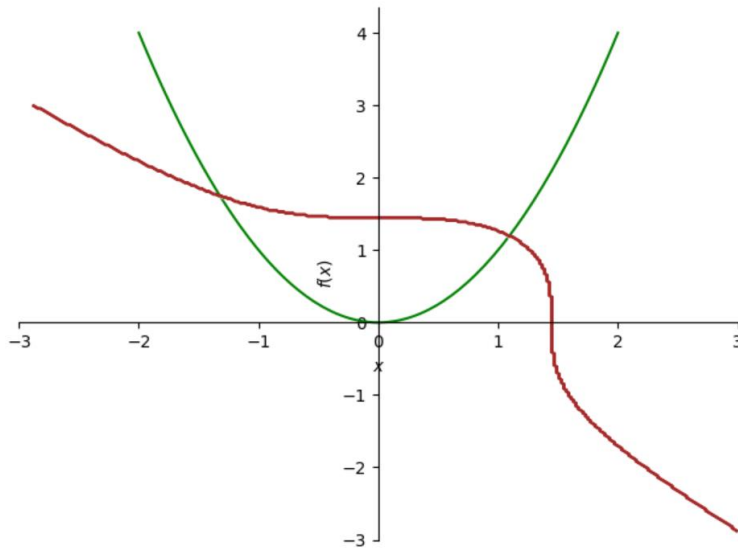
Вигляд фігури на графіку визначають дві логічні умови:

- $x^2 + y^2 \leq 4$, яка задає область всередині або на межі кола з радіусом, що дорівнює 2, і з центром в точці $(0; 0)$;
- $y \geq x - 1$, яка задає область, що розташована вище або на межі прямої $y = x - 1$.

Використання функції *And()* дозволяє включити до фігури лише ті точки площини, які одночасно належать обом визначеним областям. Для об'єднання областей використовується функція *Or()*.

Об'єднати на одному полі декілька кривих, які побудовані з використанням функцій *plot()* та/або *plot_implicit()* модуля *SymPy*, можливо з використанням методу *extend()*. Ім'я графіка, який додається до іншого графіка, вказується як параметр методу *extend()*. Ім'я графіка, до якого додається інший графік, вказується в префіксі методу *extend()*:

```
import sympy as sp
x, y = sp.symbols('x y')
p1 = sp.plot(x ** 2, (x, -2, 2), line_color = 'g', show = False)
p2 = sp.plot_implicit(x ** 3 + y ** 3 - 3, (x, -3, 3), (y, -3, 3),
                     line_color = 'brown', adaptive = False,
                     show = False)
p1.extend(p2)
p1.show()
```



3.2 Створення функцій у *Python*

Функція в *Python* – це послідовність інструкцій, що призначена для розв’язання певної задачі і яка може повертати в місце свого виклику деякі значення чи об’єкти [9].

В попередніх розділах розглядалися як вбудовані функції *Python*, так і функції математичних модулів. Однак у *Python* також можна створювати і власні функції.

Функцією є логічно незалежний фрагмент коду, який оформлено за певними правилами. Іменовані функції в *Python* створюються з використанням ключових слів *def*, *return* і мають наступний загальний синтаксис:

```
def function_name(args):
    function_body
    return result
```

де *function_name* – ім’я функції; *args* – опис формальних параметрів функції; *function_body* – програмний код функції; *result* – результат виконання функції.

Якщо під час виконання функції інтерпретатор зустрічає ключове слово *return*, то він передає значення *result* в місце виклику функції. Після цього виконання функції одразу припиняється, навіть якщо в її тілі є інші інструкції.

Функцію можливо багаторазово викликати з будь-якого місця програми. Виклик іменованої функції здійснюється за її іменем.

Функція в *Python* залежить від формальних параметрів *args*. Наприклад, створено функцію, яка має три формальні параметри *x*, *y*, *z*:

```
def function(x, y, z):
    a = x + y - z
    return a
```

Під час виклику функції до неї необхідно передати фактичні значення цих параметрів:

```
function(2, 3, 4)
```

1

При цьому формальні параметри (x, y, z) приймають передані фактичні значення (2, 3 і 4 відповідно), виконується програмний код функції, а розраховане значення повертається в місце виклику функції і виводиться на екран.

Формальні параметри функції можуть мати значення за замовчуванням (типові значення), які задаються під час її оголошення за допомогою оператора присвоєння. В описі параметрів спочатку вказуються ті, які не мають типових значень:

```
def f(a, b = 5, c = 10):  
    return a * b + c
```

Якщо під час виклику функції фактичне значення такого параметра не вказано, то використовується його типове значення. Якщо фактичне значення передано до функції, то воно замінює типове:

```
f(2, 3)
```

16

Під час виклику функції відповідність між формальними параметрами та їхніми фактичними значеннями можна встановлювати не тільки позиційно, а й за іменем параметра. В цьому випадку задавати фактичні значення параметрів можливо в довільному порядку:

```
f(b = 3, a = 2)
```

16

Функція в *Python* може повертати будь-який об'єкт – число, рядок, список, кортеж тощо. Якщо функція не містить ключового слова *return* або під час виклику функції інтерпретатор не натрапляє на нього, то функція повертає значення *None*.

Функція може мати декілька інструкцій, які починаються ключовим словом *return* (наприклад, в альтернативних блоках умовного оператора).

Одна функція може повертати декілька значень або об'єктів. Для цього їх імена (або інструкції, за якими вони розраховуються) необхідно вказати через кому після ключового слова *return*:

```
def calc(x):  
    return 2 * x, 3 * x  
calc(2)
```

(4, 6)

3.3 Розв'язання рівнянь у *Python*

Рівнянням є рівність двох математичних виразів, які містять один або кілька аргументів (змінних, невідомих):

$$f(x_1, x_2, \dots) = g(x_1, x_2, \dots). \quad (3.1)$$

Значення аргументів, які при підстановці в рівняння забезпечують рівність, є розв'язками (коренями) цього рівняння.

Аргументами і розв'язками рівнянь можуть бути як числа, так і складніші математичні об'єкти [15]. Так, розв'язками диференціальних рівнянь зазвичай є функції.

Для того, щоб розв'язати рівняння, необхідно знайти його розв'язки або довести, що їх не існує.

До нормальної форми

$$F(x_1, x_2, \dots) = 0 \quad (3.2)$$

алгебраїчне або трансцендентне рівняння можливо привести перенесенням його правої частини до лівої (або лівої до правої).

Алгебраїчне рівняння однієї змінної x степеня n має наступний загальний вигляд:

$$a_n x^n + a_{n-1} x^{n-1} \dots + a_1 x + a_0 = 0, \quad (3.3)$$

де $a_0, a_1, \dots, a_n$ – числові коефіцієнти рівняння.

До таких рівнянь відносяться, наприклад, лінійні, квадратні, кубічні рівняння тощо.

В множині комплексних чисел алгебраїчне рівняння однієї змінної степеня n завжди має рівно n коренів. Корені можуть бути дійсними, уявними або комплексно-спряженими числами. Деякі корені можуть бути кратними, тобто приймати однакові значення.

Трансцендентними рівняннями є рівняння, які містять трансцендентні функції. Наприклад, показникову, логарифмічну, експоненціальну функції, тригонометричні і обернені тригонометричні функції тощо.

Трансцендентні рівняння можуть мати один, кілька або нескінченно багато розв'язків, а можуть не мати розв'язків взагалі.

Процедура вирішення алгебраїчних та трансцендентних рівнянь складається з двох етапів: попереднє визначення кількості дійсних коренів рівняння і знаходження інтервалів, що містять лише один дійсний корінь (локалізація коренів) і подальше уточнення розв'язків (розв'язання рівняння).

Локалізацію (виокремлення) можливо проводити аналітично і графічно. При використанні графічного способу локалізації коренів рівняння $F(x) = 0$, будують графік $y = F(x)$. Кількість дійсних коренів рівняння відповідає кількості перетинів кривою осі абсцис (або дотиків до неї). Коренями є абсциси точок перетину, які за графіком складно визначити з високою точністю. Тому для кожного дійсного кореня знаходять інтервал, якому він належить, після чого переходять до процедури уточнення коренів.

Уточнити розв'язки алгебраїчних рівнянь однієї змінної довільного порядку можливо з використанням функцій *roots()* модуля *NumPy* та функції *nroots()* модуля *SymPy* [5, 7].

Параметром функції *roots()* є список (або матриця-рядок) коефіцієнтів полінома алгебраїчного рівняння. Першим в списку вказується коефіцієнт a_n (3.3) при найвищому степені змінної; останнім – вільний коефіцієнт a_0 . Поліному порядку n відповідає список, який містить $n + 1$ коефіцієнт. Деякі з цих коефіцієнтів можуть дорівнювати нулю або одиниці.

Створити поліном за списком його коефіцієнтів можливо з використанням функції *poly1d()* модуля *NumPy*.

Функція *nroots()* має наступний синтаксис:

sympy.nroots(f, n = 15, maxsteps = 50)

де f – символічний вираз полінома алгебраїчного рівняння; n – кількість значущих цифр у розрахованих коренях рівняння; *maxsteps* – максимальна кількість ітерацій.

Знайти числові розв'язки трансцендентних рівнянь, а також систем нелінійних рівнянь, можливо з використанням функцій *fsolve()* модуля *SciPy* та функції *nsolve()* модуля *SymPy* [6, 7].

Функція *fsolve()* має наступний синтаксис:

*scipy.optimize.fsolve(func, x0, xtol = 1.49012e-08, maxfev = 100 * (N+1))*

де *func* – *Python*-функція, яка повертає значення лівих частин рівнянь системи, що приведені до нормальної форми, після підстановки чергових наближень розв'язків; x_0 – колекція початкових наближень розв'язків для кожної змінної; *xtol* – точність розрахунку; *maxfev* – максимальна кількість ітерацій (викликів функції *func*). Значення за замочуванням розраховується з урахуванням кількості N рівнянь в системі. Якщо розв'язується одне рівняння, то *maxfev* = 200.

Функція *nsolve()* має наступний синтаксис:

sympy.nsolve(f, args, x0)

де f – символічне рівняння або колекція символічних рівнянь, що складають систему; *args* – перелік символічних змінних. Якщо розв'язується одне рівняння, даний параметр можна не вказувати; x_0 – початкове наближення або колекція початкових наближень розв'язків для кожної змінної.

Рівняння f зручно задавати з використанням функції $Eq()$ модуля *SymPy*. Параметрами функції $Eq()$ є символічні вирази в лівій і правій частинах рівняння. Попереднє приведення рівнянь до нормальної форми не є обов'язковим.

Початкове наближення розв'язку рівняння, яке потрібно задати в якості параметрів функцій $fsolve()$ і $nsolve()$, можливо визначити графічно при вирішенні задачі локалізації коренів рівняння.

Функції $fsolve()$ і $nsolve()$ дозволяють знайти один розв'язок рівняння, який є найближчим до заданого початкового наближення. Для знаходження всіх розв'язків рівняння дані функції необхідно застосувати декілька разів з використанням відповідних початкових наближень для кожного кореня.

3.4 Завдання до лабораторної роботи

Завдання 3.1

Побудуйте графік функції $f(x)$ на заданому інтервалі.

№	Функція	Інтервал
1	$f(x) = \ln(x)\sin(2x) + 1,8x;$	$x \in [0,8; 8,4].$
2	$f(x) = \sqrt{x} + \cos(x) + 0,2x;$	$x \in [0,5; 8].$
3	$f(x) = \cos(x) + \sin(2x) + 0,4x;$	$x \in [-2,1; 7,5].$
4	$f(x) = e^{-0,64x} - \frac{\cos(1,48x)}{2,5x};$	$x \in [0,9; 8,5].$
5	$f(x) = -0,5\sqrt{x} + \cos(1,8x) + 1,1x;$	$x \in [0,3; 10,3].$
6	$f(x) = x^5 e^{-x} \sin(0,8x) + 1,3x;$	$x \in [0,7; 8,3].$
7	$f(x) = e^{-\cos(x)} + 1,5x;$	$x \in [-1,8; 7,8].$
8	$f(x) = \ln(x) + 1,1\sin(x) + 1,6x;$	$x \in [0,6; 9].$
9	$f(x) = \frac{13,8\ln(x^2)}{0,8\sin(x) + 2,7x};$	$x \in [1,9; 12,7].$
10	$f(x) = \frac{1,23}{x} + \cos(1,44x) + 2,1x;$	$x \in [0,3; 8,7].$
11	$f(x) = \sqrt{x} + \sin(x) + 0,1x;$	$x \in [0,1; 10,1].$
12	$f(x) = e^{-0,1x} \cos(x) + 0,8x;$	$x \in [-1,4; 9,2].$
13	$f(x) = \sin(2x) + \cos(x^2) - \frac{8}{3x};$	$x \in [0,4; 5,3].$
14	$f(x) = x^2 \cos(2x) + 0,9x;$	$x \in [-1,9; 7,7].$
15	$f(x) = e^{-\sin(x) - \cos(2x)} + 2,5x;$	$x \in [1,6; 7,3].$
16	$f(x) = \ln(0,86x) \cos(3,75x) + 1,9x;$	$x \in [1,5; 7,1].$
17	$f(x) = \sin(x) + \cos(2x) + 0,3x;$	$x \in [-1,5; 9,6].$
18	$f(x) = \frac{8,4}{x^{\cos(x)} + \sin(1,2x) + 1,6} + 2,8x;$	$x \in [0,9; 10,2].$
19	$f(x) = e^{-\sin(x)} + 1,4x;$	$x \in [-1,3; 9,3].$
20	$f(x) = \cos(x) + 0,5\sin(2x) + 0,6x;$	$x \in [0,2; 10,2].$

21	$f(x) = \frac{x^2 \cos(2x)}{x + 4,7} + 2,2x;$	$x \in [0,5; 10,6].$
22	$f(x) = x^2 - e^{-0,2x} \cos(5,2x) - 2,4x;$	$x \in [-1,2; 5,4].$
23	$f(x) = \sin(x^2) + 0,5 \cos(2x) + \frac{0,5x}{4};$	$x \in [-0,2; 4,8].$
24	$f(x) = \frac{2,5}{x} + \sin(2,56x) + 2x;$	$x \in [0,4; 6,5].$
25	$f(x) = \sqrt{x^2 + 0,2x} + \cos(3x);$	$x \in [1,7; 7,9].$
26	$f(x) = (\sin(1,2x) + 2,9x)e^{-0,18x};$	$x \in [1,6; 9,3].$
27	$f(x) = \ln(2x) + 3,2 \cos(x) + 1,7x;$	$x \in [0,4; 10,5].$
28	$f(x) = \sin(x) + 0,5 \cos(2x) + 0,5x;$	$x \in [0,3; 8,7].$
29	$f(x) = \sqrt{x} + \sin(3x) + x;$	$x \in [0,5; 8,9].$
30	$f(x) = 5,18e^{-0,3x} \sin(x) + 0,7x;$	$x \in [0,6; 8,2].$

Завдання 3.2

Графічно визначте кількість дійсних коренів алгебраїчного рівняння і знайдіть інтервали, в яких вони розташовуються.

Знайдіть корені рівняння з використанням функції *roots()* модуля *NumPy* та функції *nroots()* модуля *SymPy*.

1	$x^3 + 2x^2 + 2 = 0.$	2	$2x^3 + 0,2x^2 + 0,5x + 0,8 = 0.$
3	$8x^3 - 3x^2 + 9x - 10 = 0.$	4	$-x^3 - 4x^2 - 6 = 0.$
5	$-3x^3 - 2x + 2 = 0.$	6	$3,2x^3 + 0,1x^2 + 0,4x - 1,2 = 0.$
7	$0,4x^3 + 3x - 1 = 0.$	8	$0,2x^3 + 3x^2 + 6x - 1 = 0.$
9	$9x^3 + x - 3 = 0.$	10	$-0,1x^3 - 0,1x^2 + 0,4x - 1,5 = 0.$
11	$-5x^3 + 0,4x^2 + 0,6x - 1,6 = 0.$	12	$0,2x^3 - 3x^2 + 6x - 2 = 0.$
13	$0,6x^3 - 0,2x^2 + 0,4x - 1,4 = 0.$	14	$3,6x^3 - 0,2x^2 + 0,3x - 1,2 = 0.$
15	$-0,8x^3 - 0,1x^2 + 0,4x + 2 = 0.$	16	$-0,5x^3 - 3x^2 + 12x - 9 = 0.$
17	$3x^3 + 3x^2 + 12x + 3 = 0.$	18	$1,7x^3 + 0,2x^2 + 0,5x - 2 = 0.$
19	$-4x^3 - 0,2x^2 + 0,5x - 1 = 0.$	20	$-2x^3 + 3x + 1 = 0.$
21	$2,4x^3 - 0,1x^2 + 0,4x + 1,2 = 0.$	22	$0,1x^3 + 0,2x^2 + 0,5x - 1,2 = 0.$
23	$-0,7x^3 - 3x^2 + 6x - 5 = 0.$	24	$0,6x^3 - 3x^2 + 9x + 2 = 0.$
25	$0,3x^3 - 0,2x^2 + 0,5x - 1,4 = 0.$	26	$-2,1x^3 - 0,1x^2 + 0,4x - 1,5 = 0.$

27	$14x^3 + 2x + 4 = 0.$	28	$1,8x^3 - 3x^2 + 6x + 3 = 0.$
29	$-18x^3 - 3x^2 + 12x - 12 = 0.$	30	$-24,6x^3 + 8x^2 - 12 = 0.$

Завдання 3.3

Графічно визначте кількість дійсних коренів трансцендентного рівняння і їх початкові наближення.

Знайдіть корені рівняння з використанням функції *fsolve()* модуля *SciPy* та функції *nsolve()* модуля *SymPy*.

1	$\ln(x) + (x+1)^3 = 2.$	2	$x = (x+1)^3.$
3	$x \cdot 2^x = 1.$	4	$x^2 = \sin(x).$
5	$\sqrt{x+1} = \frac{1}{x}.$	6	$x = \sqrt{\ln(x+4)}.$
7	$x - \cos(x) = 3,6.$	8	$x^2 = \ln(x+3).$
9	$3x + \cos(x) = -1.$	10	$2x + \ln(x) = 17,2.$
11	$x + \ln(x) = \frac{4,5}{x^3}.$	12	$\sin(0,5x) + 1 = x^2.$
13	$3 - x = \ln(x).$	14	$0,5x + \ln(x-1) = 2,4.$
15	$(x-8)^2 = \frac{1}{2}e^x.$	16	$\sin(0,5+x) = 2x - 2,5.$
17	$(2-x)e^x = 0,5.$	18	$\ln(2+x) + 2x = 3.$
19	$2,2x - 2^x = -0,4.$	20	$2\sin(x-0,6) = 3,2 - x.$
21	$x^2 + 4\sin(x) = -0,2.$	22	$x + \frac{\ln(1+x)}{x} = 3,7.$
23	$2x - \ln(x^2) = \frac{7}{x}.$	24	$(x+1)^2 = xe^{x-2}.$
25	$5x - 8\ln(x) = 8.$	26	$\ln(x^2 + 2x + 1) = -3 - x.$
27	$3x - e^x = -6,4.$	28	$2x + \cos(x) = 0,5.$
29	$x(x+1)^2 = 55.$	30	$x^3 = \sin(x).$

3.5 Приклади розв'язання завдань

Завдання 3.1

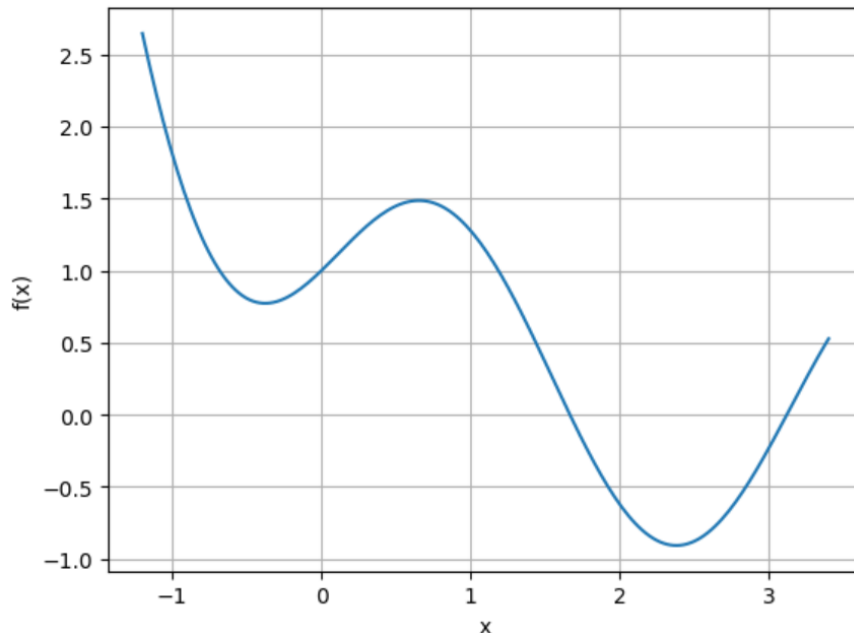
Побудуйте графік функції $f(x) = \sin(2x) + e^{-x}$ на інтервалі $x \in [-1, 2; 3, 4]$.

Розв'язання

Перший спосіб.

Розрахунок координат точок графіка функції, побудова графіка функції за цими точками.

```
import numpy as np
import matplotlib.pyplot as plt
a = -1.2 # Межі діапазону побудови графіка за віссю абсцис
b = 3.4
m = 200 # Кількість точок, за якими будується графік
x = np.linspace(a, b, m) # Матриця зі значеннями аргументу
f = np.sin(2 * x) + np.exp(-x) # Матриця зі значеннями функції
plt.plot(x, f) # Побудова графіка функції
plt.xlabel('x') # Підпис координатних осей
plt.ylabel('f(x)')
plt.grid(True) # Сітка
```



Другий спосіб.

Побудова графіка функції за її символьним виразом.

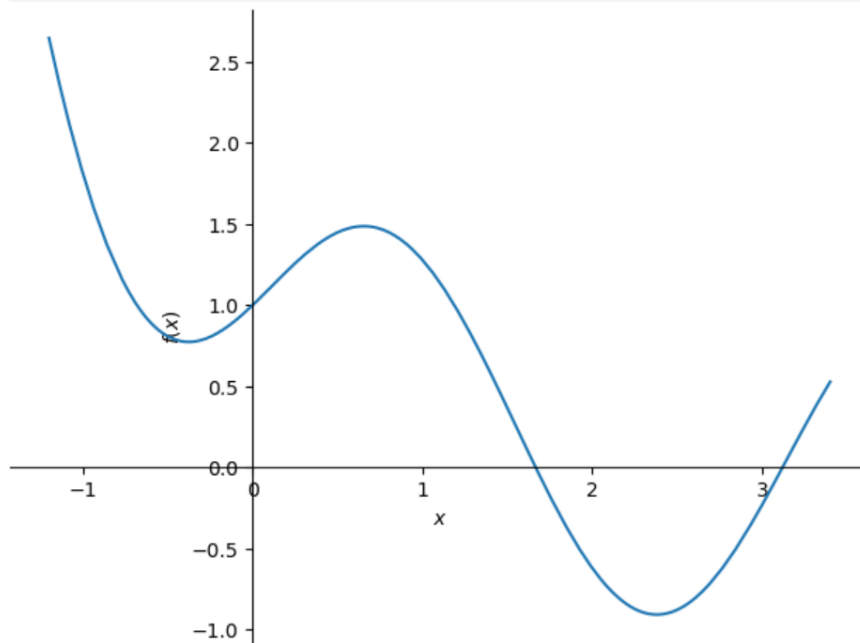
1) Створюємо символьний вираз функції:

```
import sympy as smp
x = smp.symbols('x')
f = smp.sin(2 * x) + smp.exp(-x)
f
```

$$\sin(2x) + e^{-x}$$

2) Будуємо графік функції:

```
a = -1.2 # Межі діапазону побудови графіка за віссю абсцис
b = 3.4
smp.plot(f, (x, a, b))
```



Примітка: під час виконання лабораторної роботи необхідно побудувати графік функції обома способами.

Завдання 3.2

Графічно визначте кількість дійсних коренів алгебраїчного рівняння і знайдіть інтервали, в яких вони розташовуються.

Знайдіть корені рівняння з використанням функції *roots()* модуля *NumPy* та функції *nroots()* модуля *SymPy*:

$$x^3 - 2x^2 - x - 15 = 0.$$

Розв'язання

1) Локалізуємо корені рівняння графічним методом.

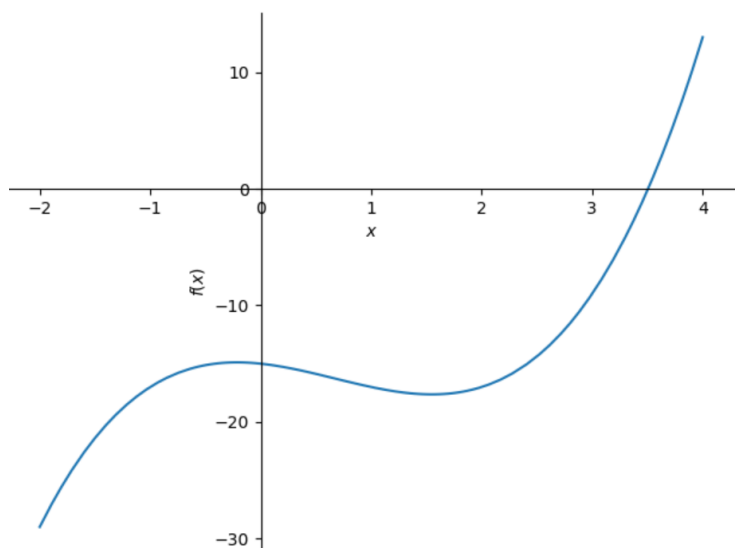
Створюємо символний вираз полінома алгебраїчного рівняння:

```
import sympy as smp
x = smp.symbols('x')
y = x ** 3 - 2 * x ** 2 - x - 15
y
```

$$x^3 - 2x^2 - x - 15$$

Будуємо графік функції:

```
a = -2 # Межі діапазону побудови графіка за віссю абсцис
b = 4
smp.plot(y, (x, a, b), axis_center = [0, 0])
```



Рівняння має один дійсний корінь.

Інтервал, в якому він розташовується: [3; 4].

2) Розв'язуємо рівняння за допомогою функції *roots()* модуля *NumPy*.

Створюємо список коефіцієнтів алгебраїчного рівняння. Для перевірки правильності списку виводимо на екран рівняння, яке вирішується:

```
import numpy as np
# Список коефіцієнтів полінома алгебраїчного рівняння:
a = [1, -2, -1, -15]
y = np.poly1d(a) # Побудова полінома за списком коефіцієнтів
print(y, '= 0')
```

$$1x^3 - 2x^2 - 1x - 15 = 0$$

Знаходимо розв'язки рівняння:

```
r = np.roots(a) # Матриця розв'язків рівняння
for i in range(len(r)):
    print("x [", i + 1, "] =", np.round(r[i], 3))
```

```
x [ 1 ] = (3.506+0j)
x [ 2 ] = (-0.753+1.927j)
x [ 3 ] = (-0.753-1.927j)
```

3) Розв'язуємо рівняння за допомогою функції `nroots()` модуля `SymPy`. Створюємо символний вираз полінома рівняння:

```
import sympy as smp
x = smp.symbols('x')
y = x ** 3 - 2 * x ** 2 - x - 15
y
```

$$x^3 - 2x^2 - x - 15$$

Знаходимо розв'язки рівняння:

```
r = smp.nroots(y) # Список розв'язків рівняння
for i in range(len(r)):
    print("x [", i + 1, "] =", round(r[i], 3))
```

```
x [ 1 ] = 3.506
x [ 2 ] = -0.753 - 1.927*I
x [ 3 ] = -0.753 + 1.927*I
```

Відповідь: задане алгебраїчне рівняння третього степеня має один дійсний і два комплексно-спряжені корені:

$$x_1 = 3,506; \quad x_2 = -0,753 - 1,927j; \quad x_3 = -0,753 + 1,927j.$$

Примітки:

1) Межі діапазону побудови графіка за віссю абсцис обираються таким чином, щоб всі дійсні корені рівняння відслідковувались на графіку.

2) Функція *range()*, подібно до функції *numpy.arange()*, дозволяє створити числову колекцію, але лише з цілих чисел.

3) Функція *len()* використовується для визначення кількості елементів в числовій колекції.

4) В відповіді потрібно вказати всі корені заданого рівняння, а не тільки дійсні.

Завдання 3.3

Графічно визначте кількість дійсних коренів трансцендентного рівняння і знайдіть інтервали, в яких вони розташовуються.

Знайдіть корені рівняння з використанням функції *fsolve()* модуля *SciPy* та функції *nsolve()* модуля *SymPy*:

$$\sin(2x - 1) + \ln(x + 2) = 0,8.$$

Розв'язання

1) Локалізуємо корені рівняння графічним методом.

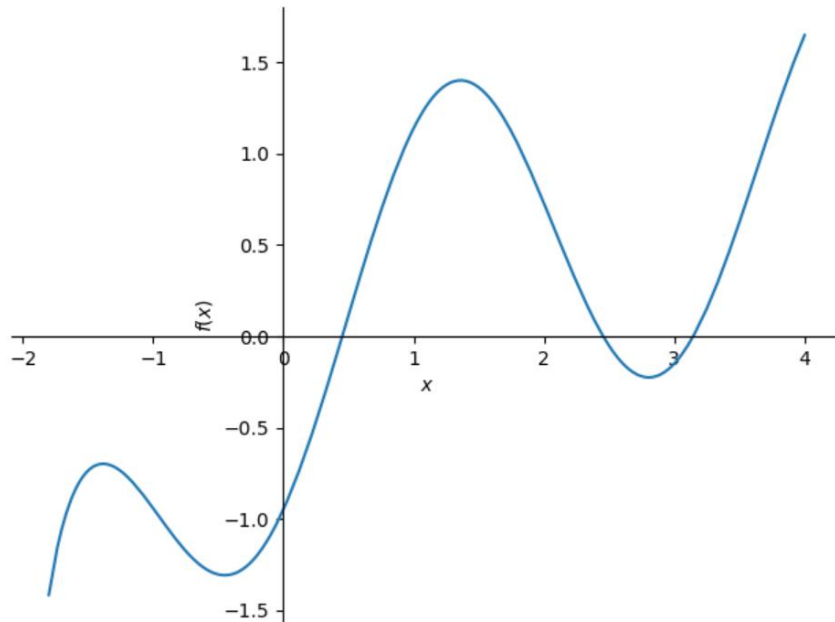
Приводимо задане рівняння до нормальної форми і створюємо символьний вираз для його лівої частини:

```
import sympy as smp
x = smp.Symbol('x')
# Вираз в лівій частині заданого рівняння:
lhs = smp.sin(2 * x - 1) + smp.log(x + 2)
# Вираз в правій частині заданого рівняння:
rhs = 0.8
# Вираз в лівій частині рівняння, що приведене до нормальної форми:
y = lhs - rhs
y
```

$$\log(x + 2) + \sin(2x - 1) - 0.8$$

Будуємо графік функції:

```
a = -1.8 # Межі діапазону побудови графіка за віссю абсцис
b = 4
smp.plot(y, (x, a, b), axis_center = [0, 0])
```



Рівняння має три дійсні корені.

Інтервали, в яких розташовуються корені: $[0; 1]$, $[2; 3]$, $[3; 4]$.

Початкові наближення коренів: 0,5; 2,5; 3,5.

2) Розв'язуємо рівняння за допомогою функції `fsolve()` модуля `SciPy`:

```
import numpy as np
from scipy.optimize import fsolve
n = 3 # Кількість розв'язків рівняння
# Список початкових наближень розв'язків:
a = [0.5, 2.5, 3.5]
# Створюємо Python-функцію:
def y(x):
    return np.sin(2 * x - 1) + np.log(x + 2) - 0.8
# Розрахунок і виведення на екран розв'язків рівняння:
for i in range(n):
    r = fsolve(y, a[i]) # Матриця, що містить розв'язок рівняння
    print("x [", i + 1, "] =", np.round(r[0], 4))
```

x [1] = 0.4516

x [2] = 2.4542

x [3] = 3.1448

3) Розв'язуємо рівняння за допомогою функції `nsolve()` модуля `SymPy`.
Створюємо символічне рівняння:

```
import sympy as smp
x = smp.Symbol('x')
y = smp.Eq(smp.sin(2 * x - 1) + smp.log(x + 2), 0.8)
y
```

$$\log(x + 2) + \sin(2x - 1) = 0.8$$

Знаходимо розв'язки рівняння:

```
n = 3 # Кількість розв'язків рівняння
# Список початкових наближень розв'язків:
a = [0.5, 2.5, 3.5]
# Розрахунок і виведення на екран розв'язків рівняння:
for i in range(n):
    r = smp.nsolve(y, a[i]) # Розв'язок рівняння
    print("x [", i + 1, "] =", round(r, 4))
```

```
x [ 1 ] = 0.4516
x [ 2 ] = 2.4542
x [ 3 ] = 3.1448
```

Відповідь: задане трансцендентне рівняння має три дійсні корені:

$$x_1 = 0,4516; \quad x_2 = 2,4542; \quad x_3 = 3,1448.$$

Примітка:

Межі діапазону побудови графіка за віссю абсцис обираються таким чином, щоб всі дійсні корені рівняння відслідковувались на графіку. В деяких випадках (наприклад, якщо функція має розриви) може потребуватись побудова кількох графіків для різних діапазонів значень аргументу рівняння.

Контрольні питання

1. Що є графіком функції?
2. Яким чином можливо змінити колір кривої на графіку, який побудовано з використанням функції *plot()* модуля *Matplotlib*?
3. Які параметри функції *plot()* модуля *Matplotlib* визначають стиль кривої та маркерів на графіку?
4. Яким чином з використанням функції *plot()* модуля *Matplotlib* можливо відобразити декілька графіків на одному полі?
5. Яким чином у *Python* можна створити символічне рівняння?
6. Яка функція модуля *SymPy* призначена для побудови графіків неявних рівнянь?
7. Яке призначення мають функції в *Python*?
8. Які ключові слова використовуються при описі іменованої функції в *Python*?
9. В чому полягають відмінності між алгебраїчним і трансцендентним рівняннями?
10. Скільки розв'язків в множині комплексних чисел має алгебраїчне рівняння четвертого степеня?
11. Як графічно визначити кількість розв'язків трансцендентного рівняння та їх початкові наближення?
12. Які функції математичних і наукових модулів *Python* можуть використовуватись для розв'язання трансцендентних рівнянь?

Лабораторна робота № 4

РОЗВ'ЯЗАННЯ СИСТЕМ ЛІНІЙНИХ І НЕЛІНІЙНИХ РІВНЯНЬ

Мета роботи: одержання практичних навичок розв'язання систем лінійних і нелінійних рівнянь.

4.1 Розв'язання систем лінійних рівнянь

Система рівнянь – це сукупність двох або більше рівнянь, які мають одні й ті самі невідомі та спільні розв'язки.

Розв'язком системи рівнянь є набір значень невідомих, який при підстановці до системи перетворює кожне її рівняння на тотожність [11].

Система лінійних рівнянь має наступний загальний вигляд:

$$\begin{cases} a_{11}x_1 + a_{12}x_2 + \dots + a_{1n}x_n = b_1, \\ a_{21}x_1 + a_{22}x_2 + \dots + a_{2n}x_n = b_2, \\ \dots \\ a_{m1}x_1 + a_{m2}x_2 + \dots + a_{mn}x_n = b_m. \end{cases} \quad (4.1)$$

де $a_{11}, \dots, a_{mn}$ – коефіцієнти рівнянь системи; $b_1, \dots, b_m$ – вільні члени; $x_1, \dots, x_n$ – невідомі, значення яких визначаються при розв'язанні системи.

Лінійно незалежними є рівняння, жодне з яких не можна отримати множенням іншого на число та/або додаванням одного рівняння до іншого.

Якщо система містить більше лінійно незалежних рівнянь, ніж змінних ($m > n$), то вона не має розв'язків. Якщо система містить менше лінійно незалежних рівнянь, ніж змінних ($m < n$), то вона має нескінченну кількість розв'язків (або не має розв'язків, якщо рівняння суперечливі).

Якщо кількість лінійно незалежних рівнянь системи дорівнює кількості змінних ($m = n$) і ці рівняння не суперечливі, то система має єдиний розв'язок.

Систему лінійних рівнянь можливо представити у вигляді матричного рівняння. Якщо виконується умова $m = n$, то

$$A \cdot X = B, \quad (4.2)$$

де A – квадратна матриця коефіцієнтів, основна матриця системи:

$$A = \begin{bmatrix} a_{11} & a_{12} & \dots & a_{1n} \\ a_{21} & a_{22} & \dots & a_{2n} \\ \dots & \dots & \dots & \dots \\ a_{n1} & a_{n2} & \dots & a_{nn} \end{bmatrix}, \quad (4.3)$$

B – матриця-стовпець вільних членів системи:

$$B = \begin{bmatrix} b_1 \\ b_2 \\ \dots \\ b_n \end{bmatrix}, \quad (4.4)$$

X – матриця-стовпець невідомих:

$$X = \begin{bmatrix} x_1 \\ x_2 \\ \dots \\ x_n \end{bmatrix}. \quad (4.5)$$

Система лінійних рівнянь має єдиний розв’язок, якщо визначник її матриці коефіцієнтів A не дорівнює нулю.

Якщо $|A| \neq 0$, то згідно з (4.2) розв’язок системи лінійних рівнянь

$$X = A^{-1} \cdot B, \quad (4.6)$$

де A^{-1} – матриця, що є оберненою до матриці коефіцієнтів A .

Знайти розв’язок системи лінійних рівнянь у *Python* можливо з використанням функції *linalg.solve()* модуля *NumPy*. Параметрами даної функції є матриці коефіцієнтів системи і вільних членів. Матриця вільних членів може бути одновимірною матрицею-рядком або двовимірною матрицею-стовпцем. Результатом є матриця, що містить значення невідомих, розмірність якої відповідає розмірності матриці вільних членів [16].

4.2 Розв'язання систем нелінійних рівнянь

Система n нелінійних рівнянь з n невідомими має наступний загальний вигляд:

$$\begin{cases} f_1(x_1, x_2, \dots, x_n) = 0, \\ f_2(x_1, x_2, \dots, x_n) = 0, \\ \dots \\ f_n(x_1, x_2, \dots, x_n) = 0. \end{cases} \quad (4.7)$$

Така система може мати один, кілька або нескінченно багато розв'язків, а може не мати розв'язків взагалі.

Розв'язання систем нелінійних рівнянь включає два етапи: локалізацію розв'язків та їх уточнення [17].

На етапі локалізації визначаються кількість розв'язків системи і їх початкові наближення. Якщо розв'язується система двох нелінійних рівнянь з двома змінними, то локалізацію зручно виконувати графічним методом. При цьому будують графіки рівнянь системи. Розв'язками системи нелінійних рівнянь є координати точок перетину або дотиків кривих.

Знайти числові розв'язки систем нелінійних рівнянь можливо з використанням функцій *fsolve()* модуля *SciPy* та функції *nsolve()* модуля *SymPy*, синтаксис яких розглянуто в розділі 3.3.

При використанні функції *fsolve()* спочатку необхідно привести кожне рівняння системи до нормальної форми. Якщо розв'язується система двох рівнянь з невідомими, наприклад, x та y , то рівняння повинні мати вигляд $f_1(x, y) = 0$ і $f_2(x, y) = 0$. Далі створюється *Python*-функція, яка для заданих значень змінних, що передаються у вигляді списку, обчислює і повертає значення $f_1(x, y)$ і $f_2(x, y)$. Дана функція і початкове наближення розв'язку є параметрами функції *fsolve()*.

В якості параметра функції *nsolve()* обов'язково вказується список імен змінних, відносно яких розв'язується система рівнянь.

Початкове наближення можливо задати у вигляді списку, що містить значення для кожної змінної системи. В списку спочатку вказують значення змінної, яка відкладається на графіку за віссю абсцис (x), потім – значення змінної, яка відкладається за віссю ординат (y). У такому ж порядку значення змінних містяться і в розв'язку системи.

Функції *fsolve()* і *nsolve()* дозволяють знайти один розв'язок системи, який є найближчим до заданого початкового наближення. Для знаходження всіх розв'язків дані функції необхідно застосувати декілька разів з використанням відповідних початкових наближень.

4.3 Завдання до лабораторної роботи

Завдання 4.1

Доведіть, що задана система лінійних рівнянь має єдиний розв'язок і знайдіть його:

- з використанням оберненої матриці коефіцієнтів;
- за допомогою функції *linalg.solve()* модуля *NumPy*.

1	$\begin{cases} -12,09x_1 - 29,03x_2 + 24,71x_3 - 14,92x_4 = -4,59; \\ -0,39x_1 + 24,21x_2 - 6,53x_3 + 1,88x_4 = -17,43; \\ -6,83x_1 + 28,85x_2 - 4,63x_3 - 21,59x_4 = -33,06; \\ 12,01x_1 - 13,8x_2 + 0,67x_3 + 16,22x_4 = -27,11. \end{cases}$
2	$\begin{cases} -6,05x_1 - 23,67x_2 - 22,09x_3 - 15,27x_4 = 15,17; \\ -17,4x_1 - 7,66x_2 - 20,33x_3 - 25,03x_4 = -22,49; \\ 28,94x_1 - 5,3x_2 + 11,08x_3 - 22,92x_4 = -30,56; \\ -18,61x_1 + 4,03x_2 - 6,41x_3 + 12,29x_4 = 28,75. \end{cases}$
3	$\begin{cases} -28,86x_1 + 10,13x_2 + 10,74x_3 - 18,51x_4 = -22,7; \\ 0,6x_1 + 23,04x_2 + 23,65x_3 - 14,5x_4 = -22,22; \\ 1,9x_1 + 29,17x_2 - 16,9x_3 - 7,35x_4 = -17,84; \\ 6,02x_1 - 21,9x_2 + 25,58x_3 - 1,47x_4 = 32,34. \end{cases}$
4	$\begin{cases} 22,61x_1 + 0,92x_2 - 10,73x_3 - 1,18x_4 = -15,3; \\ -26,17x_1 - 7,44x_2 + 6,68x_3 + 0,1x_4 = -21,42; \\ 3,63x_1 + 11,98x_2 + 22,78x_3 + 24,27x_4 = -23,08; \\ 1,38x_1 - 19,52x_2 - 8,79x_3 + 3,3x_4 = 0,6. \end{cases}$
5	$\begin{cases} 4,22x_1 - 22,88x_2 - 3x_3 + 28,59x_4 = 23; \\ -18,33x_1 - 5,93x_2 + 24,2x_3 + 22,41x_4 = -37,12; \\ -0,06x_1 - 6,87x_2 + 21,4x_3 - 29,8x_4 = 7,21; \\ -1,73x_1 + 9,58x_2 - 1,59x_3 + 25,65x_4 = -1,92. \end{cases}$

6	$\begin{cases} -10,54x_1 - 20,11x_2 + 22,05x_3 + 29,89x_4 = -9,38; \\ -5,65x_1 - 19,16x_2 - 23,53x_3 + 5,78x_4 = -13,97; \\ -11,65x_1 - 13,57x_2 - 22,8x_3 + 8,24x_4 = 36,27; \\ 9,74x_1 - 18,17x_2 - 6,89x_3 - 29,07x_4 = 4,33. \end{cases}$
7	$\begin{cases} -18,72x_1 - 2,32x_2 + 26,64x_3 + 16,96x_4 = 21,72; \\ 18,41x_1 - 8,3x_2 - 23,12x_3 - 13,58x_4 = -35,13; \\ 28,35x_1 - 8,47x_2 - 1,31x_3 - 8,27x_4 = -24; \\ -24,37x_1 - 1,44x_2 - 16,42x_3 - 23,61x_4 = -20,39. \end{cases}$
8	$\begin{cases} 20,75x_1 - 16,64x_2 + 4,76x_3 + 11,36x_4 = -31,04; \\ 28,26x_1 + 14,69x_2 - 21,17x_3 + 6,53x_4 = 38,42; \\ 24,01x_1 - 6,13x_2 + 12,2x_3 + 14,16x_4 = 22,63; \\ -1,27x_1 + 17,1x_2 - 6,04x_3 - 24,25x_4 = -15,74. \end{cases}$
9	$\begin{cases} 25,05x_1 - 0,9x_2 - 21,22x_3 - 2,86x_4 = -19,21; \\ 12,87x_1 - 3x_2 - 4,95x_3 - 10x_4 = -26,24; \\ -18,38x_1 - 18,56x_2 + 26,93x_3 - 3,98x_4 = 8,1; \\ -3,19x_1 - 17,28x_2 + 29,58x_3 + 5,41x_4 = -21,25. \end{cases}$
10	$\begin{cases} -14,9x_1 - 29,73x_2 + 15,6x_3 + 14,28x_4 = 19,45; \\ -3,78x_1 - 18,68x_2 + 3,38x_3 + 5,14x_4 = 9,89; \\ 13,06x_1 - 23,5x_2 + 15,8x_3 + 6,97x_4 = 32,27; \\ 3,52x_1 - 16,45x_2 - 21,51x_3 - 14,64x_4 = -33,4. \end{cases}$
11	$\begin{cases} 17,16x_1 + 18,4x_2 + 20,47x_3 - 0,6x_4 = -17,19; \\ -24,74x_1 - 23,83x_2 - 19,2x_3 + 8,43x_4 = 27,62; \\ 25,95x_1 - 6,09x_2 - 9,09x_3 - 11,47x_4 = 2,37; \\ -0,23x_1 + 9,64x_2 - 10,32x_3 - 16,75x_4 = 25,55. \end{cases}$
12	$\begin{cases} -0,32x_1 + 23,6x_2 - 1,02x_3 - 17,93x_4 = -35,7; \\ -20,44x_1 - 25,58x_2 + 19,25x_3 + 6,92x_4 = -4; \\ 29,8x_1 + 5,45x_2 + 10,51x_3 - 23,02x_4 = 17,11; \\ 27,31x_1 + 5,75x_2 - 7,03x_3 - 9,65x_4 = -4,06. \end{cases}$

13	$\begin{cases} 7,8x_1 + 10,48x_2 - 7,35x_3 + 29,23x_4 = 7,84; \\ 13,04x_1 + 20,61x_2 + 18x_3 - 28,61x_4 = 37,12; \\ 12,43x_1 + 21,5x_2 + 22,26x_3 + 27,41x_4 = -36,21; \\ -9,16x_1 - 12,8x_2 - 21,44x_3 - 2,74x_4 = 24,83. \end{cases}$
14	$\begin{cases} 9,66x_1 - 20,37x_2 - 25,66x_3 - 16,59x_4 = -17,24; \\ -18,18x_1 + 2,81x_2 + 23,85x_3 - 15,5x_4 = -27,3; \\ -20,14x_1 + 16,59x_2 - 10,06x_3 + 16,77x_4 = 26,87; \\ 15,64x_1 + 20,01x_2 - 0,58x_3 + 27,06x_4 = 21,59. \end{cases}$
15	$\begin{cases} -23,13x_1 - 7,48x_2 - 12,71x_3 + 4,72x_4 = -13,56; \\ 9,51x_1 + 17,01x_2 - 19,77x_3 - 1,05x_4 = 7,1; \\ 10,26x_1 + 20,74x_2 - 1,55x_3 + 10,11x_4 = -35,29; \\ 1,02x_1 - 1,03x_2 - 2,58x_3 + 19,53x_4 = 33,28. \end{cases}$
16	$\begin{cases} 6,52x_1 - 7,34x_2 + 1,19x_3 + 0,38x_4 = 26,69; \\ 14,95x_1 + 1,73x_2 + 20,15x_3 - 15,02x_4 = -5,84; \\ 12,4x_1 + 11,12x_2 + 4,04x_3 + 28,76x_4 = -11,4; \\ -14,15x_1 - 3,37x_2 - 21,45x_3 - 23,21x_4 = 14,49. \end{cases}$
17	$\begin{cases} 13,81x_1 - 9,59x_2 - 3,15x_3 - 29,95x_4 = 18,53; \\ 17,45x_1 - 26,54x_2 - 20,5x_3 + 27,3x_4 = 37,1; \\ -24,31x_1 + 13,29x_2 - 15,38x_3 - 16,85x_4 = -22,56; \\ 6,4x_1 + 0,89x_2 + 3,03x_3 + 28,4x_4 = -14,87. \end{cases}$
18	$\begin{cases} -17,23x_1 + 29,1x_2 - 16,46x_3 - 25,87x_4 = -26; \\ -8,88x_1 + 9,54x_2 - 0,79x_3 - 12,8x_4 = 38,28; \\ 19,26x_1 + 29,9x_2 - 5,43x_3 + 16,16x_4 = -35,83; \\ -6,38x_1 + 18,58x_2 + 17,11x_3 - 22,73x_4 = -29,05. \end{cases}$
19	$\begin{cases} -9,6x_1 + 7,12x_2 - 23,76x_3 + 11,13x_4 = 4,9; \\ 27,31x_1 + 6,86x_2 - 4,11x_3 + 10,26x_4 = 14,56; \\ -10,43x_1 - 19,58x_2 + 0,04x_3 + 19,24x_4 = 9,64; \\ -18,6x_1 + 5,82x_2 - 7,53x_3 - 10,62x_4 = -3,89. \end{cases}$

20	$\begin{cases} 22,01x_1 - 28,7x_2 - 18,56x_3 + 11,06x_4 = -9,41; \\ 29,09x_1 - 7,92x_2 + 27,61x_3 + 22,03x_4 = -27,91; \\ 13,11x_1 - 25,49x_2 - 6,94x_3 + 8,5x_4 = 14,31; \\ -8,28x_1 - 19,09x_2 + 7,03x_3 - 2,62x_4 = -5,52. \end{cases}$
21	$\begin{cases} 14,95x_1 + 9,94x_2 + 12,72x_3 - 9,78x_4 = -3,81; \\ 25,05x_1 - 17,81x_2 - 23,47x_3 + 25,93x_4 = 39,27; \\ -24,57x_1 + 24,89x_2 + 10,44x_3 + 9,11x_4 = -26,92; \\ -23,02x_1 - 28,82x_2 - 27,96x_3 + 23,28x_4 = -28,78. \end{cases}$
22	$\begin{cases} -11,95x_1 + 16,94x_2 - 3,07x_3 + 17,9x_4 = 17,72; \\ 4,14x_1 + 22,11x_2 - 4,26x_3 - 17,77x_4 = -15,65; \\ -27,71x_1 + 3,18x_2 + 20,57x_3 + 2,73x_4 = 34,27; \\ 22,75x_1 - 0,79x_2 - 20,33x_3 + 8,99x_4 = 12. \end{cases}$
23	$\begin{cases} -10,54x_1 - 10,89x_2 - 13,45x_3 - 23,66x_4 = -24,37; \\ -22,75x_1 + 1,02x_2 - 24,61x_3 + 15,33x_4 = 36,97; \\ -27,96x_1 - 14,54x_2 - 24,81x_3 - 17,95x_4 = -29,55; \\ 18,74x_1 + 2,37x_2 - 26,48x_3 - 21x_4 = -36,51. \end{cases}$
24	$\begin{cases} -1,09x_1 - 22,12x_2 - 18,22x_3 + 5,91x_4 = -31,61; \\ 23,33x_1 - 1,71x_2 - 9,94x_3 - 5,75x_4 = -28,11; \\ -17,46x_1 - 21,25x_2 + 23,08x_3 - 12,81x_4 = 12,11; \\ -25,42x_1 + 22,96x_2 - 11,6x_3 + 25,28x_4 = -35,96. \end{cases}$
25	$\begin{cases} -20,7x_1 - 2,07x_2 + 9,01x_3 + 11,38x_4 = -23,77; \\ -22,24x_1 - 25,67x_2 + 15,88x_3 - 3,29x_4 = 32,94; \\ -18,6x_1 - 8,26x_2 + 5,81x_3 - 20,95x_4 = -5,32; \\ 2,63x_1 - 26,17x_2 + 4,34x_3 + 7,12x_4 = 12,18. \end{cases}$
26	$\begin{cases} -21,59x_1 + 16x_2 - 22,52x_3 + 11,42x_4 = 3,23; \\ -18,7x_1 + 29,47x_2 + 26,05x_3 + 22,74x_4 = -33,78; \\ -3,59x_1 + 21,02x_2 - 5,9x_3 - 3,43x_4 = -14,82; \\ 11,17x_1 + 1,28x_2 + 25,47x_3 - 1,18x_4 = 39,08. \end{cases}$
27	$\begin{cases} 19,03x_1 + 27,07x_2 - 12,18x_3 + 13,88x_4 = -21,68; \\ -23,7x_1 + 7,91x_2 - 18,34x_3 - 17,07x_4 = -7,07; \\ 2,32x_1 + 17,73x_2 - 2,64x_3 + 3,09x_4 = -4,38; \\ -25,28x_1 - 8,69x_2 - 9,59x_3 - 19,8x_4 = 22,52. \end{cases}$

28	$\begin{cases} -0,28x_1 - 23,26x_2 - 5,21x_3 + 14,96x_4 = 15,66; \\ -23,11x_1 - 25,89x_2 + 19,06x_3 + 7,23x_4 = -17,71; \\ -8,61x_1 + 17,58x_2 + 8,19x_3 - 5,15x_4 = 31,52; \\ -2,2x_1 + 9,31x_2 - 23,06x_3 - 6,62x_4 = -29,42. \end{cases}$
29	$\begin{cases} 19,81x_1 - 29,51x_2 - 21,8x_3 + 8,92x_4 = -26,22; \\ -5,54x_1 - 8,09x_2 - 27,95x_3 + 18,89x_4 = 9,85; \\ 23,44x_1 - 25,54x_2 - 13,98x_3 - 23,14x_4 = 0,43; \\ 10,09x_1 + 15,71x_2 + 19,56x_3 - 17,27x_4 = 37,7. \end{cases}$
30	$\begin{cases} -3,89x_1 + 18,85x_2 + 5,16x_3 + 11,77x_4 = 34,94; \\ 0,08x_1 + 14,43x_2 + 28,46x_3 - 2,47x_4 = -32,03; \\ -20,59x_1 - 9,79x_2 - 14,34x_3 - 15,8x_4 = -26,5; \\ 21,73x_1 + 3,96x_2 + 12,49x_3 - 20,51x_4 = 20,64. \end{cases}$

Завдання 4.2

Графічним способом визначте кількість розв'язків системи нелінійних рівнянь і їх початкові наближення.

Знайдіть розв'язки системи нелінійних рівнянь з використанням функції *fsolve()* модуля *SciPy* та функції *nsolve()* модуля *SymPy*.

№	Система рівнянь	№	Система рівнянь
1	$\begin{cases} x = e^x \cos y - 1; \\ y = e^x \sin y + 1. \end{cases}$	16	$\begin{cases} x = -x^2 - y^2 - 0,1; \\ y = -2xy - 0,1. \end{cases}$
2	$\begin{cases} x = \frac{(x^2 - y^2)^2}{4} - x^2 y^2 + 0,5; \\ y = xy(x^2 - y^2) + 0,5. \end{cases}$	17	$\begin{cases} x = 0,1 - x^2 - y^2; \\ y = -2xy - 0,1. \end{cases}$
3	$\begin{cases} x = \frac{x}{x^2 + y^2} + 0,4; \\ y = -\frac{y}{x^2 + y^2} + 1,4. \end{cases}$	18	$\begin{cases} x = \frac{x}{x^2 + y^2} - 0,4; \\ y = \frac{y}{x^2 + y^2} + 1. \end{cases}$
4	$\begin{cases} x = x^2 + 0,8y^2 + 0,1; \\ y = 2xy + 0,1. \end{cases}$	19	$\begin{cases} x = e^x \cos y - 1; \\ y = e^x \sin y - 1. \end{cases}$
5	$\begin{cases} x = x^2 - y^2 + 0,1; \\ x = 2xy + 0,1. \end{cases}$	20	$\begin{cases} x = \frac{(x^2 - y^2)^2}{4} - x^2 y^2 + 0,5; \\ y = xy(x^2 - y^2) - 0,5. \end{cases}$
6	$\begin{cases} x = x^2 - y^2 - 0,1; \\ y = 2xy + 0,1. \end{cases}$	21	$\begin{cases} x = \frac{x}{x^2 + y^2} + 0,4; \\ y = -\frac{y}{x^2 + y^2} - 1,4. \end{cases}$
7	$\begin{cases} x = x^2 + y^2 + 0,1; \\ y = 2xy - 0,1. \end{cases}$	22	$\begin{cases} x = x^2 + 0,8y^2 + 0,1; \\ y = 2xy - 0,1. \end{cases}$
8	$\begin{cases} x = 1 - e^x \cos y; \\ y = e^{-x} \sin y + 1. \end{cases}$	23	$\begin{cases} x = x^2 - y^2 + 0,1; \\ x = 2xy - 0,1. \end{cases}$

9	$\begin{cases} x = x^2 + y^2 - 0,1; \\ y = 2xy - 0,1. \end{cases}$	24	$\begin{cases} x = x^2 - y^2 - 0,1; \\ x = 2xy - 0,1. \end{cases}$
10	$\begin{cases} x = \frac{x}{x^2 + y^2} + 0,2; \\ 2y = -\frac{y}{x^2 + y^2} + 1. \end{cases}$	25	$\begin{cases} x = x^2 + y^2 + 0,1; \\ 0,5x = 2xy + 0,1. \end{cases}$
11	$\begin{cases} x = x^2 y^2 - \frac{(x^2 - y^2)^2}{4} - 0,5; \\ y = xy(y^2 - x^2) + 0,5. \end{cases}$	26	$\begin{cases} x = x^2 + y^2 - 0,1; \\ x = 2xy + 0,1. \end{cases}$
12	$\begin{cases} x = \frac{x}{x^2 + y^2} - 0,4; \\ y = 1,4 - \frac{y}{x^2 + y^2}. \end{cases}$	27	$\begin{cases} x = \frac{x}{x^2 + y^2} + 0,4; \\ y = \frac{y + 0,5}{x^2 + y^2} - 1. \end{cases}$
13	$\begin{cases} x = -x^2 - 0,8y^2 - 0,1; \\ y = 0,1 - 2xy. \end{cases}$	28	$\begin{cases} x = \cos(xy); \\ y = x^2 - 5. \end{cases}$
14	$\begin{cases} x = x^2 - y^2 - 0,1; \\ x = 0,1 - 2xy. \end{cases}$	29	$\begin{cases} 5,5x^3 - y^2 + 5 = 0; \\ xy^3 - y - 4 = -1. \end{cases}$
15	$\begin{cases} x = 0,1 - x^2 + y^2; \\ x = 0,1 - 2xy. \end{cases}$	30	$\begin{cases} 7,5x^3 - y^2 + 1 = x; \\ xy^3 - 6y + 4 = 0. \end{cases}$

4.4 Приклади розв'язання завдань

Завдання 4.1

Доведіть, що задана система лінійних рівнянь має єдиний розв'язок і знайдіть його:

- з використанням оберненої матриці коефіцієнтів;
- за допомогою функції `linalg.solve()` модуля `NumPy`.

$$\begin{cases} 0,41x_1 + x_2 + 7,26x_3 - x_4 = 1,21; \\ 0,84x_1 + 2,65x_2 - 6,73x_3 + 0,8x_4 = 1,44; \\ 4,26x_1 - 1,55x_2 + 1,13x_3 - 0,44x_4 = 10,54; \\ -0,45x_1 + 2,52x_2 + 1,54x_3 - 0,12x_4 = -2,67. \end{cases}$$

Розв'язання

1) Створюємо матрицю коефіцієнтів системи і розраховуємо її визначник:

```
import numpy as np
A = np.array([[0.41, 1, 7.26, -1],
              [0.84, 2.65, -6.73, 0.8],
              [4.26, -1.55, 1.13, -0.44],
              [-0.45, 2.52, 1.54, -0.12]])
print('|A| =', round(np.linalg.det(A), 3))

|A| = -9.737
```

Визначник матриці коефіцієнтів системи не дорівнює нулю.
Задана система лінійних рівнянь має єдиний розв'язок.

2) Знаходимо розв'язок системи лінійних рівнянь з використанням оберненої матриці коефіцієнтів:

```
# Матриця коефіцієнтів системи:
A = np.array([[0.41, 1, 7.26, -1],
              [0.84, 2.65, -6.73, 0.8],
              [4.26, -1.55, 1.13, -0.44],
              [-0.45, 2.52, 1.54, -0.12]])
# Матриця вільних членів системи:
B = np.array([[1.21],
              [1.44],
              [10.54],
              [-2.67]])
#  $A \cdot X = B \rightarrow X = A^{-1} \cdot B$ :
X = np.dot(np.linalg.inv(A), B) # Матриця розв'язку системи
# Виведення значень невідомих на екран:
for i in range(len(X)):
    print('x [', i + 1, '] =', round(X[i][0], 3))

x [ 1 ] = 1.16
x [ 2 ] = -0.3
x [ 3 ] = -2.266
x [ 4 ] = -17.485
```

3) Знаходимо розв'язок системи лінійних рівнянь з використанням функції `linalg.solve()` модуля `NumPy`:

```
# Матриця коефіцієнтів системи:
A = np.array([[0.41, 1, 7.26, -1],
              [0.84, 2.65, -6.73, 0.8],
              [4.26, -1.55, 1.13, -0.44],
              [-0.45, 2.52, 1.54, -0.12]])
# Матриця вільних членів системи:
B = np.array([[1.21],
              [1.44],
              [10.54],
              [-2.67]])
X = np.linalg.solve(A, B) # Матриця розв'язку системи
# Виведення значень невідомих на екран:
for i in range(len(X)):
    print('x [', i + 1, '] =', round(X[i][0], 3))

x [ 1 ] = 1.16
x [ 2 ] = -0.3
x [ 3 ] = -2.266
x [ 4 ] = -17.485
```

Відповідь:

$$x_1 = 1,16; \quad x_2 = -0,3; \quad x_3 = -2,266; \quad x_4 = -17,485.$$

Завдання 4.2

Графічним способом визначте кількість розв'язків системи нелінійних рівнянь і їх початкові наближення.

Знайдіть розв'язки системи нелінійних рівнянь з використанням функції *fsolve()* модуля *SciPy* та функції *nsolve()* модуля *SymPy*:

$$\begin{cases} x^2 + y^2 = 4; \\ x + 2 = e^y. \end{cases}$$

Розв'язання

1) Локалізуємо розв'язки системи нелінійних рівнянь графічним методом. Створюємо символіні рівняння системи:

```
import sympy as smp
x, y = smp.symbols('x y')
a = smp.Eq(x ** 2 + y ** 2, 4)
a
```

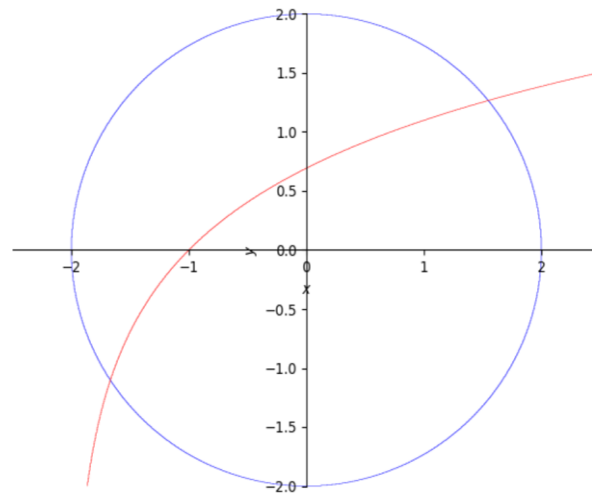
$$x^2 + y^2 = 4$$

```
b = smp.Eq(x + 2, smp.exp(y))
b
```

$$x + 2 = e^y$$

Будуємо графіки рівнянь системи:

```
import matplotlib.pyplot as plt
a_x = -3 # Межі діапазону побудови графіка за віссю абсцис
b_x = 3
a_y = -2 # Межі діапазону побудови графіка за віссю ординат
b_y = 2
plot1 = smp.plot_implicit(a, (x, a_x, b_x),
                        (y, a_y, b_y), show = False)
plot2 = smp.plot_implicit(b, (x, a_x, b_x), (y, a_y, b_y),
                        line_color = 'r', show = False)
plot1.extend(plot2)
plot1.show()
```



Задана система нелінійних рівнянь має два розв'язки.

Початкові наближення розв'язків: $(-1,5; -1)$ і $(1,5; 1)$.

2) Розв'язуємо систему нелінійних рівнянь за допомогою функції `fsolve()` модуля `SciPy`.

Приводимо рівняння системи до нормальної форми:

$$\begin{cases} x^2 + y^2 - 4 = 0; \\ x + 2 - e^y = 0. \end{cases}$$

```
import math
from scipy.optimize import fsolve
n = 2 # Кількість розв'язків системи нелінійних рівнянь
a = [[-1.5, -1], [1.5, 1]] # Список початкових наближень розв'язків
# Створюємо Python-функцію:
# p[0] - наближення першої змінної (x) системи, p[1] - другої змінної (y)
def func(p):
    Expr1 = p[0] ** 2 + p[1] ** 2 - 4
    Expr2 = p[0] + 2 - math.exp(p[1])
    return Expr1, Expr2
# Розрахунок і виведення на екран розв'язків системи нелінійних рівнянь:
for i in range(n):
    r = fsolve(func, a[i]) # Список розв'язку системи
    print("Розв'язок", i + 1, ":")
    print("x [", i + 1, "] =", round(r[0], 4))
    print("y [", i + 1, "] =", round(r[1], 4))
    print()
```

Розв'язок 1 :

x [1] = -1.6682

y [1] = -1.1032

Розв'язок 2 :

x [2] = 1.548

y [2] = 1.2664

3) Розв'язуємо систему нелінійних рівнянь за допомогою функції `nsolve()` модуля `SymPy`:

```
import sympy as smp
x, y = smp.symbols('x y')
# Символьні рівняння системи:
Eq1 = smp.Eq(x ** 2 + y ** 2, 4)
Eq2 = smp.Eq(x + 2, smp.exp(y))
n = 2 # Кількість розв'язків системи нелінійних рівнянь
# Список початкових наближень розв'язків:
a = [[-1.5, -1], [1.5, 1]]
# Розрахунок і виведення на екран розв'язків системи нелінійних рівнянь:
for i in range(n):
    r = smp.nsolve([Eq1, Eq2], [x, y], a[i]) # Матриця розв'язку системи
    print("Розв'язок", i + 1, ":")
    print("x [", i + 1, "] =", round(r[0], 4))
    print("y [", i + 1, "] =", round(r[1], 4))
    print()
```

Розв'язок 1 :
x [1] = -1.6682
y [1] = -1.1032

Розв'язок 2 :
x [2] = 1.5480
y [2] = 1.2664

Відповідь: задана система нелінійних рівнянь має два розв'язки:

$$\begin{aligned}x_1 &= -1,6682, & y_1 &= -1,1032; \\x_2 &= 1,548, & y_2 &= 1,2664.\end{aligned}$$

Примітки:

1) Межі інтервалу побудови графіка (або графіків) обираються таким чином, щоб відслідковувались усі розв'язки рівняння і можливо було визначити їх початкові наближення.

2) Якщо задана система нелінійних рівнянь має безліч розв'язків, то необхідно знайти 4 розв'язки, які є найближчими до точки з координатами (0; 0).

Контрольні питання

1. За яких умов множина рівнянь є системою, а не сукупністю незалежних рівнянь?
2. Що є розв'язком системи рівнянь?
3. Яка умова повинна виконуватись, щоб система лінійних рівнянь мала єдиний розв'язок?
4. Чи може система лінійних рівнянь мати рівно два розв'язки?
5. Як систему лінійних рівнянь представити у вигляді матричного рівняння?
6. Як знайти розв'язок системи лінійних рівнянь у *Python*?
7. Чи є система рівнянь лінійною, якщо вона містить три лінійні рівняння і одне нелінійне?
8. Яка мета локалізації розв'язків системи нелінійних рівнянь?
9. Як графічно визначити початкові наближення розв'язків системи двох нелінійних рівнянь з двома невідомими?
10. Чи можливо застосувати функції *fsolve()* і *nsolve()* для розв'язання системи лінійних рівнянь?
11. Скільки значень містить початкове наближення розв'язку системи з п'яти нелінійних рівнянь із п'ятьма невідомими?
12. Як перевірити правильність розв'язку системи рівнянь?

Лабораторна робота № 5

АПРОКСИМАЦІЯ ТАБЛИЧНИХ ФУНКЦІЙ

Мета роботи: одержання практичних навичок глобальної та багатоінтервальної інтерполяції, а також середньоквадратичного наближення табличних функцій.

5.1 Загальні відомості про апроксимацію функцій

Апроксимація функції $f(x)$ полягає в знаходженні такого аналітичного виразу $\varphi(x)$, який наближено відтворює поведінку заданої функції $f(x)$ в певній області з мінімальною похибкою [18].

Апроксимація використовується, коли аналітичний вираз функції $f(x)$ невідомий, а є лише таблиця її значень, яка одержана, наприклад, експериментально. В цьому випадку обчислити значення функції в проміжних точках без додаткових вимірювань можливо за наближеним виразом $\varphi(x)$.

Якщо ж вираз $f(x)$ занадто складний і його обчислення потребує значних ресурсів, апроксимація дає змогу замінити його простішим виразом $\varphi(x)$ із мінімальною похибкою.

Метою апроксимації також може бути побудова графіка функції, яка задана таблично, згладжування експериментальних даних, побудова формул числового диференціювання і інтегрування тощо.

Зазвичай для апроксимації функцій використовують поліноми вигляду

$$\varphi(x) = a_0 + a_1x + \dots + a_nx^n = \sum_{i=0}^n a_i x^i, \quad (5.1)$$

де a_i – числові коефіцієнти, які визначаються при розв’язанні задачі апроксимації; n – порядок полінома.

Залежно від допустимого відхилення в заданих точках наближеної функції $\varphi(x)$ від вихідної $f(x)$ розрізняють задачі інтерполяції та середньоквадратичного наближення.

5.2 Глобальна інтерполяція табличних функцій

Табличною є функція, значення якої задані лише в окремих, дискретних точках, зазвичай у вигляді таблиці. Тобто, для певного набору значень аргументу x відомі відповідні значення функції $f(x)$ [19].

Інтерполяція полягає в знаходженні такого полінома $\varphi(x)$ (5.1), який в заданих точках x_j на відрізку $[a, b]$ набуває тих самих значень, що й функція $f(x)$:

$$\varphi(x_j) = f(x_j), \quad j = 0, 1, \dots, m, \quad (5.2)$$

$$x_0 = a,$$

$$x_m = b.$$

де x_j – вузли інтерполяції, $m + 1$ – кількість вузлів.

Інтерполяційний поліном $\varphi(x)$ проходить через табличні точки, а в інших точках відрізка $[a, b]$ наближує функцію $f(x)$ із деякою точністю.

Якщо один і той самий поліном $\varphi(x)$ наближує функцію $f(x)$ на всьому відрізку $[a, b]$, то така інтерполяція є глобальною. При багатоінтервальної інтерполяції для різних частин відрізка $[a, b]$ використовують різні поліноми.

Інтерполяційний поліном, який одержано для інтервалу $[a, b]$, можливо використовувати для розрахунків значень функції і поза цим інтервалом, тобто при $x < a$ або $x > b$. Таке наближення має назву екстраполяції.

Для знаходження невідомих коефіцієнтів a_i ($i = 0, 1, \dots, n$) інтерполяційного полінома використовується умова (5.2) і формується система з $m + 1$ лінійних алгебраїчних рівнянь.

Якщо рівняння такої системи лінійно незалежні (таблична функція не містить однакових вузлів) і кількість $n + 1$ невідомих коефіцієнтів полінома дорівнює кількості $m + 1$ вузлів ($n = m$), то система має єдиний розв'язок.

Таким чином, існує лише один поліном n -го або нижчого степеня, що проходить через $n + 1$ різних вузлів і приймає в них задані значення табличної функції. Коефіцієнти такого полінома a_i знаходять розв'язанням наступної системи алгебраїчних рівнянь:

$$\begin{cases} a_0 + a_1 x_0 + a_2 x_0^2 + \dots + a_n x_0^n = f(x_0), \\ a_0 + a_1 x_1 + a_2 x_1^2 + \dots + a_n x_1^n = f(x_1), \\ \dots \\ a_0 + a_1 x_n + a_2 x_n^2 + \dots + a_n x_n^n = f(x_n). \end{cases} \quad (5.3)$$

Основна матриця такої системи має назву матриці Вандермонда:

$$V = \begin{bmatrix} 1 & x_0 & x_0^2 & \dots & x_0^n \\ 1 & x_1 & x_1^2 & \dots & x_1^n \\ \dots & \dots & \dots & \dots & \dots \\ 1 & x_n & x_n^2 & \dots & x_n^n \end{bmatrix}. \quad (5.4)$$

Матриця-стовпець вільних членів системи містить значення табличної функції $f(x_0), f(x_1), \dots, f(x_n)$.

Водночас існує нескінченно багато поліномів степенів $n + 1$ та вищих, що проходять через $n + 1$ різних вузлів і приймають в них задані значення табличної функції.

Наприклад, через три точки можливо провести лише одну параболу, яка описується поліномом другого степеня, та нескінченну множину графіків поліномів вищих степенів. Якщо три точки лежать на одній прямій, то єдиний інтерполяційний поліном, графік якого проходить через них, має перший степінь. Якщо ця пряма є горизонтальною, то відповідний поліном має нульовий степінь.

В *Python* для розв'язання задачі інтерполяції і середньоквадратичного наближення табличних функцій використовується функція *polyfit()* модуля *NumPy*:

numpy.polyfit(x, y, deg)

де *x* – колекція вузлів табличної функції, *y* – колекція значень функції в цих вузлах; *deg* – степінь полінома, який використовується для апроксимації.

Для одержання коефіцієнтів інтерполяційного полінома значення *deg* має дорівнювати $n - 1$, де n – кількість вузлів табличної функції.

Функція повертає матрицю-рядок з коефіцієнтами полінома, в якій першим елементом є коефіцієнт при найвищому степені аргументу.

Для створення полінома за списком його коефіцієнтів використовується функція *poly1d()* модуля *NumPy*.

Розрахувати значення полінома для заданих значень аргументу можливо з використанням функції *polyval()* модуля *NumPy*:

numpy.polyval(p, x)

де *p* – колекція коефіцієнтів полінома (першим вказується коефіцієнт при найвищому степені аргументу), або результати, які повертаються функціями *polyfit()* та *poly1d()*; *x* – значення або колекція значень аргументу, за яких розраховуються значення полінома.

Функція повертає значення або матрицю зі значеннями полінома, розмірність якої відповідає розмірності параметра *x*.

5.3 Інтерполяційні сплайни

Степінь інтерполяційного полінома залежить від кількості вузлів. При інтерполяції на великих відрізках з великими відстанями між вузлами точність наближення є низькою. Зі зменшенням відстані між вузлами збільшується їх кількість і степінь полінома, що призводить до збільшення часу, обсягу і похибок обчислень [20].

В таких випадках доцільно застосовувати багатоінтервальну інтерполяцію, зокрема, інтерполяцію сплайнами.

Сплайном є кускова функція, яка разом із певною кількістю своїх похідних є неперервною на всьому відрізку $[a, b]$, а на кожному підінтервалі $[x_k,$

$x_{k+1}]$ описується певним поліномом. Підінтервал може охоплювати кілька вузлів в залежності від степеня поліномів.

Максимальний степінь поліномів на всіх підінтервалах є порядком сплайна.

Для багатоінтервальної інтерполяції в *Python* можливо використати функцію *interp1d()* модуля *SciPy*:

`scipy.interpolate.interp1d(x, y, kind = 'linear')`

де x – колекція вузлів табличної функції, y – колекція значень функції в цих вузлах; *kind* – тип інтерполяції, який визначає степінь інтерполяційного полінома на кожному підінтервалі. Може приймати наступні значення:

- 'zero' – ступінчата інтерполяція, поліном нульового степеня;
- 'linear' або 'slinear' – лінійна інтерполяція, поліном першого степеня;
- 'quadratic' – квадратична інтерполяція, сплайн другого порядку;
- 'cubic' – кубічна інтерполяція, сплайн третього порядку.

Функція *interp1d()* повертає об'єкт, який можна викликати як звичайну *Python*-функцію. Ця функція розраховує значення полінома або сплайна для значення аргументу, яке вказується як параметр.

5.4 Середньоквадратичне наближення табличних функцій

Інтерполяційні поліноми і сплайни приймають в вузлах ті ж самі значення, що й таблична функція. Однак, якщо табличні дані містять похибки округлення чи вимірювання, результати розрахунків за інтерполяційними поліномами точно відтворюють ці похибки.

Тому в деяких випадках для апроксимації табличної функції обирають поліном нижчого степеня, а коефіцієнти розраховують таким чином, щоб графік полінома проходив не через табличні точки, а поряд із ними з мінімальним відхиленням [11].

Така задача має назву середньоквадратичного наближення табличної функції. При її вирішенні за допомогою методу найменших квадратів сумарне відхилення наближеного полінома $\varphi(x)$ від табличної функції $f(x)$ в вузлах x_j розраховують за наступною формулою:

$$g = \sum_{j=0}^m (\varphi(x_j) - f(x_j))^2. \quad (5.5)$$

Таким чином, задача середньоквадратичного наближення табличної функції методом найменших квадратів полягає в знаходженні таких числових значень коефіцієнтів $a_0, a_1, \dots, a_n$ полінома $\varphi(x)$ степеня n , за яких досягається мінімум функції g :

$$g(a_0, a_1, \dots, a_n) = \sum_{j=0}^m (a_0 + a_1 x_j + \dots + a_n x_j^n - f(x_j))^2 \rightarrow \min. \quad (5.6)$$

Розв'язати задачу (5.6) в *Python* можливо з використанням функції *polyfit()* модуля *NumPy*, синтаксис якої розглянуто в розділі 5.2. Значення параметру *deg* повинно відповідати порядку *n* полінома, який використовується для середньоквадратичного наближення функції.

Функція повертає матрицю-рядок з коефіцієнтами наближеного полінома, кількість яких становить $n + 1$. Першим елементом в матриці є коефіцієнт при найвищому степені аргументу.

5.5 Завдання до лабораторної роботи

Завдання 5.1

Побудуйте графік заданої табличної функції.

Наведіть її інтерполяційний поліном в загальному вигляді. Визначте коефіцієнти полінома за допомогою функції *polyfit()* модуля *NumPy*.

За одержаним інтерполяційним поліномом розрахуйте значення функції в проміжних точках x_{01} , x_{02} і x_{03} .

Побудуйте графік інтерполяційного полінома, на графіку покажіть табличні і проміжні точки.

j	x_j	$f(x_j)$				
		1	2	3	4	5
0	0,1	0,0998	0,995	1,105	0,905	0,1
1	0,5	0,479	0,878	1,649	0,606	0,521
2	0,8	0,717	0,696	2,226	0,449	0,888
3	1,3	0,964	0,278	3,669	0,273	1,698
4	1,8	0,973	-0,277	6,05	0,1653	2,942
5	2,6	0,516	-0,857	13,464	0,0743	6,695
$x_{01} = 0,2$		$x_{02} = 1,4$		$x_{03} = 2,5$		

j	x_j	$f(x_j)$				
		6	7	8	9	10
0	0,1	0,0998	0,995	1,105	0,905	0,1
1	0,6	0,565	0,825	1,822	0,549	0,637
2	1,2	0,932	0,362	3,32	0,301	1,509
3	1,8	0,974	-0,227	6,05	0,165	2,942
4	2,6	0,516	-0,857	13,463	0,0743	6,694
5	3	0,141	-0,99	20,086	0,0498	10,018
$x_{01} = 0,3$		$x_{02} = 1,5$		$x_{03} = 2,8$		

j	x_j	$f(x_j)$				
		11	12	13	14	15
0	0,3	0,296	0,955	1,35	0,741	0,305
1	0,9	0,783	0,621	2,46	0,407	1,267
2	1,5	0,997	0,0708	4,481	0,223	2,129
3	2	0,909	-0,416	7,389	0,135	3,627
4	2,5	0,598	-0,801	12,183	0,082	6,05
5	3,1	0,0416	-0,999	22,198	0,045	11,077

$x_{01} = 0,5$ $x_{02} = 1,7$ $x_{03} = 2,9$

j	x_j	$f(x_j)$				
		16	17	18	19	20
0	1	0,842	0,54	2,748	0,368	1,175
1	1,5	0,998	0,0707	4,482	0,223	2,129
2	2,3	0,746	-0,667	9,974	0,1	4,367
3	3	0,141	-0,99	20,086	0,0498	10,018
4	3,6	-0,443	-0,897	36,398	0,0273	18,289
5	4,5	-0,978	-0,211	90,017	0,0111	45,003

$x_{01} = 1,2$ $x_{02} = 2,7$ $x_{03} = 4,3$

j	x_j	$f(x_j)$				
		21	22	23	24	25
0	1	0,841	0,54	2,748	0,368	1,175
1	1,6	0,99	-0,29	4,953	0,202	2,376
2	2,5	0,598	-0,801	12,183	0,0821	6,05
3	3,1	0,0416	-0,99	22,198	0,0451	11,077
4	3,8	-0,612	-0,791	44,701	0,0224	22,339
5	4,5	-0,978	-0,211	90,017	0,0111	45,003

$x_{01} = 1,2$ $x_{02} = 2,7$ $x_{03} = 4,3$

j	x_j	$f(x_j)$				
		26	27	28	29	30
0	0,1	2,96	0,995	40,415	0,368	0,123
1	0,6	3,45	0,362	29,456	0,202	10,678
2	1,5	8,41	0,0416	28,482	0,0451	1,632
3	2,3	9,81	0,878	13,389	0,621	8,977
4	3,1	15,16	0,415	2,183	0,0111	3,561
5	3,8	19,12	0,154	0,339	0,1	5,145

$x_{01} = 0,5$ $x_{02} = 1,9$ $x_{03} = 3,6$

Завдання 5.2

Виконайте інтерполяцію табличної функції, яка наведена в завданні 5.1, сплайнами нульового, першого, другого і третього порядків.

За одержаними сплайнами розрахуйте значення функції в проміжних точках x_{01} , x_{02} і x_{03} .

Побудуйте графіки сплайнів, на графіках покажіть табличні і проміжні точки.

Завдання 5.3

Розв'яжіть задачу середньоквадратичного наближення заданої табличної функції з використанням поліномів першого, другого, третього та четвертого степенів.

Наведіть одержані наближені поліноми.

Покажіть на графіках табличні точки і побудуйте наближені криві.

1		2		3		4	
x_j	f_j	x_j	f_j	x_j	f_j	x_j	f_j
0	-5,86	-100	-22,19	-80	-15,94	-60	-10,68
16,54	1,52	-76,05	7,83	-76,82	-4,17	-36,11	3,82
17,02	6,14	-75,94	24,73	-58,06	4,68	-20,66	14,27
17,69	8,43	-64,49	24,42	-25,35	5,61	-12,75	21,14
17,77	8,75	-36,85	28,67	-19,35	13,06	-11,48	11,7
27,54	7,84	-25,29	34,36	-13,81	14,5	-10	15,29
34,57	6,49	-12,68	38,06	1,27	13,49	-6,02	15,48
36,77	4,03	-0,77	39,32	3,76	16,85	-5,85	28,9
48,41	4,12	4,98	31,36	5,26	10,35	1,8	23,13
49,06	1,9	19,46	29,08	13,88	11,74	17,16	19,37
49,99	1,53	35,46	33,57	23,22	16,08	21,89	20,36
58,41	0,36	36,83	34,87	27,33	19,06	27,43	11,99
61,03	-0,62	41,7	32,21	28,51	12,39	31,7	20,17
65,58	-1,18	41,96	27,59	32,85	12,86	32,04	17,03
69,92	-1,98	46,4	24,49	49,01	15,22	39,93	8,57
77,75	-3,84	59,21	28,87	62,73	20,21	42,81	2,79
78,43	-4,4	64,86	28,59	62,78	16,69	46,36	-8,08
87,96	-4,4	67,16	33,67	66,5	40,37	48,52	-16,38
91,02	-3,66	80,6	26,59	67,67	41,23	54,64	-31,93
100	0,1	100	20,13	80	82,15	60	-87,42

5		6		7		8	
x_j	f_j	x_j	f_j	x_j	f_j	x_j	f_j
-40	-57,58	-20	15,85	-200	2,81	-20	-59,95
-30,62	-53,64	-19,98	8,38	-192,4	-1,39	-19,98	-44,24
-27,64	-41,89	-18	12,72	-84,42	-4,55	-18,58	-16,98
-26,77	-30,28	-17,62	11,77	-72,98	-6,07	-18,27	-20,38
-15,45	-16,3	-6,98	7,53	-68,13	-13,98	-18,15	-14,4
-13,62	-11,94	-5,58	9,31	-68,04	-10,85	-15,41	-13,72
-11,8	-17,98	-0,51	4,68	-55,44	-19,71	-14,56	-14,45
3,74	-10,6	2,69	5,58	-47,38	-17,79	-13,67	-12,38
7,16	-8,85	4,35	5,11	37,34	-18,48	-13,1	-6,63
7,52	-7,52	5,05	5,95	40,14	-22,71	-10,43	-11,81
9,97	-6,91	7,92	6,09	56,53	-23,98	-8,79	-8,17
11,07	-1,94	11,66	10,73	56,54	-15,85	-8,56	-5,8
14,35	-0,45	12,74	10,75	108,04	-21,57	-7,66	-5,08
21	2,79	14,34	9,8	121,88	-12,88	-7,51	-7,67
32,2	7,19	14,58	12,91	126,02	-11,53	-6,65	-7,04
36,39	14,49	14,89	8,45	138,69	-13,67	-6,32	-5,81
36,82	35,26	15,87	8,07	139,06	-4,61	-2,17	-6,67
37,75	31,43	19,2	10,05	156,11	2,15	-0,84	-16,87
38,31	62,75	19,25	13,39	174,66	19,09	-0,34	-11,24
40	133,8	20	24,67	200	42,92	0	-21,33

9		10		11		12	
x_j	f_j	x_j	f_j	x_j	f_j	x_j	f_j
-1	44,61	20	-17,45	40	-1,25	0	231,37
-0,96	27,62	21,84	-9,37	40,05	-4,3	2,66	123,73
-0,95	30,35	25,86	-13,53	41,97	-9,31	3,97	95,63
-0,92	49,29	37,98	-8,05	42,04	-11,65	6,56	91,32
-0,86	28,51	38,84	-11,23	42,09	-20,75	30,13	50,91
-0,81	24,21	48,05	-6,34	59,55	-38,66	31,71	82,96
-0,8	40,42	56	-6,71	61,61	-30,51	32,35	67,04
-0,54	32,92	57,33	-6,11	61,7	-54,04	33,69	44,09
-0,42	30,58	58,43	-2,19	67,45	-55,68	38,92	76,69
-0,23	28,12	61,81	0,74	68,59	-31,7	39,9	79,95
-0,14	31,11	62,58	2,74	68,86	-58,34	41,95	61,14
0,03	18,51	67,53	5,55	73,66	-36,72	46,2	43,42

0,11	34,54	78,07	5,83	76,67	-39,47	49,57	99,2
0,62	29,84	80,92	11,29	81,43	-47,22	64,87	65,88
0,66	27,39	85,34	12,6	82,49	-75,82	68,67	81,16
0,82	10,56	90,86	9,41	82,72	-101,3	70,08	103,75
0,83	1,03	92,91	13,39	83,95	-83,86	71,14	109,65
0,85	-18,62	93,7	14,24	95,52	-63,72	79,43	58,82
0,99	-25,43	98,52	6,29	97,15	-30,22	79,95	61,68
1	-81,83	100	5,42	100	37,71	80	40,01

13		14		15		16	
x_j	f_j	x_j	f_j	x_j	f_j	x_j	f_j
0	5,85	0	72,27	0	46,67	0	177,81
0,93	12,12	3,94	61,79	0,12	44,83	1,1	111,61
19,17	8,92	6,92	46	0,43	22,51	1,53	77,76
19,87	11,6	10,04	28,99	2,12	24,22	1,61	59,16
22,66	10,92	11,8	32,23	3,7	25,14	2,09	43
22,71	11,44	15,66	30,05	4,72	17,88	3,8	32,93
23,47	11,88	17,2	21,2	5,35	23,44	3,88	28,27
28,67	15,82	20,25	24,51	6,05	30,7	4,41	24,23
28,83	11,82	22,8	23,52	7,06	30,11	4,87	24,48
29,18	17,65	23,48	28,15	8,83	18,08	5,41	28,23
30,21	16,26	24,99	28,13	12,04	17,82	5,66	30,57
36,76	21,24	28,23	11,64	13,11	11,89	5,78	28,8
44,52	12,36	28,53	17,22	14,49	5,24	5,95	30,03
47,99	13,81	32,8	5,2	15,66	-0,85	6,2	33,44
49,55	17,22	33,07	-1	15,9	-5,6	8,07	39,24
50,37	23,53	33,23	-4,21	16,46	-14,33	8,44	40,89
51,71	29,05	34,75	-4,07	16,59	-13,57	8,66	35,9
52,42	12,07	35,92	8,23	17,15	-7,44	9,09	41,45
56,08	8,13	37,95	48,78	17,93	14,27	9,24	51,47
60	-8,64	40	137,79	20	140,64	10	72,74

17		18		19		20	
x_j	f_j	x_j	f_j	x_j	f_j	x_j	f_j
0	-22,92	0	169,85	-40	-30,59	-60	23,11
0,02	-22,43	10,05	85,6	-29,43	-18,63	-54,62	-18,91
0,05	-22,44	35,23	59,23	-28,82	-14,79	-52,99	-32,43
0,1	-23,63	45,57	42,74	-28,25	-15,53	-51,58	-37,64
0,17	-22,3	58,09	43,37	-27,7	-12,69	-48,95	-28,3
0,23	-24,49	62,39	52,18	-25,43	-17,39	-45,2	-27,31
0,3	-32,1	81,19	50	-24,83	-14,4	-41,76	-16,24
0,39	-34,46	89,43	63,81	-22,35	-18,56	-29,89	-8,07
0,47	-39,09	115,35	64,91	-17,42	-18,7	-26,16	-2,05
0,5	-41,11	117,44	79,81	-15,66	-18,58	-25,61	5,16
0,51	-36,98	120,15	73,92	-14,21	-13,27	-16,27	10,65
0,6	-44,31	125,9	76,67	-12,33	-14,49	-14,65	12,48
0,61	-37,37	129,26	86,2	-12,24	-17,51	-10,98	11,76
0,64	-36,17	138,21	87,27	-11,51	-20,58	-10,05	13,25
0,71	-31,54	154,94	92	-4,72	-17,4	-7,31	6,63
0,78	-21,11	175,97	81,68	-4,68	-21,43	-5,44	0,58
0,9	-3,73	184,01	64,98	-3,4	-22,18	-5,14	-8,98
0,92	22,01	185,02	43,6	-2,02	-40,22	-4,77	-29,11
0,98	60,3	185,58	-16,49	-1,6	-46,74	-0,06	-44,82
1	98,2	200	-136,3	0	-76,34	0	-50,55

21		22		23		24	
x_j	f_j	x_j	f_j	x_j	f_j	x_j	f_j
-80	-41,07	-100	173,67	0	155,49	-2	-29,83
-79,42	-50,88	-99,2	83,79	28,91	54,88	-1,4	-10,72
-76,48	-49,08	-95,74	49,05	54,84	18,06	-1,36	-3,39
-74,49	-60,77	-94,97	23,46	61,65	-0,21	-1,3	-1,44
-55,41	-62,38	-84,72	13,88	104,03	-6,69	-1,12	-1,07
-43,54	-67,27	-84,06	6,11	105,46	-8,78	-1,05	-1,09
-37,52	-59,37	-81,95	1,9	109,27	-8,56	-0,82	-1,35
-34,42	-55,34	-77,63	-1,72	110,78	-8,5	-0,8	-1,28
-26,82	-67,57	-72,25	-5,37	111,21	-6,38	-0,72	-0,99
-26,78	-47,25	-72,13	-7,86	120,65	-5,76	-0,54	-0,34
-20,05	-45,02	-64,73	-11,05	140,9	-3,56	0,68	0,4
-16,92	-51,82	-64,65	-14,43	170,5	-2,29	0,83	1,52

-15,03	-42,94	-61,59	-18,16	179,57	-0,58	0,85	2,33
-11,63	-28,26	-43,32	-18,34	187,32	1,76	1,17	3,93
-11,1	-27,39	-42,98	-17,21	189,86	4,49	1,48	4,74
-10,58	-21,55	-32,79	-19,01	241,1	8,31	1,53	7,03
-9,08	-26,44	-19,01	-18,43	249,92	13,42	1,63	7,22
-5,85	-31,01	-12,48	-22,58	258,67	21	1,91	9,21
-4,94	-59,24	-9,88	-35,73	319,95	31,21	1,95	10,86
0	-206,5	0	-73,71	400	42,3	2	8,34

25		26		27		28	
x_j	f_j	x_j	f_j	x_j	f_j	x_j	f_j
60	2,47	80	-67,98	-10	-10,58	0	7,81
60,68	-0,07	82,33	-44,37	-9,47	-5,43	0,22	13
62,79	-1,44	83,64	-38,28	-8,44	16,55	0,66	13,73
63,61	-2,06	83,93	-20,03	-8,39	12,48	1,15	11,89
64,45	-2,7	84,68	-10,28	-7,73	27,38	1,24	8,89
66,07	-2,79	85,93	3	-7,69	18,9	1,54	7,68
66,27	-2,4	86,33	13,86	-7,44	18,29	1,67	7,78
70,3	-2,65	86,41	24,63	-6,82	14,28	1,69	6,64
72,21	-2,55	89,09	30,65	-5,04	11,15	1,78	5
73,18	-2,39	90,58	47,89	-5,01	8,56	1,84	4,11
75,24	-2,47	91,27	49,87	-4,73	8,36	1,99	4,71
77,28	-2,85	94,37	66,59	-4,03	4,86	2,16	3,26
79,24	-3	94,76	64,74	-3,71	6,56	2,18	3,07
85,87	-3,43	95,17	59,98	-3,14	5,67	2,2	3,64
87,01	-4,2	95,64	83,57	-2,96	3,18	2,71	2,73
89,88	-4,77	96,5	77,66	-2,18	1,55	3,05	3,37
92,04	-7	97,08	54,42	-1,07	0,66	3,46	4,67
93,1	-9,44	97,52	15	-0,54	-0,37	3,83	5,72
96,58	-10,99	99,87	-47,21	-0,33	-1,27	3,93	6,89
100	-16,14	100	-118,1	0	0,13	5	11,06

29		30	
x_j	f_j	x_j	f_j
-5	55,49	-10	-139,7
-4,98	23,62	-9,93	-30,66
-4,41	10,81	-8,21	-18,04
-2,98	9,7	-7,84	-16,31
-1,57	13,74	-7,66	-9,42
-1,23	17,92	-7,61	-8,09
-0,86	26,47	-6,74	-9,5
-0,82	33,74	-4,76	-9,23
-0,65	42,04	-4,18	-6,82
-0,45	31,52	-4,04	-3,72
0,03	36,23	-3,79	-7,3
0,73	36,33	-3,36	-21
1,56	39,88	-1,47	-21,02
2,4	39,83	-0,85	-15,89
2,85	29,92	-0,24	-20,56
3,77	34,97	3,43	-23,2
4,36	24,98	5,05	-72,05
4,91	24,93	7,27	-40,04
4,94	15,37	9,27	-59,42
5	9,08	10	-69,5

5.6 Приклади розв'язання завдань

Завдання 5.1

Побудуйте графік заданої табличної функції.

Наведіть її інтерполяційний поліном в загальному вигляді. Визначте коефіцієнти полінома за допомогою функції *polyfit()* модуля *NumPy*.

За одержаним інтерполяційним поліномом розрахуйте значення функції в проміжних точках x_{01} , x_{02} і x_{03} .

Побудуйте графік інтерполяційного полінома, на графіку покажіть табличні і проміжні точки.

j	x_j	$f(x_j)$
0	1,1	1,259
1	2,2	4,267
2	2,8	8,0248
3	3,1	15,264
4	4,5	18,0633
5	4,8	22,449
$x_{01} = 1,4$		$x_{02} = 2,5$
		$x_{03} = 4$

Розв'язання

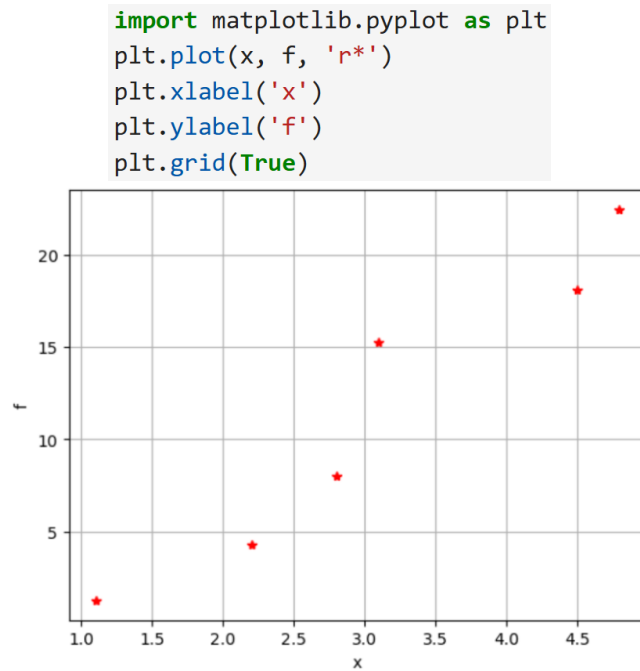
1) Створюємо матриці, що містять вузли та відповідні значення табличної функції в цих вузлах. Виводимо на екран задану табличну функцію:

```
import numpy as np
x = np.array([1.1, 2.2, 2.8, 3.1, 4.5, 4.8])
f = np.array([1.259, 4.267, 8.0248, 15.264, 18.0633, 22.449])
print('Задана таблична функція:')
for i in range(len(x)):
    print('f (', x[i], ') =', f[i])
```

Задана таблична функція:

```
f ( 1.1 ) = 1.259
f ( 2.2 ) = 4.267
f ( 2.8 ) = 8.0248
f ( 3.1 ) = 15.264
f ( 4.5 ) = 18.0633
f ( 4.8 ) = 22.449
```

Будуємо графік табличної функції:



2) Таблична функція містить 6 різних вузлів. Для даної функції існує єдиний інтерполяційний поліном п'ятого степеня:

$$IPol(x) = a_0x^5 + a_1x^4 + a_2x^3 + a_3x^2 + a_4x + a_5.$$

Розраховуємо коефіцієнти інтерполяційного полінома:

```
n = len(x) # Кількість вузлів
# Матриця коефіцієнтів інтерполяційного полінома:
a = np.polyfit(x, f, n - 1)
for i in range(n): # Виведення результатів на екран
    print('a [', i, '] = ', round(a[i], 3))
```

```
a [ 0 ] = 4.391
a [ 1 ] = -66.971
a [ 2 ] = 387.615
a [ 3 ] = -1053.039
a [ 4 ] = 1332.46
a [ 5 ] = -615.204
```

За списком коефіцієнтів створюємо інтерполяційний поліном:

```
IPol = np.poly1d(a)
print('Інтерполяційний поліном IPol(x) =')
print(IPol)
```

```
Інтерполяційний поліном IPol(x) =
      5      4      3      2
4.391 x - 66.97 x + 387.6 x - 1053 x + 1332 x - 615.2
```

3) Розраховуємо значення функції в проміжних точках:

```
x0 = np.array([1.4, 2.5, 4]) # Матриця з проміжними значеннями аргументу
# Матриця зі значеннями функції в проміжних точках:
f0 = np.polyval(IPol, x0)
for i in range(len(x0)): # Виведення результатів на екран
    print('f [', x0[i], ']' '=', round(f0[i], 3))
```

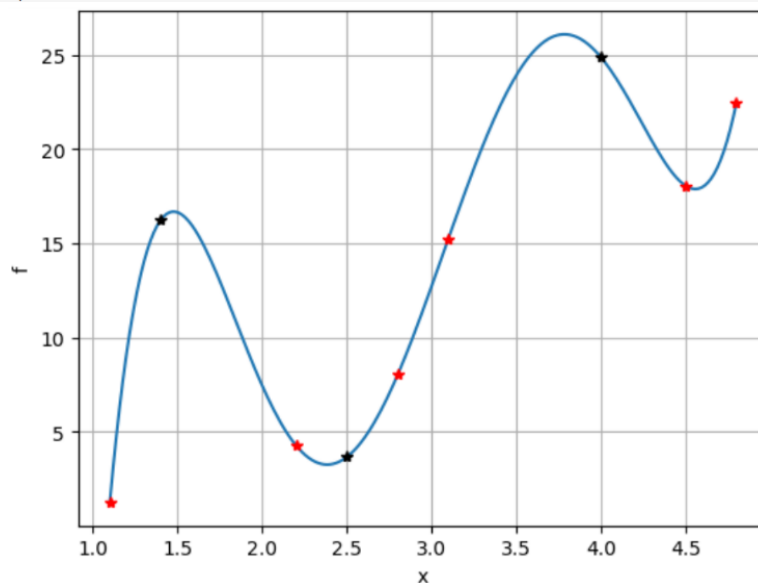
f [1.4] = 16.237

f [2.5] = 3.66

f [4.0] = 24.89

4) Будуємо графік інтерполяційного полінома, позначаємо вузлові і проміжні точки:

```
m = 200 # Кількість точок, за якими будується графік полінома
x1 = np.linspace(x[0], x[len(x) - 1], m) # Матриця зі значеннями аргументу
# Матриця зі значеннями інтерполяційного полінома:
f1 = np.polyval(IPol, x1)
# Побудова графіка інтерполяційного полінома.
# Позначення вузлових і проміжних точок:
plt.plot(x1, f1, x, f, 'r*', x0, f0, 'k*')
plt.xlabel('x')
plt.ylabel('f')
plt.grid(True)
```



Відповідь: Інтерполяційний поліном:

$$IPol(x) = 4,391x^5 - 66,97x^4 + 387,6x^3 - 1053x^2 + 1332x - 615,2.$$

Значення функції в проміжних точках:

$$f(1,4) = 16,237;$$

$$f(2,5) = 3,66;$$

$$f(4) = 24,89.$$

Завдання 5.2

Виконайте інтерполяцію табличної функції сплайнами нульового, першого, другого і третього порядків.

За одержаними сплайнами розрахуйте значення функції в проміжних точках x_{01} , x_{02} і x_{03} .

Побудуйте графіки сплайнів, на графіках покажіть табличні і проміжні точки.

j	x_j	$f(x_j)$
0	1,1	1,259
1	2,2	4,267
2	2,8	8,0248
3	3,1	15,264
4	4,5	18,0633
5	4,8	22,449
$x_{01} = 1,4$		$x_{02} = 2,5$
		$x_{03} = 4$

Розв'язання

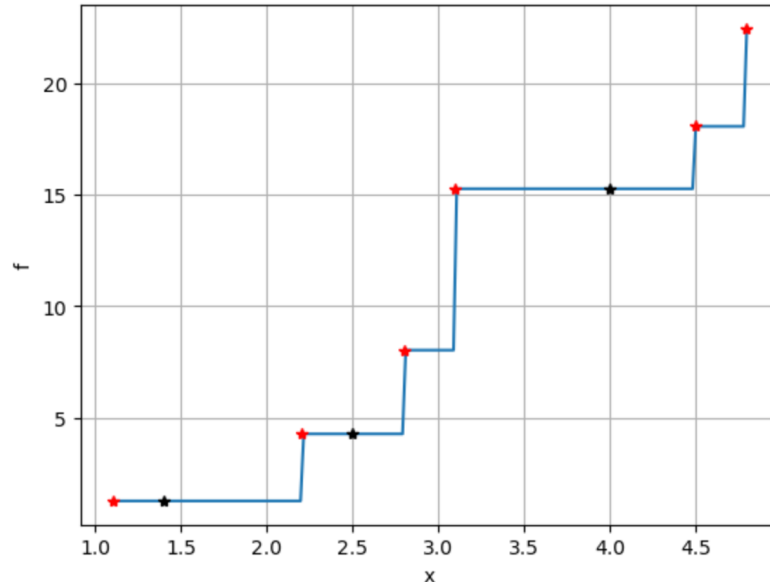
1) Виконуємо інтерполяцію табличної функції сплайном нульового порядку і розраховуємо значення функції в проміжних точках:

```
import numpy as np
from scipy.interpolate import interp1d
import matplotlib.pyplot as plt
# Матриці, що містять вузли і значення функції:
x = np.array([1.1, 2.2, 2.8, 3.1, 4.5, 4.8])
f = np.array([1.259, 4.267, 8.0248, 15.264, 18.0633, 22.449])
# Функція для розрахунку значень сплайна нульового порядку:
Sp0 = interp1d(x, f, 'zero')
x0 = np.array([1.4, 2.5, 4]) # Матриця з проміжними значеннями аргументу
for i in range(len(x0)): # Розрахунок і виведення результатів на екран
    print('f (', x0[i], ') =', np.round(Sp0(x0[i]), 3))

f ( 1.4 ) = 1.259
f ( 2.5 ) = 4.267
f ( 4.0 ) = 15.264
```

Будуємо графік інтерполяційного сплайна нульового порядку, позначаємо вузлові і проміжні точки:

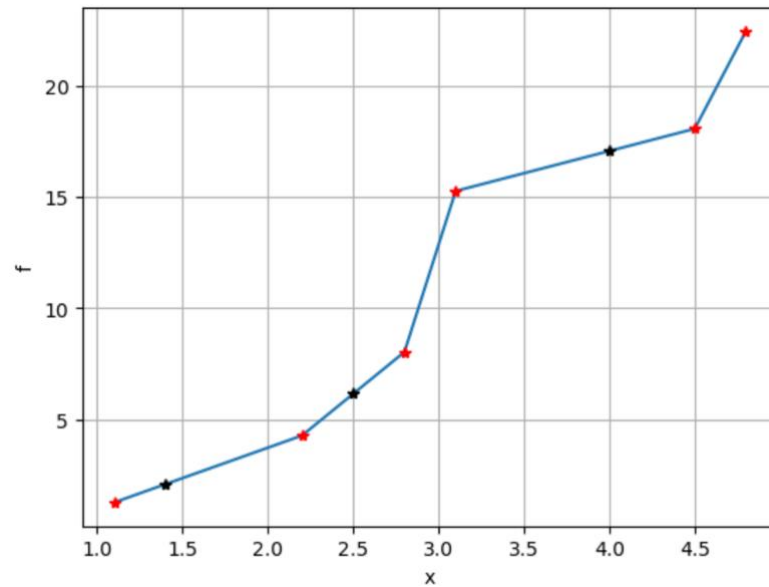
```
m = 200 # Кількість точок, за якими будується графік сплайна
x1 = np.linspace(x[0], x[len(x) - 1], m) # Матриця зі значеннями аргументу
plt.plot(x1, Sp0(x1), x, f, 'r*', x0, Sp0(x0), 'k*') # Побудова графіка
plt.xlabel('x')
plt.ylabel('f')
plt.grid(True)
```



2) Виконуємо інтерполяцію табличної функції сплайном першого порядку, розраховуємо значення функції в проміжних точках і будуємо графік:

```
# Функція для розрахунку значень сплайна першого порядку:
Sp1 = interp1d(x, f, 'slinear')
for i in range(len(x0)): # Розрахунок і виведення результатів на екран
    print('f (', x0[i], ') =', np.round(Sp1(x0[i]), 3))
plt.plot(x1, Sp1(x1), x, f, 'r*', x0, Sp1(x0), 'k*') # Побудова графіка
plt.xlabel('x')
plt.ylabel('f')
plt.grid(True)
```

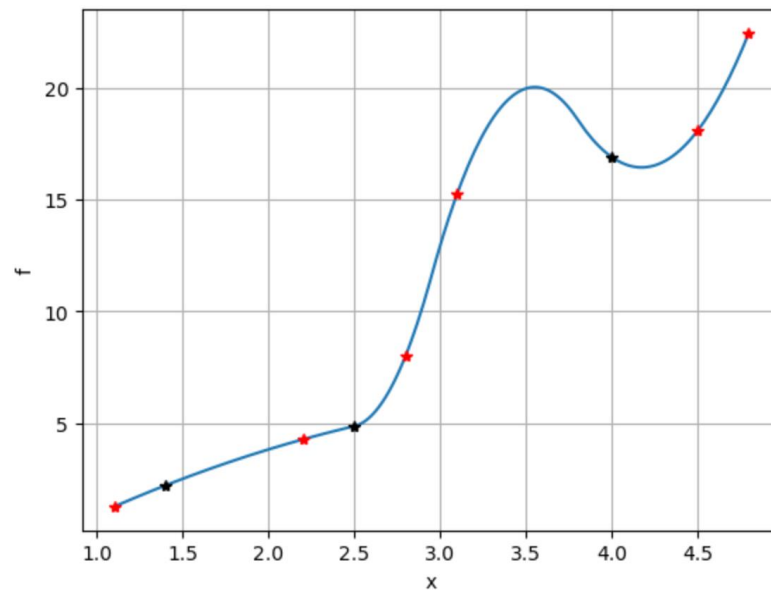
```
f ( 1.4 ) = 2.079
f ( 2.5 ) = 6.146
f ( 4.0 ) = 17.064
```



3) Виконуємо інтерполяцію табличної функції сплайном другого порядку, розраховуємо значення функції в проміжних точках і будуємо графік:

```
# Функція для розрахунку значень сплайна другого порядку
Sp2 = interp1d(x, f, 'quadratic')
for i in range(len(x0)): # Розрахунок і виведення результатів на екран
    print('f (', x0[i], ') =', np.round(Sp2(x0[i]), 3))
plt.plot(x1, Sp2(x1), x, f, 'r*', x0, Sp2(x0), 'k*') # Побудова графіка
plt.xlabel('x')
plt.ylabel('f')
plt.grid(True)
```

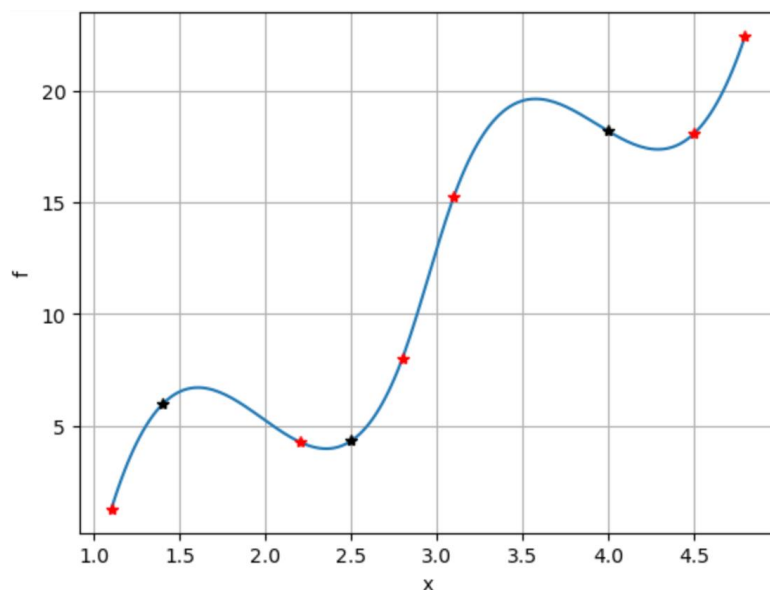
```
f ( 1.4 ) = 2.211
f ( 2.5 ) = 4.857
f ( 4.0 ) = 16.909
```



4) Виконуємо інтерполяцію табличної функції сплайном третього порядку, розраховуємо значення функції в проміжних точках і будуємо графік:

```
# Функція для розрахунку значень сплайна третього порядку:
Sp3 = interp1d(x, f, 'cubic')
for i in range(len(x0)): # Розрахунок і виведення результатів на екран
    print('f (', x0[i], ') =', np.round(Sp3(x0[i]), 3))
plt.plot(x1, Sp3(x1), x, f, 'r*', x0, Sp3(x0), 'k*') # Побудова графіка
plt.xlabel('x')
plt.ylabel('f')
plt.grid(True)
```

```
f ( 1.4 ) = 5.965
f ( 2.5 ) = 4.315
f ( 4.0 ) = 18.189
```



Завдання 5.3

Розв'яжіть задачу середньоквадратичного наближення заданої табличної функції з використанням поліномів першого, другого, третього та четвертого степенів.

Наведіть одержані наближені поліноми.

Покажіть на графіках табличні точки і побудуйте наближені криві.

x_j	f_j
0	11,42
10	12,6
20	10,55
30	11,02
40	10,92
50	9,87
60	10,05
70	10,45
80	10,88
90	11,04
100	10,49
110	9,22
120	9,02
130	8,76
140	8,31
150	7,54
160	6,86
170	6,75
180	8,12
190	10,67

Розв'язання

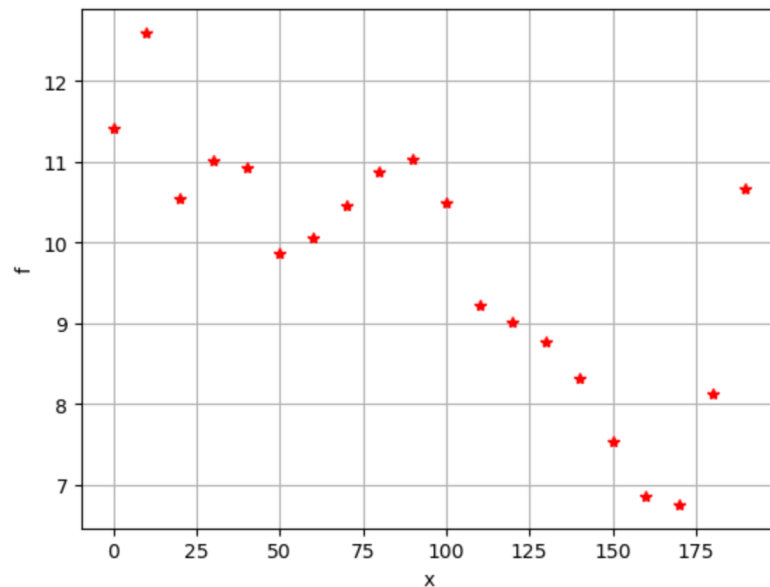
1) Створюємо матриці, що містять вузли та відповідні значення табличної функції в цих вузлах. Виводимо на екран задану табличну функцію:

```
import numpy as np
x = np.array([0, 10, 20, 30, 40, 50, 60, 70, 80, 90, 100,
              110, 120, 130, 140, 150, 160, 170, 180, 190])
f = np.array([11.42, 12.6, 10.55, 11.02, 10.92, 9.87, 10.05,
              10.45, 10.88, 11.04, 10.49, 9.22, 9.02, 8.76,
              8.31, 7.54, 6.86, 6.75, 8.12, 10.67])
for i in range(len(x)):
    print('f (', x[i], ') = ', f[i])
```

```
f ( 0 ) = 11.42
f ( 10 ) = 12.6
f ( 20 ) = 10.55
f ( 30 ) = 11.02
f ( 40 ) = 10.92
f ( 50 ) = 9.87
f ( 60 ) = 10.05
f ( 70 ) = 10.45
f ( 80 ) = 10.88
f ( 90 ) = 11.04
f ( 100 ) = 10.49
f ( 110 ) = 9.22
f ( 120 ) = 9.02
f ( 130 ) = 8.76
f ( 140 ) = 8.31
f ( 150 ) = 7.54
f ( 160 ) = 6.86
f ( 170 ) = 6.75
f ( 180 ) = 8.12
f ( 190 ) = 10.67
```

Будуємо графік заданої табличної функції:

```
import matplotlib.pyplot as plt
plt.plot(x, f, 'r*')
plt.xlabel('x')
plt.ylabel('f')
plt.grid(True)
```



2) Створюємо і виводимо на екран наближений поліном I-го степеня:

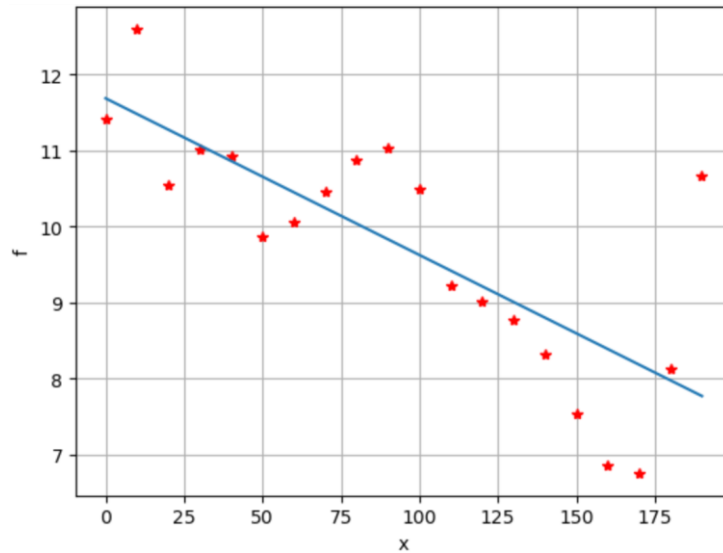
```
a = np.polyfit(x, f, 1) # Матриця коефіцієнтів наближеного полінома
APol_I = np.poly1d(a)
print('Наближений поліном I-го степеня:')
print(APol_I)
```

Наближений поліном I-го степеня:

$-0.02063 x + 11.69$

Будуємо графік наближеного полінома I-го степеня:

```
m = 200 # Кількість точок, за якими будується графік
x1 = np.linspace(x[0], x[len(x) - 1], m) # Матриця зі значеннями аргументу
f1 = np.polyval(APol_I, x1) # Матриця зі значеннями наближеного полінома
# Побудова графіка наближеного полінома, позначення вузлових точок:
plt.plot(x, f, 'r*', x1, f1)
plt.xlabel('x')
plt.ylabel('f')
plt.grid(True)
```

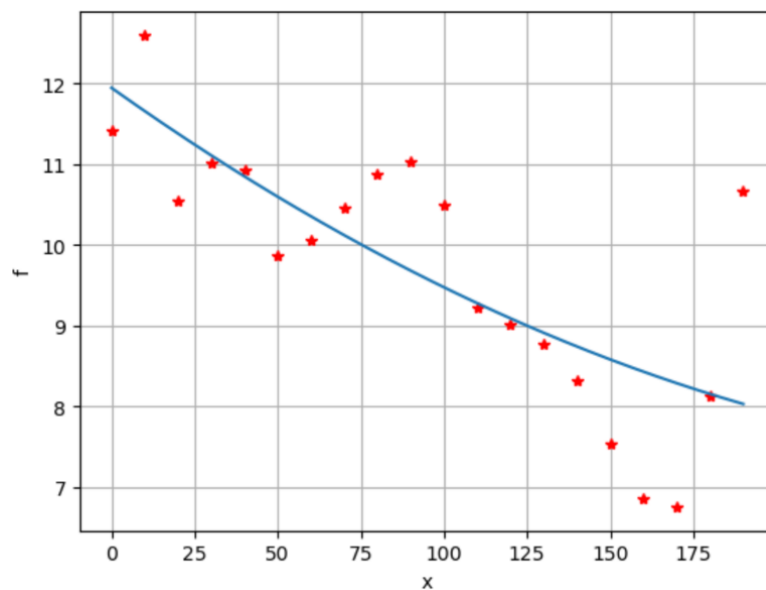


3) Створюємо наближений поліном II-го степеня і будуємо його графік:

```
APol_II = np.poly1d(np.polyfit(x, f, 2))
print('Наближений поліном II-го степеня:')
print(APol_II)
f1 = np.polyval(APol_II, x1) # Матриця зі значеннями наближеного полінома
# Побудова графіка наближеного полінома, позначення вузлових точок:
plt.plot(x, f, 'r*', x1, f1)
plt.xlabel('x')
plt.ylabel('f')
plt.grid(True)
```

Наближений поліном II-го степеня:

$$4.558e-05 x^2 - 0.02929 x + 11.95$$

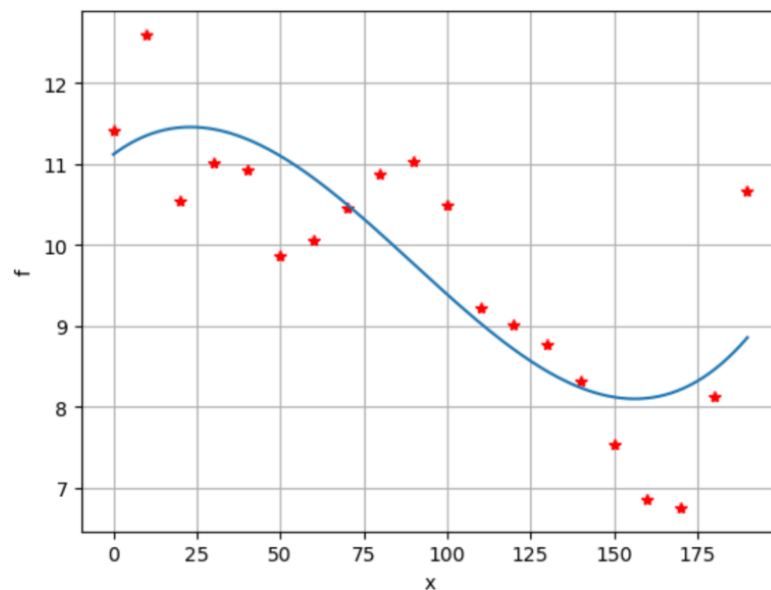


4) Створюємо наближений поліном III-го степеня і будуємо його графік:

```
APol_III = np.poly1d(np.polyfit(x, f, 3))
print('Наближений поліном III-го степеня:')
print(APol_III)
f1 = np.polyval(APol_III, x1) # Матриця зі значеннями наближеного полінома
# Побудова графіка наближеного полінома, позначення вузлових точок:
plt.plot(x, f, 'r*', x1, f1)
plt.xlabel('x')
plt.ylabel('f')
plt.grid(True)
```

Наближений поліном III-го степеня:

$$2.848e-06 x^3 - 0.0007662 x^2 + 0.03083 x + 11.12$$

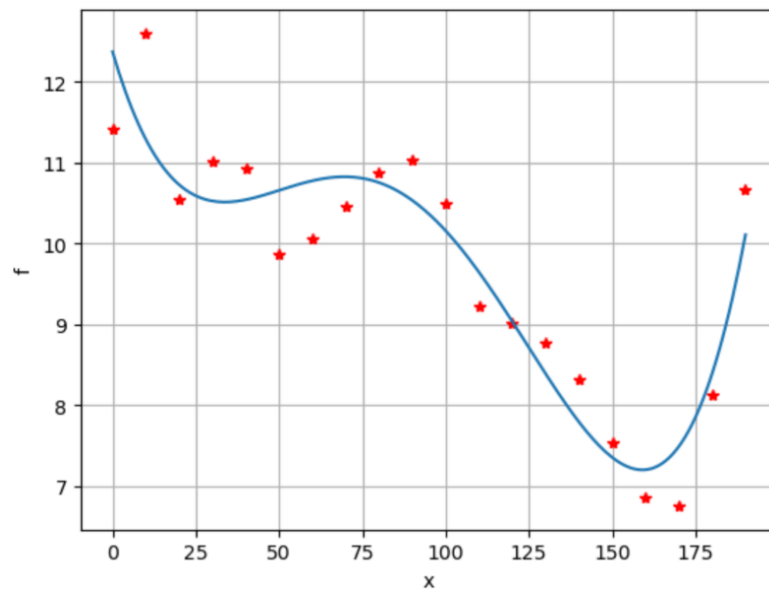


5) Створюємо наближений поліном IV -го степеня і будуємо його графік:

```
APol_IV = np.poly1d(np.polyfit(x, f, 4))
print('Наближений поліном IV-го степеня:')
print(APol_IV)
f1 = np.polyval(APol_IV, x1) # Матриця зі значеннями наближеного полінома
# Побудова графіка наближеного полінома, позначення вузлових точок:
plt.plot(x, f, 'r*', x1, f1)
plt.xlabel('x')
plt.ylabel('f')
plt.grid(True)
```

Наближений поліном IV-го степеня:

$$9.429e-08 x^4 - 3.298e-05 x^3 + 0.00354 x^2 - 0.1406 x + 12.37$$



Контрольні питання

1. Для чого використовується апроксимація функцій?
2. Як задати функцію табличним способом?
3. За якої умови система лінійних рівнянь для знаходження коефіцієнтів інтерполяційного полінома має єдиний розв'язок?
4. Які критерії точності наближення функцій використовуються при апроксимації?
5. За якої умови таблична функція, що задана в шести точках, має інтерполяційний поліном першого порядку?
6. Яка відмінність між багатоінтервальною інтерполяцією з використанням квадратичних поліномів та інтерполяцією з використанням сплайнів другого порядку?
7. В яких випадках надають перевагу наближеному поліному замість інтерполяційного?
8. Скільки інтерполяційних поліномів одержують в результаті розв'язання задачі глобальної інтерполяції табличної функції?
9. В чому полягає відмінність між інтерполяцією та екстраполяцією табличної функції?
10. Що є порядком сплайна?
11. Як оцінюється сумарне відхилення наближеного полінома від табличної функції за методом найменших квадратів?
12. Яке значення приймає параметр *deg* функції *polyfit()* модуля *NumPy* при розв'язанні задачі інтерполяції табличної функції?

Лабораторна робота № 6

ДИФЕРЕНЦІЮВАННЯ І ІНТЕГРУВАННЯ ФУНКЦІЙ

Мета роботи: одержання практичних навичок розрахунків похідних функцій і визначених інтегралів.

6.1 Диференціювання функцій

Диференціюванням функції є процес знаходження її похідних.

Перша похідна функції $f(x)$ характеризує миттєву швидкість зміни цієї функції відносно аргументу x і визначається як границя відношення приросту функції $\Delta f(x)$ до приросту аргументу Δx , коли приріст аргументу прямує до нуля:

$$\frac{df(x)}{dx} = f'(x) = \lim_{\Delta x \rightarrow 0} \frac{\Delta f(x)}{\Delta x} = \lim_{\Delta x \rightarrow 0} \frac{f(x + \Delta x) - f(x)}{\Delta x}. \quad (6.1)$$

Друга похідна функції $f(x)$:

$$\frac{d^2 f(x)}{dx^2} = f''(x) \quad (6.2)$$

характеризує миттєву швидкість зміни першої похідної, тобто прискорення зміни функції $f(x)$ тощо.

Функція є диференційованою на області визначення, якщо в кожній точці цієї області існує і має скінченне значення її перша похідна. Функція є n разів диференційованою, якщо в кожній точці області визначення існує та має скінченне значення її похідна n -го порядку [21].

Аналітичні вирази похідних отримують застосуванням відомих правил диференціювання (добутку, частки, композиції функцій тощо) та таблиць похідних елементарних функцій. За аналітичними виразами розраховують точні значення похідних для заданих значень аргументу.

В *Python* одержати аналітичний вираз похідної можливо з використанням функції `diff()` модуля *SymPy*:

`sympy.diff(expr, var, n = 1)`

де *expr* – символічний вираз функції, похідна якої знаходиться; *var* – символічна змінна, за якою виконується диференціювання; *n* – порядок похідної (за замовчуванням дорівнює 1).

Функція `diff()` повертає символьний вираз похідної n -го порядку функції `expr` за змінною `var`:

```
import sympy as smp
x = smp.Symbol('x')
f = smp.exp(-x) + smp.cos(x)
print('f(x) =')
f
```

$f(x) =$
 $\cos(x) + e^{-x}$

```
f3 = smp.diff(f, x, 3)
print("f'''(x) =")
f3
```

$f'''(x) =$
 $\sin(x) - e^{-x}$

Аналогічний результат можливо одержати за допомогою методу `.diff()`, який викликається безпосередньо для символьного виразу функції:

`expr.diff(var, n = 1)`

```
f.diff(x, 3)
```

$\sin(x) - e^{-x}$

Для розрахунку похідної в заданій точці в одержаний символьний вираз необхідно підставити відповідне числове значення аргументу функції. Для цього використовують метод `subs()` модуля `SymPy` [7]. Аргументом цього методу є словник, який задається у фігурних дужках. В словнику через кому вказуються пари «ключ: значення», де ключами є символьні змінні, а значеннями – числа або вирази, що підставляються замість відповідних змінних:

```
f3.subs({x: 2})
```

$-\frac{1}{e^2} + \sin(2)$

Для розрахунку числового значення виразу можливо використати метод `.n()`. В дужках вказується кількість значущих цифр, які необхідно залишити в результаті:

```
f3.subs({x: 2}).n(4)
```

0.774

Також для розрахунків значень символьного виразу можливо створити відповідну *Python*-функцію. Для цього використовують функцію *lambdify()* модуля *SymPy*:

```
sympy.lambdify(args, expr, modules = 'numpy')
```

де *args* – символьна змінна або список символьних змінних, які є аргументами *Python*-функції; *expr* – символьний вираз або колекція виразів; *modules* – модулі, які використовуються для розрахунків (за замовчуванням – *NumPy*).

Функція *lambdify()* створює *Python*-функцію, яка приймає числові значення для змінних *args* і повертає результат обчислення виразів *expr*:

```
func = smp.lambdify(x, f3)
func(2)
```

```
0.773962143589069
```

Якщо аналітичний вираз функції $f(x)$ є невідомим, занадто складним або функція задана лише таблично, то для розрахунків значень похідних в заданих точках застосовують числове диференціювання.

При цьому функцію $f(x)$ на відрізку $[a, b]$ наближують такою інтерполяційною функцією $\varphi(x)$, що похідні $f'(x)$ і $\varphi'(x)$ є рівними на даному відрізку. Тоді обчислення похідної в вузлі x_i зводиться до розрахунків значень функції $f(x)$ в цьому і в сусідніх вузлах.

В табл. 6.1 і 6.2 наведено формули числового диференціювання для випадку рівновіддалених вузлів, коли відстань між сусідніми вузлами дорівнює h :

$$\begin{aligned} & \dots \\ & x_{i-2} = x_i - 2h; \\ & x_{i-1} = x_i - h; \\ & x_{i+1} = x_i + h; \\ & x_{i+2} = x_i + 2h; \\ & \dots \end{aligned} \tag{6.3}$$

При одержанні формул в якості $\varphi(x)$ використовували лінійний (перші формули в табл. 6.1 і 6.2) та квадратичний (другі формули) інтерполяційні поліноми.

При прямому диференціюванні похідна в вузлі x_i розраховується за значенням функції $f(x_i)$ в цьому вузлі і за її значеннями в наступних (правосторонніх) вузлах $f(x_{i+1})$, $f(x_{i+2})$, ...

При зворотному диференціюванні похідна в вузлі x_i розраховується за значенням функції $f(x_i)$ в цьому вузлі і за її значеннями в попередніх (лівосторонніх) вузлах $f(x_{i-1})$, $f(x_{i-2})$, ...

Якщо ж для розрахунку похідної використовуються значення функції як в лівих, так і в правих вузлах відносно точки обчислення, то таке диференціювання є центральним або симетричним.

Табл. 6.1 – Формули числового диференціювання для розрахунку першої похідної функції

Номер формули	Формули	Порядок точності
Несиметричні прямі:		
1	$f'(x_i) = \frac{f(x_{i+1}) - f(x_i)}{h}$	h
2	$f'(x_i) = \frac{-f(x_{i+2}) + 4f(x_{i+1}) - 3f(x_i)}{2h}$	h^2
Несиметричні обернені:		
3	$f'(x_i) = \frac{f(x_i) - f(x_{i-1}))}{h}$	h
4	$f'(x_i) = \frac{3f(x_i) - 4f(x_{i-1}) + f(x_{i-2}))}{2h}$	h^2
Симетричні:		
5	$f'(x_i) = \frac{f(x_{i+1}) - f(x_{i-1}))}{2h}$	h^2
6	$f'(x_i) = \frac{-f(x_{i+2}) + 8f(x_{i+1}) - 8f(x_{i-1}) + f(x_{i-2}))}{12h}$	h^4

Табл. 6.2 – Формули числового диференціювання для розрахунку другої похідної функції

Номер формули	Формули	Порядок точності
Несиметричні прямі:		
1	$f''(x_i) = \frac{f(x_{i+2}) - 2f(x_{i+1}) + f(x_i)}{h^2}$	h
2	$f''(x_i) = \frac{-f(x_{i+3}) + 4f(x_{i+2}) - 5f(x_{i+1}) + 2f(x_i)}{h^2}$	h^2
Несиметричні обернені:		
3	$f''(x_i) = \frac{f(x_i) - 2f(x_{i-1}) + f(x_{i-2}))}{h^2}$	h
4	$f''(x_i) = \frac{2f(x_i) - 5f(x_{i-1}) + 4f(x_{i-2}) - f(x_{i-3}))}{h^2}$	h^2
Симетричні:		
5	$f''(x_i) = \frac{f(x_{i+1}) - 2f(x_i) + f(x_{i-1}))}{h^2}$	h^2
6	$f''(x_i) = \frac{-f(x_{i+2}) + 16f(x_{i+1}) - 30f(x_i) + 16f(x_{i-1}) - f(x_{i-2}))}{12h^2}$	h^4

Похибка числового диференціювання є пропорційною h^k , де k – порядок точності формули. Крок h зазвичай набагато менший за одиницю, тому похибка розрахунку похідної за наведеними формулами тим менша, чим більший порядок точності k .

Зменшення кроку h дозволяє знизити похибку диференціювання, але лише до певного граничного значення. Подальше підвищення точності можливе лише за рахунок збільшення степеня інтерполяційного поліному.

6.2 Інтегрування функцій

Інтегрування функції $f(x)$ – це обернена до диференціювання операція, яка полягає у визначенні функції $F(x)$ за її відомою похідною $f(x)$:

$$F'(x) = f(x). \quad (6.4)$$

Функція $F(x)$ є первісною функції $f(x)$.

Невизначеним інтегралом від функції $f(x)$ є множина всіх її первісних:

$$\int f(x)dx = F(x) + C, \quad (6.5)$$

де $f(x)$ – підінтегральна функція, x – змінна інтегрування, C – довільна стала інтегрування.

Аналітичні вирази первісних $F(x)$ знаходять із застосуванням методів інтегрування (підстановки, інтегрування частинами, інтегрування добутку, частки, складеної функції тощо) та таблиць інтегралів елементарних функцій [21].

Визначеним інтегралом є число, що дорівнює приросту первісної функції $F(x)$ на відрізку $[a, b]$ і розраховується за формулою Ньютона-Лейбніца:

$$\int_a^b f(x)dx = F(b) - F(a). \quad (6.6)$$

За даною формулою розраховується точне значення визначеного інтегралу.

Визначений інтеграл від функції $f(x)$ на відрізку $[a, b]$ дорівнює алгебраїчній сумі площ фігур, які обмежені графіком функції $f(x)$, віссю Ox та прямими $x = a$ і $x = b$ [15]. Площі тих фігур, які розташовані під віссю Ox (де $f(x) < 0$), мають знак «мінус».

Для одержання символьного виразу первісної використовується функція `integrate()` модуля `SymPy`:

`sympy.integrate(f, x)`

З використанням цієї ж функції можливо розрахувати точне значення визначеного інтегралу:

`sympy.integrate(f, (x, a, b))`

де f – символічний вираз підінтегральної функції, x – змінна інтегрування, a і b – межі інтегрування.

Створити в *Python* символічне представлення невизначеного або визначеного інтегралу можливо з використанням функції *Integral()* модуля *SymPy*. Параметри даної функції аналогічні параметрам функції *integrate()*:

`sympy.Integral(f, x)`
`sympy.Integral(f, (x, a, b))`

Проте не для всіх функцій можливо визначити первісну у вигляді скінченної комбінації елементарних функцій. В таких випадках, а також коли підінтегральна функція задана таблично, для розрахунку визначеного інтегралу використовуються числові методи [11].

Інтервал $[a, b]$ можливо поділити на n рівних підінтервалів довжиною Δx , і на кожному i -му підінтервалі обрати довільну точку ξ_i . Тоді площу під графіком функції $f(x)$, тобто точне значення визначеного інтегралу, можливо обчислити як суму добутків значень функції в точках ξ_i на довжину підінтервалу Δx за умови, що $\Delta x \rightarrow 0$, а кількість підінтервалів $n \rightarrow \infty$:

$$\int_a^b f(x)dx = \lim_{\Delta x \rightarrow 0} \left(\sum_{i=0}^{n-1} f(\xi_i) \Delta x \right). \quad (6.7)$$

При розрахунках визначених інтегралів числовими методами приймають, що довжина окремого підінтервалу Δx і їх кількість n є скінченними числами [16]. Тоді можливо виконати наближену заміну визначеного інтегралу скінченною сумою:

$$\int_a^b f(x)dx = \sum_{i=0}^{n-1} f(\xi_i) \Delta x. \quad (6.8)$$

Обчислення визначеного інтегралу числовими методами зводиться до багаторазових розрахунків значень підінтегральної функції на кожному підінтервалі відрізка $[a, b]$.

Для розрахунків визначених інтегралів числовими методами у *Python* використовують функцію *quad()* підмодуля *integrate* модуля *Scipy*:

`scipy.integrate.quad(func, a, b)`

де *func* – *Python*-функція, що розраховує значення підінтегральної функції. Її параметром є змінна інтегрування. a і b – межі інтегрування.

Функція *quad()* повертає числове значення інтегралу і похибку розрахунку.

6.3 Завдання до лабораторної роботи

Завдання 6.1

З використанням функції *diff()* модуля *SymPy* одержите аналітичні вирази першої і другої похідних заданої функції і розрахуйте точні значення похідних в точці x_0 .

Побудуйте графіки першої і другої похідних функції на заданому інтервалі.

Розрахуйте значення першої і другої похідних функції в точці x_0 за вказаними формулами числового диференціювання (табл. 6.1 і 6.2). Визначте похибки одержаних результатів.

№	Функція	Інтервал	x_0	Номер формули для розрахунку $f'(x)$	Номер формули для розрахунку $f''(x)$
1	$f(x) = \arccos\left(\frac{1}{x}\right) - x.$	[1,5; 2,5]	2	1	3
2	$f(x) = \frac{1}{x^2 + 1} - \cos\left(\frac{1}{x^3}\right).$	[-2; -1]	-2	2	4
3	$f(x) = \frac{1}{x} + \sin\left(\frac{1}{x^2}\right).$	[-2; -1]	-1	3	5
4	$f(x) = \operatorname{ctg}\left(\frac{1}{x^2 + 2}\right) + x^3.$	[0,5; 1,5]	0,5	4	6
5	$f(x) = \operatorname{ctg}(x^2) + x.$	[0,5; 1,5]	1	5	1
6	$f(x) = \cos\left(\frac{1}{x^2 - 1}\right) - \frac{x^3}{4}.$	[-3; -2]	-3	6	2
7	$f(x) = \cos\left(\frac{1}{x^2}\right) - \frac{1}{x}.$	[-2; -1]	-2	1	4
8	$f(x) = \operatorname{ctg}\left(\frac{4}{x^2 + 2}\right) - 2x.$	[1; 2]	1	2	5
9	$f(x) = \operatorname{tg}\left(\frac{1}{x^2}\right) + x.$	[1; 2]	1	3	6
10	$f(x) = \cos(x^3) - \frac{2}{x^2 - 1}.$	[3; 4]	3,5	4	1

11	$f(x) = \cos(x^2) + \frac{1}{x}.$	[1; 2]	2	5	2
12	$f(x) = \operatorname{arctg}\left(\frac{1}{2x^2 + 3}\right) + 4x.$	[-1,5; -0,5]	-1,5	6	3
13	$f(x) = \operatorname{arctg}\left(\frac{1}{x^2}\right) - x.$	[-1,5; -0,5]	-2,5	1	5
14	$f(x) = x + \arccos\left(\frac{1}{2x^2 - 1}\right).$	[-2; -1]	-2	2	6
15	$f(x) = x + \arcsin\left(\frac{1}{x}\right).$	[-3; -2]	-2	3	1
16	$f(x) = 2x + \arcsin\left(\frac{1}{x+4}\right).$	[-2; -1]	-1	4	2
17	$f(x) = x^2 \operatorname{ctg}(x).$	[0,5; 1,5]	1,5	5	3
18	$f(x) = 2x^3 \operatorname{tg}\left(\frac{1}{x+1}\right).$	[0,5; 1,5]	1	6	4
19	$f(x) = 2 \operatorname{ctg}(2x) - x.$	[-1,5; -0,5]	-1	1	6
20	$f(x) = \frac{1}{x^3} + e^x.$	[1; 2]	2	2	3
21	$f(x) = \sin(x^2) - \frac{1}{x}.$	[1; 2]	1	3	2
22	$f(x) = \frac{1}{x} - \sin\left(\frac{1}{x^3}\right).$	[-2; -1]	-1	4	5
23	$f(x) = e^{-x} \operatorname{ctg}(x).$	[2; 3]	3	5	4
24	$f(x) = \operatorname{tg}\left(\frac{2}{x^3}\right) - 6x.$	[1,5; 2,5]	2	6	1
25	$f(x) = (x^2 + 1) \operatorname{ctg}(x).$	[1,5; 2,5]	2,5	1	3
26	$f(x) = \arccos\left(\frac{1}{x^2}\right) + 3x^3.$	[1; 2]	2	2	4
27	$f(x) = \sin\left(\frac{1}{x^2}\right) - \frac{1}{x}.$	[1; 2]	-2	3	5
28	$f(x) = 3\cos(x^2) - \frac{x}{x^2 + 2}.$	[1; 2]	2	4	6
29	$f(x) = \arcsin\left(\frac{1}{x^2 - 1}\right) - 3x.$	[2; 3]	3	5	1
30	$f(x) = e^x \cos(e^x).$	[0; 1]	0,5	6	2

Завдання 6.2

Побудуйте фігуру, площа якої обчислюється за заданим визначеним інтегралом.

З використанням функції *integrate()* модуля *SymPy* отримайте аналітичний вираз первісної підінтегральної функції і розрахуйте точне значення визначеного інтегралу.

Розрахуйте визначений інтеграл з використанням функції *quad()* модуля *SciPy*.

Номер варіанта	Інтеграл	Номер варіанта	Інтеграл
1	$\int_0^{\sqrt{3}} x \cdot \sqrt[3]{1+x^2} dx$	2	$\int_0^{2\sqrt{3}} \frac{12x^5 dx}{\sqrt{x^6+1}}$
3	$\int_0^1 \frac{x^2 dx}{x^2+1}$	4	$\int_0^{\pi/2} \sin(x) \cos^2(x) dx$
5	$\int_0^{\pi/2} \frac{\cos(x)}{1+\cos(x)} dx$	6	$\int_{3/4}^{4/3} \frac{dx}{x^2+1}$
7	$\int_{-3}^0 \frac{dx}{\sqrt{25+3x}}$	8	$\int_0^2 \frac{x^3 dx}{\sqrt{x^4+4}}$
9	$\int_1^e \frac{1+\ln(x)}{x} dx$	10	$\int_0^1 \frac{z^3}{z^8+1} dz$
11	$\int_{\pi/4}^{\pi/2} \frac{dx}{1-\cos^2(x)}$	12	$\int_{-1}^1 \frac{dx}{\sqrt{5-3x^2}}$
13	$\int_0^1 x^3 \cdot \sqrt{4+5x^4} dx$	14	$\int_{-\pi}^{\pi} \sin^2\left(\frac{x}{2}\right) dx$
15	$\int_1^2 \frac{e^{1/x}}{x^2} dx$	16	$\int_0^{1/2} \frac{xdx}{\sqrt{1-x^2}}$
17	$\int_0^1 3(x^2 + x^2 e^x) dx$	18	$\int_{\pi^2/9}^{\pi^2} \frac{\cos(\sqrt{x})}{\sqrt{x}} dx$
19	$\int_1^{\sqrt{3}} \frac{x^2 dx}{1+x^6}$	20	$\int_1^e \frac{\sin(\ln(x))}{x} dx$

21	$\int_1^{\sqrt{e}} \frac{dx}{x\sqrt{1-\ln(x)}}$	22	$\int_3^8 \sqrt{x^2+3} dx$
23	$\int_{\pi/6}^{\pi/2} \sin(\alpha)\cos^3(\alpha)d\alpha$	24	$\int_{\pi/18}^{\pi/6} 12\operatorname{ctg}(3x)dx$
25	$\int_0^1 \frac{dx}{\sqrt{4-3x}}$	26	$\int_1^{\sqrt{2}} \frac{xdx}{\sqrt{4-x^2}}$
27	$\int_1^e \frac{\ln^2(x)}{x} dx$	28	$\int_{-1}^0 \frac{dx}{4x^2-9}$
29	$\int_{\pi/6}^{\pi} \cos(\alpha)\sin^2(\alpha)d\alpha$	30	$\int_0^{\sqrt{\pi}/4} \frac{xdx}{\cos^2(x^2)}$

Завдання 6.3

Задано підінтегральну функцію і межі інтегрування.

Побудуйте фігуру, площа якої обчислюється за заданим визначеним інтегралом. Розрахуйте визначений інтеграл з використанням функції *quad()* модуля *SciPy*.

№	Підінтегральна функція	Межі інтегрування	№	Підінтегральна функція	Межі інтегрування
1	$0,37e^{\sin(x)}$	[0; 1]	2	$0,5x + x \lg(x)$	[1; 2]
3	$(x + 1,9)\sin\left(\frac{x}{3}\right)$	[1; 2]	4	$\frac{\ln(x+2)}{x}$	[2; 3]
5	$\frac{3\cos(x)}{2x+1,7}$	[0; 1]	6	$(2x+0,6)\cos\left(\frac{x}{2}\right)$	[1; 2]
7	$2,6x^2 \ln(x)$	[1,2; 2,2]	8	$(x^2+1)\sin(x-0,5)$	[0,5; 1,5]
9	$x^2 \cos\left(\frac{x}{4}\right)$	[2; 3]	10	$\sin(0,2x+3)(x^2+1)$	[3; 4]
11	$3x + \ln(x)$	[1; 2]	12	$4xe^x$	[-1; 0]
13	$3x^2 + \operatorname{tg}(x)$	[-0,5; 0,5]	14	$3x^2 + \sin(x)$	[0; 1]
15	$3xe^{\cos(x)}$	[0,2; 1,2]	16	$x^2 \operatorname{tg}\left(\frac{x}{2}\right)$	[0,5; 1,5]
17	$\frac{x^2}{1+0,25x}$	[1,1; 2,1]	18	$\frac{x^3 - 0,3x}{\sqrt{1+2x}}$	[2; 3]
19	$2e^{-x}(2+x^3)$	[1; 2]	20	$\cos(x^2)x$	[0; 1]
21	$\sqrt{1+x}\sin(x)$	[2; 3]	22	$e^x + x^2 - 1$	[0; 1]
23	$(e^x + x)\sin(x)$	[0; 1]	24	$\sqrt{3+x}\lg(x)$	[1; 2]
25	$(4+x)\sin(x^2)$	[1; 2]	26	$xe^{\sin(x)}$	[2; 3]
27	$\sin(x)\cos(x)x$	[2; 3]	28	$\sin(x)\ln(x)$	[1; 2]
29	$\cos(x)\ln(x)$	[3; 4]	30	$x^2 \lg(x)$	[2; 3]

6.4 Приклади розв'язання завдань

Завдання 6.1

З використанням функції *diff()* модуля *SymPy* одержіть аналітичні вирази першої і другої похідних заданої функції і розрахуйте точні значення похідних в точці $x_0 = 1,5$.

Побудуйте графіки першої і другої похідних функції на інтервалі $[0; 2]$.

Розрахуйте значення першої і другої похідних функції в точці x_0 за формулами прямого диференціювання (табл. 6.1 і 6.2, формули 1). Визначте похибки одержаних результатів.

$$f(x) = \frac{(x+2)^4}{\sqrt{e^{3x}}} + \operatorname{ctg}\left(\frac{1}{x-5}\right).$$

Розв'язання

Функція для розрахунку котангенса відсутня в бібліотеках *math* та *NumPy* (табл. 1.2). Тому скористаємось співвідношенням

$$\operatorname{tg}(\alpha) \cdot \operatorname{ctg}(\alpha) = 1$$

і перепишемо задану функцію у вигляді:

$$f(x) = \frac{(x+2)^4}{\sqrt{e^{3x}}} + \frac{1}{\operatorname{tg}\left(\frac{1}{x-5}\right)}.$$

1) Задаємо символну функцію, похідні якої знаходяться:

```
import sympy as smp
x = smp.Symbol('x')
f = (x + 2) ** 4 / smp.sqrt(smp.exp(3 * x)) + 1 / smp.tan(1 / (x - 5))
print('f( x ) =')
f
```

$$f(x) = \frac{(x+2)^4}{\sqrt{e^{3x}}} + \frac{1}{\tan\left(\frac{1}{x-5}\right)}$$

Одержуємо аналітичний вираз першої похідної заданої функції:

```
f1 = symp.diff(f, x, 1)
print("f'( x ) =")
f1
```

$$f'(x) = -\frac{3(x+2)^4}{2\sqrt{e^{3x}}} + \frac{4(x+2)^3}{\sqrt{e^{3x}}} + \frac{\tan^2\left(\frac{1}{x-5}\right) + 1}{(x-5)^2 \tan^2\left(\frac{1}{x-5}\right)}$$

Розраховуємо точне значення першої похідної в точці $x_0 = 1,5$:

```
x0 = 1.5
f10 = f1.subs({x : x0}).n(5) # Значення першої похідної в точці x0
print("f'(", x0, ') =', f10)
```

$$f'(1.5) = -4.6211$$

Одержуємо аналітичний вираз другої похідної заданої функції:

```
f2 = symp.diff(f, x, 2)
print('f''( x ) =')
f2
```

$$f''(x) = \frac{9(x+2)^4}{4\sqrt{e^{3x}}} - \frac{12(x+2)^3}{\sqrt{e^{3x}}} + \frac{12(x+2)^2}{\sqrt{e^{3x}}} - \frac{2\left(\tan^2\left(\frac{1}{x-5}\right) + 1\right)}{(x-5)^3 \tan^2\left(\frac{1}{x-5}\right)} + \frac{2\left(\tan^2\left(\frac{1}{x-5}\right) + 1\right)^2}{(x-5)^4 \tan^3\left(\frac{1}{x-5}\right)} - \frac{2\left(\tan^2\left(\frac{1}{x-5}\right) + 1\right)}{(x-5)^4 \tan\left(\frac{1}{x-5}\right)}$$

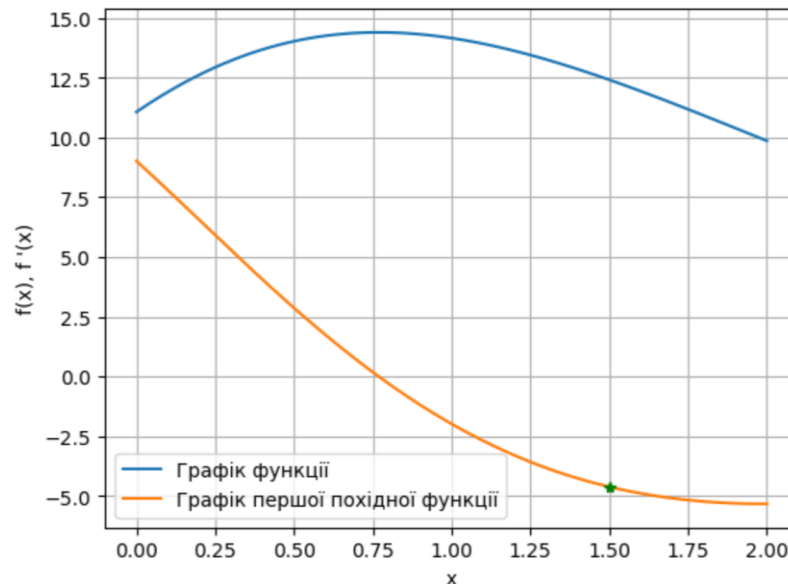
Розраховуємо точне значення другої похідної в точці $x_0 = 1,5$:

```
f20 = f2.subs({x : x0}).n(5) # Значення другої похідної в точці x0
print('f''(', x0, ') =', f20)
```

$$f''(1.5) = -3.1311$$

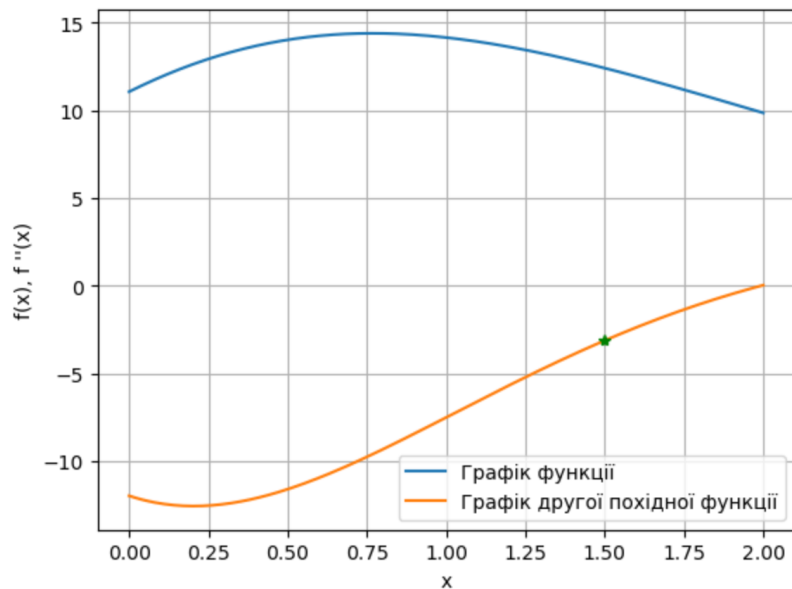
Будуємо графік першої похідної заданої функції:

```
import numpy as np
import matplotlib.pyplot as plt
a = 0 # Інтервал побудови графіка
b = 2
m = 200 # Кількість точок, за якими будується графік
X = np.linspace(a, b, m) # Матриця зі значеннями аргументу
# Python-функції для розрахунків значень функції  $f(x)$  і її першої похідної:
func_f = smp.lambdify(x, f)
func_f1 = smp.lambdify(x, f1)
plt.plot(X, func_f(X), X, func_f1(X), x0, f10, 'g*') # Побудова графіка
plt.legend(['Графік функції', 'Графік першої похідної функції']) # Додавання легенди
plt.xlabel('x') # Підпис координатних осей
plt.ylabel("f(x), f'(x)")
plt.grid(True) # Сітка
```



Будуємо графік другої похідної заданої функції:

```
# Python-функція для розрахунків значень другої похідної:
func_f2 = smp.lambdify(x, f2)
plt.plot(X, func_f(X), X, func_f2(X), x0, f20, 'g*') # Побудова графіка
plt.legend(['Графік функції', 'Графік другої похідної функції']) # Додавання легенди
plt.xlabel('x') # Підпис координатних осей
plt.ylabel("f(x), f''(x)")
plt.grid(True) # Сітка
```



2) Розраховуємо значення першої похідної функції в заданій точці за формулою прямого диференціювання і визначаємо похибку результату:

```
import math
def f(x): # Python-функція для розрахунку значень функції f(x)
    return (x + 2) ** 4 / math.sqrt(math.exp(3 * x)) + 1 / math.tan(1 / (x - 5))
x0 = 1.5
h = 0.001 # Крок
f_ch_10 = (f(x0 + h) - f(x0)) / h # Значення першої похідної
print("f'", x0, ") =", round(f_ch_10, 4))
# Відносна похибка результату розрахунку першої похідної за формулою прямого диференціювання:
print('Відносна похибка результату:', round(abs(f_ch_10 - f10) / abs(f10) * 100, 4), '%')

f'( 1.5 ) = -4.6226
Відносна похибка результату: 0.0339 %
```

Розраховуємо значення другої похідної функції в заданій точці за формулою прямого диференціювання і визначаємо похибку результату:

```
f_ch_20 = (f(x0 + 2 * h) - 2 * f(x0 + h) + f(x0)) / h ** 2 # Значення другої похідної
print('f"', x0, ') =', round(f_ch_20, 4))
# Відносна похибка результату розрахунку другої похідної за формулою прямого диференціювання:
print('Відносна похибка результату:', round(abs(f_ch_20 - f20) / abs(f20) * 100, 3), '%')

f''( 1.5 ) = -3.1233
Відносна похибка результату: 0.248 %
```

Відповідь: $f'(1,5) = -4,6211$; $f''(1,5) = -3,1311$.

Завдання 6.2

Побудуйте фігуру, площа якої обчислюється за заданим визначеним інтегралом.

З використанням функції *integrate()* модуля *SymPy* отримайте аналітичний вираз первісної підінтегральної функції і розрахуйте точне значення визначеного інтегралу.

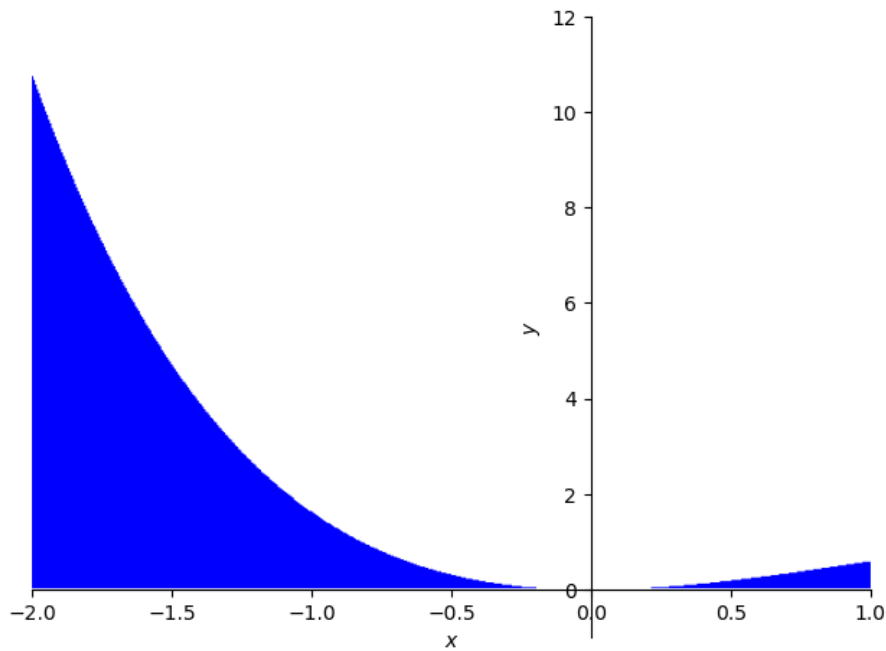
Розрахуйте визначений інтеграл з використанням функції *quad()* модуля *SciPy*.

$$\int_{-2}^1 x^2 e^{-x/2} dx.$$

Розв'язання

1) Будуємо фігуру, площа якої обчислюється за заданим визначеним інтегралом:

```
import sympy as smp
import matplotlib.pyplot as plt
# Символьні змінні: x – змінна інтегрування,
# y – змінна, що позначає ординату точки на графіку
x, y = smp.symbols('x y')
f = x ** 2 * smp.exp(-x / 2) # Підінтегральна функція
a_x = -2 # Межі інтегрування
b_x = 1
a_y = -1 # Межі інтервалу побудови графіка за віссю ординат
b_y = 12
# Додатні площі:
plot1 = smp.plot_implicit(smp.And(y < f, y > 0), (x, a_x, b_x),
                          (y, a_y, b_y), show = False)
# Від'ємні площі:
plot2 = smp.plot_implicit(smp.And(y > f, y < 0), (x, a_x, b_x),
                          (y, a_y, b_y), line_color = 'r', show = False)
plot1.extend(plot2) # Об'єднання графіків
plot1.show()
```



Створюємо символічне представлення визначеного інтегралу, який розраховується:

```
smp.Integral(f, (x, a_x, b_x))
```

$$\int_{-2}^1 x^2 e^{-\frac{x}{2}} dx$$

Визначаємо первісну підінтегральної функції:

```
F = smp.integrate(f, x)
print('F(x) =')
F
```

$$F(x) = (-2x^2 - 8x - 16)e^{-\frac{x}{2}}$$

Розраховуємо визначений інтеграл:

```
J = smp.integrate(f, (x, a_x, b_x)).n(5)
J
```

5.9765

2) Розраховуємо визначений інтеграл з використанням функції `quad()` модуля `SciPy`:

```
import math
from scipy import integrate
# Python-функція для розрахунку значень підінтегральної функції:
def f(x):
    return x ** 2 * math.exp(-x / 2)
a = -2 # Межі інтегрування
b = 1
J, err = integrate.quad(f, a, b)
print('Результат розрахунку визначеного інтегралу:', round(J, 4))
print('Похибка результату:', err)
```

Результат розрахунку визначеного інтегралу: 5.9765

Похибка результату: 6.635200694597879e-14

Відповідь: $\int_{-2}^1 x^2 e^{-x/2} dx = 5,9765.$

Примітка:

Фігуру, площа якої обчислюється за заданим визначеним інтегралом, необхідно зобразити повністю і так, щоб вона займала більшу частину графіка.

Завдання 6.3

Задано підінтегральну функцію і межі інтегрування.

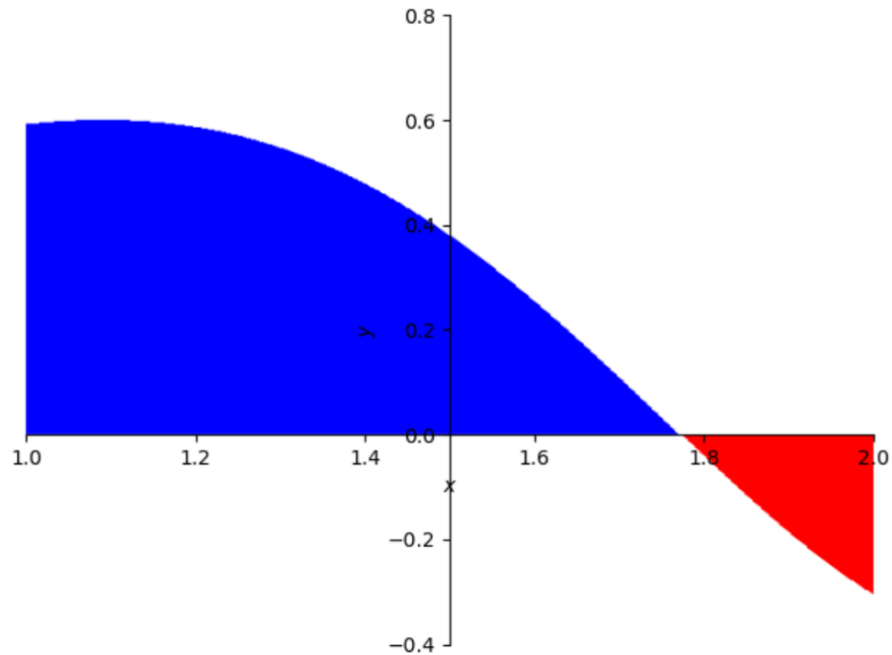
Побудуйте фігуру, площа якої обчислюється за заданим визначеним інтегралом. Розрахуйте визначений інтеграл з використанням функції *quad()* модуля *SciPy*.

$$\frac{\sin(x^2)}{x\sqrt{2 - \ln^2(x)}}; \quad x \in [1; 2].$$

Розв'язання

1) Будуємо фігуру, площа якої обчислюється за заданим визначеним інтегралом:

```
import sympy as smp
import matplotlib.pyplot as plt
# Символьні змінні: x - змінна інтегрування,
# y - змінна, що позначає ординату точки на графіку
x, y = smp.symbols('x y')
f = smp.sin(x ** 2) / (x * smp.sqrt(2 - smp.log(x) ** 2)) # Підінтегральна функція
a_x = 1 # Межі інтегрування
b_x = 2
a_y = -0.4 # Межі інтервалу побудови графіку за віссю ординат
b_y = 0.8
# Додатні площі:
plot1 = smp.plot_implicit(smp.And(y < f, y > 0), (x, a_x, b_x),
                          (y, a_y, b_y), show = False)
# Від'ємні площі:
plot2 = smp.plot_implicit(smp.And(y > f, y < 0), (x, a_x, b_x),
                          (y, a_y, b_y), line_color = 'r', show = False)
plot1.extend(plot2) # Об'єднання графіків
plot1.show()
```



2) Розраховуємо визначений інтеграл з використанням функції *quad()* модуля *SciPy*:

```
import math
from scipy import integrate
# Python-функція для розрахунку значень підінтегральної функції:
def f(x):
    return smp.sin(x ** 2) / (x * smp.sqrt(2 - smp.log(x) ** 2))
a = 1 # Межі інтегрування
b = 2
J, err = integrate.quad(f, a, b)
print('Результат розрахунку визначеного інтегралу:', round(J, 4))
print('Похибка результату:', err)
```

Результат розрахунку визначеного інтегралу: 0.2897

Похибка результату: 4.043825450851084e-15

Відповідь: $\int_1^2 \frac{\sin(x^2)dx}{x\sqrt{2-\ln^2(x)}} = 0,2897.$

Контрольні питання

1. Який фізичний і геометричний зміст має перша похідна функції?
2. Як у *Python* отримати аналітичні вирази похідних функцій?
3. Що є результатом числового диференціювання функції?
4. Які відмінності мають розрахунки похідних за формулами несиметричного прямого, несиметричного оберненого та симетричного диференціювання?
5. Що є кроком в формулах числового диференціювання?
6. Як зміна кроку впливає на точність числового диференціювання?
7. Що є порядком точності формул числового диференціювання?
8. Який зв'язок між первісною та підінтегральною функцією?
9. Як розраховується визначений інтеграл за формулою Ньютона-Лейбніца?
10. Який геометричний зміст має визначений інтеграл?
11. Що є результатом числового інтегрування функції?
12. Чи потрібно знаходити первісну при числовому інтегруванні функції?

Лабораторна робота № 7

РОЗВ'ЯЗАННЯ ЗВИЧАЙНИХ ДИФЕРЕНЦІАЛЬНИХ РІВНЯНЬ ТА ЇХ СИСТЕМ

Мета роботи: одержання практичних навичок розв'язання задачі Коші для звичайних диференціальних рівнянь та їх систем.

7.1 Загальні відомості про звичайні диференціальні рівняння

Диференціальне рівняння описує взаємозв'язок між незалежними змінними, функціями цих змінних і похідними цих функцій.

Звичайне диференціальне рівняння (ЗДР, *ODE*) містить одну невідому функцію однієї незалежної змінної та похідні цієї функції. Диференціальне рівняння з частинними похідними містить одну невідому функцію кількох незалежних змінних та її частинні похідні [11].

Порядком диференціального рівняння є найвищий порядок похідної, яка входить до цього рівняння.

Загальний вигляд ЗДР n -го порядку:

$$F(x, y, y', y'', \dots, y^{(n)}) = 0, \quad (7.1)$$

де x – незалежна змінна, $y(x)$ – невідома функція, $y^{(k)} = \frac{d^k y(x)}{dx^k}$ – k -та похідна функції $y(x)$, $k = 1, 2, \dots, n$.

ЗДР є явним, якщо похідну найвищого порядку виражено через незалежну змінну, невідому функцію та похідні нижчих порядків:

$$y^{(n)} = F(x, y, y', y'', \dots, y^{(n-1)}). \quad (7.2)$$

Інтегруванням диференціального рівняння є процес знаходження його розв'язків.

Загальним розв'язком ЗДР n -го порядку є сімейство n разів диференційованих функцій, кожна з яких при підстановці в рівняння перетворює його на тотожність:

$$y_3 = f(x, C_1, C_2, \dots, C_n), \quad (7.3)$$

де $C_1, C_2, \dots, C_n$ – сталі інтегрування, кількість яких відповідає порядку ЗДР [15].

Графік розв'язку ЗДР має назву інтегральної кривої. Геометрично загальний розв'язок ЗДР є сімейством інтегральних кривих.

Для визначення n сталих інтегрування необхідно задати n додаткових умов, тобто n рівнянь, які встановлюють значення функції $y(x)$ та її похідних при певних значеннях незалежної змінної x .

Загальний розв'язок ЗДР з певними значеннями сталих інтегрування, які задовольняють ці додаткові умови, є частинним розв'язком ЗДР:

$$y_c = f(x). \quad (7.4)$$

Якщо всі додаткові умови задано в одній початковій точці x_0 , то вони мають назву початкових умов. Задача знаходження частинного розв'язку ЗДР, що задовольняє початкові умови, має назву задачі Коші.

Якщо додаткові умови задано більш ніж в одній точці, то вони мають назву граничних або крайових умов. Задача знаходження частинного розв'язку ЗДР, що задовольняє ці умови, має назву крайової задачі.

Задача Коші для ЗДР n -ого порядку:

$$\begin{cases} y^{(n)} = F(x, y, y', y'', \dots, y^{(n-1)}), \\ y(x_0) = y_0, \\ y'(x_0) = y_1, \\ \dots\dots\dots \\ y^{(n-1)}(x_0) = y_{n-1}. \end{cases} \quad (7.5)$$

де $x_0, y_0, y_1, \dots, y_{n-1}$ – певні числа. Задача містить n початкових умов: значення невідомої функції та її похідних до порядку $n - 1$ включно в початковій точці x_0 [21].

Розв'язком задачі Коші для ЗДР n -го порядку є функція $y(x)$, яка при підстановці в перше рівняння (7.5) перетворює його на тотожність і разом зі своїми похідними до порядку $n - 1$ включно набуває в точці x_0 значень, що задані початковими умовами.

Задача Коші для ЗДР першого порядку:

$$\begin{cases} y' = f(x, y), \\ y(x_0) = y_0. \end{cases} \quad (7.6)$$

Геометрично задача Коші для ЗДР першого порядку полягає у знаходженні на площині такої інтегральної кривої, яка проходить через точку з координатами (x_0, y_0) .

Задача Коші для системи ЗДР першого порядку:

[illegible]

де m – кількість невідомих функцій в системі, що дорівнює кількості рівнянь. Задача містить m початкових умов: значення кожної невідомої функції в початковій точці x_0 [17].

Розв'язком задачі Коші для системи ЗДР першого порядку є набір функцій $y_j(x)$, $j = 1, \dots, m$, які при підстановці в систему перетворюють кожне її рівняння на тотожність і в точці x_0 набувають значень $y_{j,0}$, що задані початковими умовами.

Задачу Коші для ЗДР n -го порядку (7.5) можливо звести до задачі Коші для системи ЗДР першого порядку (7.7), що містить n рівнянь [22]. Для цього виконують наступні заміни:

$$\begin{aligned} y(x) &= u_1(x), \\ y'(x) &= u_1'(x) = u_2(x), \\ &\dots \\ y^{(n-1)}(x) &= u_{n-1}'(x) = u_n(x). \end{aligned} \tag{7.8}$$

Найстарша похідна $y^{(n)}(x) = u'_n(x)$.

Тоді система, що розв'язується, має вигляд:

$$\left\{ \begin{array}{l} u_1' = u_2, \quad u_1(x_0) = y_0; \\ u_2' = u_3, \quad u_2(x_0) = y_1; \\ \\ u_n' = F(x, u_1, u_2, ..., u_n), \quad u_n(x_0) = y_{n-1}. \end{array} \right. \quad (7.9)$$

Розв'язком рівняння є функція $u_1(x)$, яка відповідає $y(x)$, тоді як решта функцій $u_2(x), \dots, u_n(x)$ є допоміжними і відповідають похідним $y'(x), \dots, y^{(n-1)}(x)$.

7.2 Аналітичне розв'язання звичайних диференціальних рівнянь та їх систем у *Python*

Аналітичні загальні розв'язки ЗДР знаходять за допомогою методів розділення змінних, заміни змінних, невизначених коефіцієнтів, варіації сталої, інтегрувального множника тощо. У *Python* аналітичні розв'язки можливо одержати з використанням функцій бібліотеки *SymPy*.

Для створення символного об'єкта, що представляє невідому функцію, в *SymPy* призначена функція *Function()*:

```
import sympy as smp
x = smp.symbols('x')
f = smp.Function('y')
f(x)
```

$y(x)$

де f – ім'я змінної, що містить об'єкт функції, y – ім'я функції в символних виразах.

Диференціальне рівняння в символному вигляді створюється з використанням функції *Eq()* модуля *SymPy*, яку розглянуто в розділі 3. Похідні можливо задавати за допомогою методу *.diff()*, що викликається для відповідної функції [16]:

```
D = smp.Eq(f(x).diff(x, 3), x ** 2 - f(x) - f(x).diff(x) - f(x).diff(x, 2))
D
```

$$\frac{d^3}{dx^3}y(x) = x^2 - y(x) - \frac{d}{dx}y(x) - \frac{d^2}{dx^2}y(x)$$

Методи *.rhs* та *.lhs* повертають відповідно праву та ліву частини символного рівняння:

```
D.rhs
```

$$x^2 - y(x) - \frac{d}{dx}y(x) - \frac{d^2}{dx^2}y(x)$$

```
D.lhs
```

$$\frac{d^3}{dx^3}y(x)$$

Для отримання аналітичних загального та частинного розв'язків ЗДР і їх систем використовують функцію *dsolve()* модуля *SymPy*:

```
sympy.dsolve(Eq, ics = None)
```

де *Eq* – ЗДР або список ЗДР системи в символному вигляді;

ics – словник початкових умов. Задається для отримання частинного розв’язку. Має наступний загальний вигляд:

$$\{f(x_0): y_0, f(x).diff(x).subs(\{x: x_1\}): y_1, \dots\},$$

де y_0 – значення функції $y(x)$ в точці x_0 , y_1 – значення першої похідної функції $y(x)$ в точці x_1 тощо.

Функція *dsolve()* повертає рівняння функції $y(x)$, яка є розв’язком ЗДР, або список рівнянь функцій $y_j(x)$, які є розв’язками системи ЗДР.

Розглянемо приклад аналітичного розв’язання ЗДР третього порядку

$$y''' = x^2 - y - y' - y''. \quad (7.10)$$

з початковими умовами

$$y(0) = 1, \quad y'(0) = 2, \quad y''(0) = 3. \quad (7.11)$$

```
# ЗДР, що розв'язується:
```

```
D
```

$$\frac{d^3}{dx^3}y(x) = x^2 - y(x) - \frac{d}{dx}y(x) - \frac{d^2}{dx^2}y(x)$$

```
# Загальний розв'язок:
```

```
smp.dsolve(D)
```

$$y(x) = C_1 e^{-x} + C_2 \sin(x) + C_3 \cos(x) + x^2 - 2x$$

```
# Початкові умови:
```

```
x0 = 0
```

```
y0 = 1
```

```
y1 = 2
```

```
y2 = 3
```

```
# Частинний розв'язок:
```

```
A_Sol = smp.dsolve(D, ics = {f(x0): y0,
                             f(x).diff(x).subs({x: x0}): y1,
                             f(x).diff(x, 2).subs({x: x0}): y2})
```

```
A_Sol
```

$$y(x) = x^2 - 2x + 5 \sin(x) + e^{-x}$$

```
# Значення частинного розв'язку в точці x = 4:
```

```
A_Sol_4 = A_Sol.rhs.subs({x: 4})
```

```
print('y(4) =')
```

```
round(A_Sol_4, 5)
```

```
y(4) =
```

```
4.2343
```

7.3 Числове розв'язання звичайних диференціальних рівнянь та їх систем у *Python*

Не всі диференціальні рівняння мають розв'язки, які можливо представити у вигляді скінченної комбінації елементарних функцій. В багатьох випадках розв'язок можливо знайти лише числовими методами.

При використанні числових методів знаходять наближені частинні розв'язки ЗДР у вигляді набору значень невідомої функції в дискретній множині точок, що належать певному інтервалу. Числовий розв'язок завжди містить похибку, яка залежить від кроку інтегрування та точності обраного числового методу [18].

У *Python* числовий розв'язок задачі Коші для звичайних диференціальних рівнянь першого порядку або їх систем можливо знайти за допомогою функції *solve_ivp()* підмодуля *integrate* модуля *SciPy*:

```
scipy.integrate.solve_ivp(fun, t_span, y0, method = 'RK45',  
                           t_eval = None, dense_output = False)
```

де *fun* – *Python*-функція для розрахунку виразу в правій частині явного ЗДР (7.6) або виразів в правих частинах явних рівнянь системи (7.7);

t_span – колекція двох значень незалежної змінної x_0 і x_n , які визначають межі інтервалу інтегрування;

y0 – колекція, що містить значення y_0 (або значення $y_{j,0}$ при розв'язанні системи ЗДР) в початковій точці x_0 ;

method – текстовий рядок з назвою числового методу розв'язання задачі Коші. За замовчуванням використовується явний метод Рунге-Кутти п'ятого порядку;

t_eval – впорядкована колекція значень незалежної змінної від x_0 до x_n , для яких обчислюється розв'язок. Якщо параметр не задано, то точки обираються розв'язувачем;

dense_output – логічна змінна; при значенні *True* виконується наближення розв'язку неперервною функцією.

Функція *solve_ivp()* повертає об'єкт, що містить наступні атрибути:

t – матриця значень незалежної змінної, для яких обчислено розв'язок. Якщо параметр *t_eval* задано, то саме його значення використовуються для формування *t*.

y – двовимірна матриця розмірності m на r , де m – кількість рівнянь в системі (при розв'язанні одного ЗДР $m = 1$), а r – кількість значень в матриці *t*. Кожен її рядок містить значення окремої функції $y_j(x)$ розв'язку для відповідних значень незалежної змінної з матриці *t*.

`sol` – об’єкт, що приймає значення незалежної змінної і повертає матрицю з наближеним значенням функції $y(x)$ або наближеними значеннями функцій $y_i(x)$ в цій точці, якщо `dense_output = True`.

Для доступу до конкретного атрибуту його ім’я необхідно вказати через крапку після імені об’єкта, що повертається функцією `solve_ivp()` [6].

Розглянемо приклад числового розв’язання задачі Коші для ЗДР третього порядку (7.10) з початковими умовами (7.11). Сформуємо еквівалентну систему трьох ЗДР першого порядку з врахуванням того, що індексація елементів у *Python* починається з 0:

$$\begin{cases} u_0' = u_1, & u_0(0) = 1; \\ u_1' = u_2, & u_1(0) = 2; \\ u_2' = x^2 - u_0 - u_1 - u_2, & u_2(0) = 3. \end{cases} \quad (7.12)$$

де $y(x) = u_0(x)$, $y'(x) = u_1(x)$, $y''(x) = u_2(x)$.

```
from scipy.integrate import solve_ivp
# Python-функція для розрахунку виразів
# в правих частинах явних диференціальних рівнянь системи:
def f(x, u):
    return [u[1], u[2], x ** 2 - u[0] - u[1] - u[2]]
a = 0 # Інтервал інтегрування
b = 4
u0_a = 1 # Значення функцій в початковій точці
u1_a = 2
u2_a = 3
# Об'єкт, що містить числовий розв'язок задачі Коші для заданого рівняння:
N_Sol = solve_ivp(f, [a, b], [u0_a, u1_a, u2_a], dense_output = True)
# Значення функції, що наближає числовий розв'язок, в точці x = 4:
N_Sol_4 = N_Sol.sol(4)[0]
print('y(4) =') # Виведення на екран округленого значення
round(N_Sol_4, 5)
```

```
y(4) =
4.23346
```

Відносна похибка числового розв’язку в точці $x = 4$:

```
delta = abs((N_Sol_4 - A_Sol_4) / A_Sol_4) * 100
print(round(delta, 4), '%')
```

```
0.0198 %
```

7.4 Завдання до лабораторної роботи

Завдання 7.1

1) З використанням функції *dsolve()* модуля *SymPy* знайдіть загальний і частинний аналітичні розв'язки заданого звичайного диференціального рівняння першого порядку. Початкова умова: $y(x_0) = y_0$.

Побудуйте графік частинного розв'язку на інтервалі $[x_0, x_{end}]$.

Знайдіть значення розв'язку в точці x_k .

2) З використанням функції *solve_ivp()* модуля *SciPy* знайдіть числовий розв'язок задачі Коші для заданого звичайного диференціального рівняння першого порядку на інтервалі $[x_0, x_{end}]$. Початкова умова: $y(x_0) = y_0$.

Варіант	Рівняння	x_0	y_0	x_{end}	x_k
1	$(x^2 + 1)y' + 4xy = 3$	0	0	4	1
2	$(1 - x)(y' + y) = e^{-x}$	2	1	6	4
3	$xy' - 2y = 2x^4$	1	0	2	2
4	$3y' = 2x(x^2 + y)$	0	1	1	1
5	$y' - y = e^x$	0	1	2	2
6	$xy' + y + xe^{-x^2} = 0$	1	$\frac{1}{2e}$	2	1,5
7	$\cos(x) = y + y'$	1	0	4	3
8	$x^2y' + xy + 1 = 0$	1	0	10	2
9	$xy' + y = 4x^3 + 3x^2$	1	2	3	2
10	$2x = 5y' + 4\ln(x)$	2	1	6	4
11	$\frac{y'}{x} = 2y$	1	1	2	2
12	$x(y' - y) = e^x$	1	0	4	3
13	$y = x(y' - x\cos(x))$	$\pi / 2$	0	π	π
14	$(xy' - 1)\ln(x) = 2y$	e	0	100	10
15	$2e^x y' = y + 1$	0	1	3	2
16	$xy' + (x + 1)y = 3x^2 e^{-x}$	1	0	10	4
17	$x^2 + xy = 2xy'$	1	2	6	5
18	$(x + 1)y' + y = x^3 + x^2$	0	0	3	1

19	$x^2 + xy' - 2y = 0$	1	0	4	3
20	$xy' + y = \sin(x)$	$\pi / 2$	$2 / \pi$	3π	9
21	$(x^2 - 1)y' - xy = x^3 - x$	$\sqrt{2}$	1	4	2
22	$(x^2 + 1)y' + y = 1$	0	2	3	2
23	$y' \operatorname{ctg}(x) - y = 2 \cos^2(x) \operatorname{ctg}(x)$	0	0	1,5	1
24	$x^2 y' = 2xy + 3$	1	-1	8	2
25	$6y' + 2xy = xe^{-x^2}$	0	0	5	2
26	$2y' - x^2 e^{x^3} - 3x^2 y = 0$	0	1	1	1
27	$xy' + y = \ln(x) + 1$	1	0	20	e
28	$y + 3y' = y\sqrt{x}$	4	-2	8	6
29	$3y' + 2y = y^2 e^x$	0	1	2	1
30	$-2x^2 y - xy' = 4y$	1	$1 / e$	2	1,5

Завдання 7.2

З використанням функції `solve_ivp()` модуля `SciPy` знайдіть числовий розв'язок задачі Коші для заданої системи двох звичайних диференціальних рівнянь першого порядку на інтервалі $[a, b]$.

№	y_1'	y_2'	$y_1(a)$	$y_2(a)$	a	b
1	$\operatorname{arctg}\left(\frac{1}{1+y_1^2+y_2^2}\right)$	$\sin(y_1 y_2)$	1	2	-1	1
2	$\operatorname{arctg}(x^2+y_2^2)$	$\sin(x+y_1)$	0,5	1,5	0	2
3	$x^2 y_1 + y_2$	$\cos(y_1 + x y_2)$	-1	1	0	2
4	$x^2 + y_2^2 - y_1^2$	$\cos(x y_1 y_2)$	2	0	0	5
5	$\frac{x}{\sqrt{1+x^2+y_2^2}}$	$\frac{y_2}{\sqrt{1+x^2+y_1^2}}$	0,2	-0,4	-1	1
6	$\sin(x^2+y_2^2)$	$\cos(x y_1)$	0	0	0	4
7	$\sin(y_2)$	$\cos(y_1)$	0,5	-0,5	-1	3
8	$x \cos(y_1 + y_2)$	$\sin(y_1 - y_2)$	-0,6	2	2	5
9	$\sin(y_1) \cos^2(y_2)$	$\cos(y_1) \cos(y_2)$	0,75	0	-1	3
10	$2\sqrt{3x^2+y_1^2+y_1}$	$\sqrt{x^2+y_1^2+y_2}$	0,5	1,2	0	2
11	$\frac{y_1^2}{y_2}$	$y_1^2 - y_2$	1	1	1	3
12	$y_2 + \sqrt{x^2+y_1^2}$	$y_2 \cos(x) - \sin(2x)$	0,8	3,5	2	4
13	$\frac{y_1+x}{e^{y_1+y_2}}$	$\frac{y_2-x}{e^{y_1+y_2}}$	1	-1	2	4
14	$\frac{1}{3+e^{-y_1}}$	$\frac{1}{2+e^{-y_1}}$	0	-3	2	5
15	$\cos(y_1 y_2)$	$\sin(y_1 + y_2)$	0	0	0	2
16	$y_2 \ln(x)$	$y_1 + y_2^2$	-2	-1	1	4
17	$x + y_1^2$	$(y_1 - y_2)^2$	0	0	-1	1
18	$\operatorname{arctg}(x + y_2)$	$\sin(y_1)$	0	0	-2	1
19	$y_1 + y_2$	$\frac{1}{1+y_1^2+y_2^2}$	0	0	0	4

20	$y_1^2 + y_2^2$	$y_1 y_2$	-1	1	0	3
21	$y_2 + 2e^x$	$y_1 + x^2$	1	0	1	2
22	$3y_1 + 2y_2 + 4e^{5x}$	$y_1 + 2y_2$	0	1	0	1
23	$4y_1 + y_2 - e^{2t}$	$y_2 - 2y_1$	1	1	0	1
24	$5y_1 - 3y_2 + 2e^{3t}$	$y_1 + y_2 + 5e^{-t}$	0	0	0	2
25	$y_1 + 2y_2$	$y_1 - 5\sin(x)$	0,5	0,5	0	3
26	$2y_1 - y_2$	$y_1 + 2y_2 - 18x$	0	0,5	0	2
27	$2y_1 + 4y_2 - 8$	$3y_1 + 6y_2$	0,5	0	1	2
28	$y_1 - y_2 + 2\sin(x)$	$2y_1 - y_2$	0	0	1	3
29	$4y_1 - 3y_2 + \sin(x)$	$2y_1 - y_2 - 2\cos(x)$	0	1	0	1
30	$2y_1 - 4y_2 + \sin(x)$	$y_1 - 3y_2 - \cos(x)$	1	0	1	2

Завдання 7.3

З використанням функції `solve_ivp()` модуля `SciPy` знайдіть числовий розв'язок задачі Коші для заданого звичайного диференціального рівняння другого порядку на інтервалі $[0, 2]$.

№	Рівняння	$y(0)$	$y'(0)$
1	$y'' - 2y' + y = -12\cos(2x) - 9\sin(2x)$	-2	0
2	$y'' - 6y' + 9y = 9x^2 - 39x + 65$	-1	1
3	$y'' + 2y' + 2y = 2x^2 + 8x + 6$	1	4
4	$y'' - 6y' + 25y = 9\sin(4x) - 24\cos(4x)$	2	-2
5	$y'' - 14y' + 53y = 53x^3 - 42x^2 + 59x - 14$	0	7
6	$y'' + 6y = e^x(\cos(4x) - 8\sin(4x))$	0	5
7	$y'' - 4y' + 20y = 16xe^{2x}$	1	2
8	$y'' - 12y' + 36y = 32\cos(2x) + 24\sin(2x)$	2	4
9	$y'' + y = x^3 - 4x^2 + 7x - 10$	2	3
10	$y'' - y = (14 - 16x)e^{-x}$	0	-1
11	$y'' + 8y' + 16y = 16x^2 - 16x + 66$	3	0
12	$y'' + 10y' + 34y = -9e^{-5x}$	0	6
13	$y'' - 6y' + 25y = (32x - 12)\sin(x) - 36x\cos(3x)$	4	0
14	$y'' + 25y = e^x(\cos(5x) - 10\sin(5x))$	3	-4
15	$y'' + 2y' + 5y = -8e^{-x}\sin(2x)$	2	6
16	$y'' - 10y' + 25y = e^{5x}$	1	0
17	$y'' + y' - 12y = (16x + 22)e^{4x}$	3	5
18	$y'' - 2y' + 5y = 5x^2 + 6x - 12$	0	2
19	$y'' + 8y' + 16y = 16x^3 + 24x^2 - 10x + 8$	1	3
20	$y'' - 2y' + 37y = 36e^x\cos(6x)$	0	6
21	$y'' - 8y' = 16 + 48x^2 - 128x^3$	-1	14
22	$y'' + 12y' + 36y = 72x^3 - 18$	1	0
23	$y'' + 3y' = (40x + 58)e^{2x}$	0	2
24	$y'' - 9y' + 18y = 26\cos(x) - 8\sin(x)$	0	2
25	$y'' + 8y' = 18x + 60x^2 - 32x^3$	5	2

26	$y'' - 3y' + 2y = -\sin(x) - 7\cos(x)$	2	7
27	$y'' + 2y' = 6x^2 + 2x + 1$	2	2
28	$y'' + 16y = 32e^{4x}$	2	0
29	$y'' + 5y' + 6y = 52\sin(2x)$	-2	-2
30	$y'' - 4y = 8e^{2x}$	1	-8

7.5 Приклади розв'язання завдань

Завдання 7.1

1) З використанням функції *dsolve()* модуля *SymPy* знайдіть загальний і частинний аналітичні розв'язки заданого звичайного диференціального рівняння першого порядку. Початкова умова: $y(1) = 0$.

Побудуйте графік частинного розв'язку на інтервалі $[1, 3]$.

Знайдіть значення розв'язку в точці $x_k = 2$.

2) З використанням функції *solve_ivp()* модуля *SciPy* знайдіть числовий розв'язок задачі Коші для заданого звичайного диференціального рівняння першого порядку на інтервалі $[1, 3]$. Початкова умова: $y(1) = 0$.

$$(1+x)y' - e^{-x} + x = xy.$$

Розв'язання

1) Створюємо задане диференціальне рівняння в символьній формі:

```
import sympy as smp
y = smp.Function('y')
x = smp.Symbol('x')
diff_eq = smp.Eq((1 + x) * y(x).diff(x) - smp.exp(-x) + x, x * y(x))
diff_eq
```

$$x + (x + 1) \frac{d}{dx} y(x) - e^{-x} = xy(x)$$

Знаходимо загальний розв'язок диференціального рівняння:

```
gen_sol = smp.dsolve(diff_eq)
gen_sol
```

$$y(x) = \frac{C_1 e^x + x + 1 - \frac{e^{-x}}{2}}{x + 1}$$

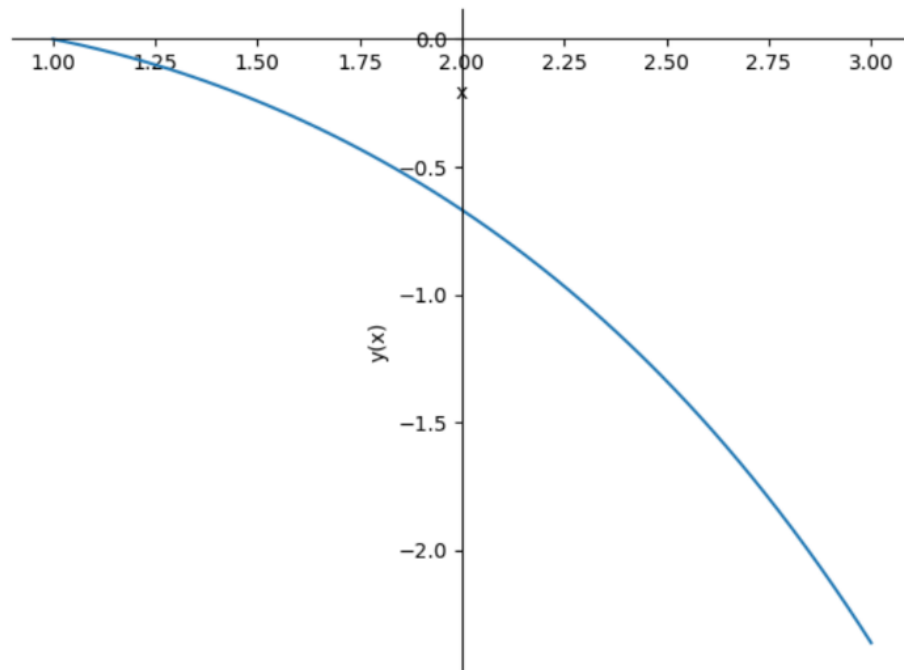
Знаходимо частинний розв'язок диференціального рівняння:

```
x_0 = 1 # Початкова умова
y_0 = 0
part_sol = smp.dsolve(diff_eq, ics = {y(x_0) : y_0})
part_sol
```

$$y(x) = \frac{x + \frac{(1-4e)e^x}{2e^2} + 1 - \frac{e^{-x}}{2}}{x + 1}$$

Будуємо графік частинного розв'язку диференціального рівняння:

```
import matplotlib.pyplot as plt
y = part_sol.rhs
x_end = 3
smp.plot(y, (x, x_0, x_end), xlabel = 'x', ylabel = 'y(x)')
```



Виконуємо підстановку заданого значення x_k замість символічної змінної x у вираз $y(x)$ і знаходимо значення розв'язку в цій точці:

```
x_k = 2
print('y(', x_k, ') =', y.subs({x : x_k}).n(3))

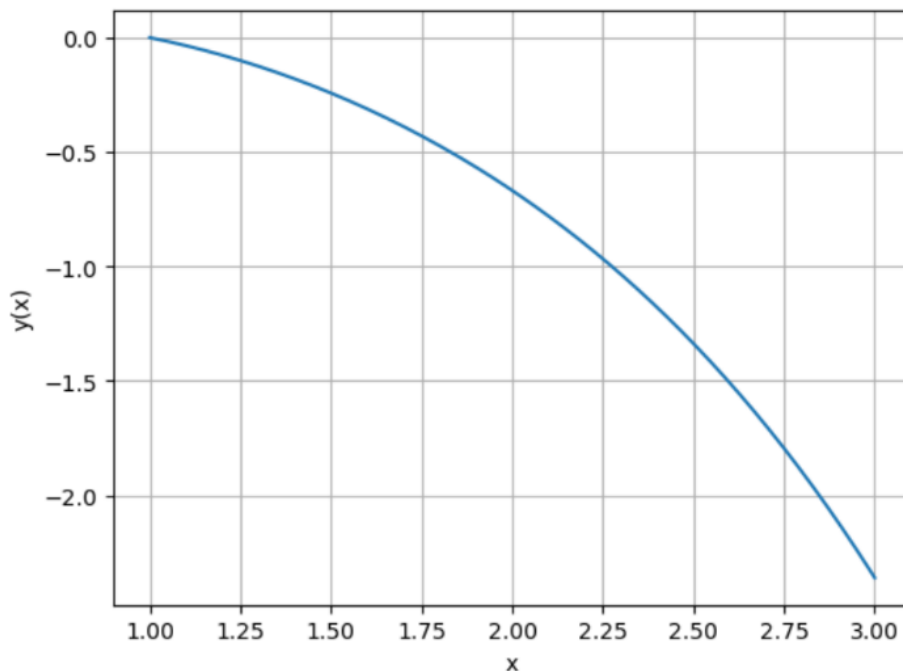
y( 2 ) = -0.668
```

2) Перепишемо задане диференціальне рівняння в явному вигляді:

$$y' = \frac{xy - x + e^{-x}}{1 + x}.$$

Знаходимо числовий розв'язок задачі Коші для звичайного диференціального рівняння першого порядку:

```
from scipy.integrate import solve_ivp
import numpy as np
import matplotlib.pyplot as plt
# Python-функція для розрахунку виразу в правій частині явного диференціального рівняння:
def f(x, y):
    return (x * y - x + np.exp(-x)) / (1 + x)
x_0 = 1 # Інтервал інтегрування
x_end = 3
y_0 = 0 # Значення функції в початковій точці
m = 200 # Кількість точок, в яких обчислюється розв'язок
# Матриця зі значеннями незалежної змінної, для яких обчислюється розв'язок:
X = np.linspace(x_0, x_end, m)
# Об'єкт, що містить числовий розв'язок задачі Коші для заданого рівняння:
ivp_result = solve_ivp(f, [x_0, x_end], [y_0], t_eval = X)
plt.plot(ivp_result.t, ivp_result.y[0]) # Побудова графіка числового розв'язку
plt.xlabel('x'); plt.ylabel('y(x)') # Назви координатних осей
plt.grid(True) # Сітка
```



Завдання 7.2

З використанням функції `solve_ivp()` модуля `SciPy` знайдіть числовий розв'язок задачі Коші для заданої системи двох звичайних диференціальних рівнянь першого порядку на інтервалі $[0, 2]$:

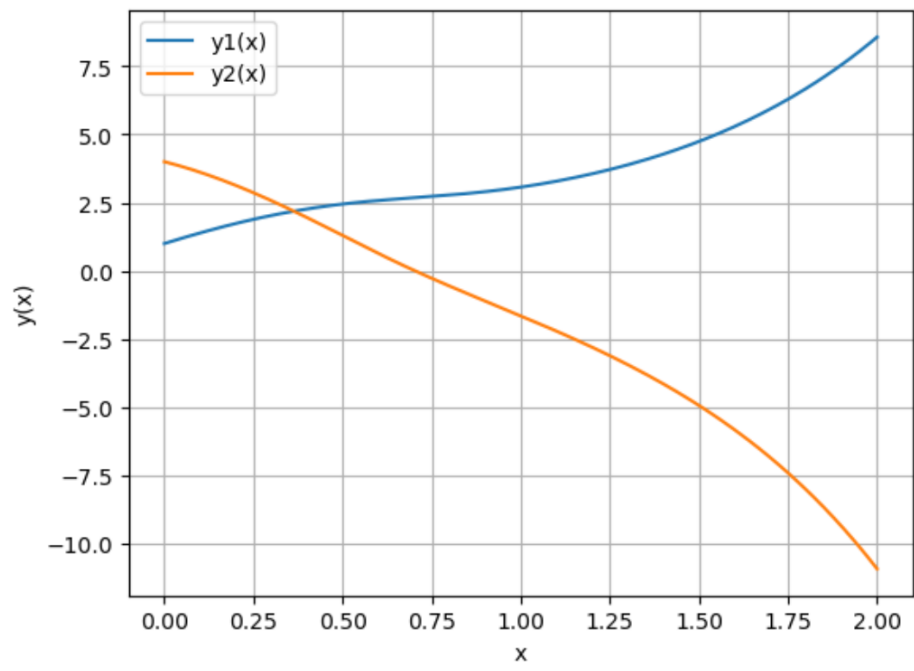
$$\begin{cases} y_1' = \sqrt{\frac{1}{2y_1} + y_2^2 + \sin(x)}, \\ y_2' = -3y_1 + e^x - \operatorname{arctg}(y_2). \end{cases}$$

Початкові умови: $y_1(0) = 1$; $y_2(0) = 4$.

Розв'язання

Знаходимо числовий розв'язок задачі Коші для системи звичайних диференціальних рівнянь першого порядку:

```
from scipy.integrate import solve_ivp
import numpy as np
import matplotlib.pyplot as plt
# Python-функція для розрахунку виразів в правих частинах явних диференціальних
# рівнянь системи. Індекс 0 використовується для функції y1, індекс 1 – для y2:
def f(x, y):
    return [np.sqrt(1 / (2 * y[0]) + y[1]**2 + np.sin(x)),
            -3 * y[0] + np.exp(x) - np.arctan(y[1])]
a = 0 # Інтервал інтегрування
b = 2
y1_a = 1 # Значення функцій в початковій точці
y2_a = 4
m = 200 # Кількість точок, в яких обчислюється розв'язок
# Матриця зі значеннями незалежної змінної, для яких обчислюється розв'язок:
X = np.linspace(a, b, m)
# Об'єкт, що містить числовий розв'язок задачі Коші для заданої системи рівнянь:
ivp_result = solve_ivp(f, [a, b], [y1_a, y2_a], t_eval = X)
# Побудова графіка числового розв'язку:
plt.plot(ivp_result.t, ivp_result.y[0], ivp_result.t, ivp_result.y[1])
plt.xlabel('x'); plt.ylabel('y(x)') # Назви координатних осей
plt.legend(['y1(x)', 'y2(x)']) # Додавання легенди
plt.grid(True) # Сітка
```



Завдання 7.3

З використанням функції `solve_ivp()` модуля `SciPy` знайдіть числовий розв'язок задачі Коші для заданого звичайного диференціального рівняння другого порядку на інтервалі $[0, 2]$.

$$y'' + 10y' - 8y = (6x + 1)e^{2x}.$$

Початкові умови: $y(0) = 1$; $y'(0) = -1$.

Розв'язання

Перепишемо задане диференціальне рівняння в явному вигляді:

$$y'' = (6x + 1)e^{2x} + 8y - 10y'.$$

Виконаємо заміни:

$$y = u_0,$$

$$y' = u'_0 = u_1.$$

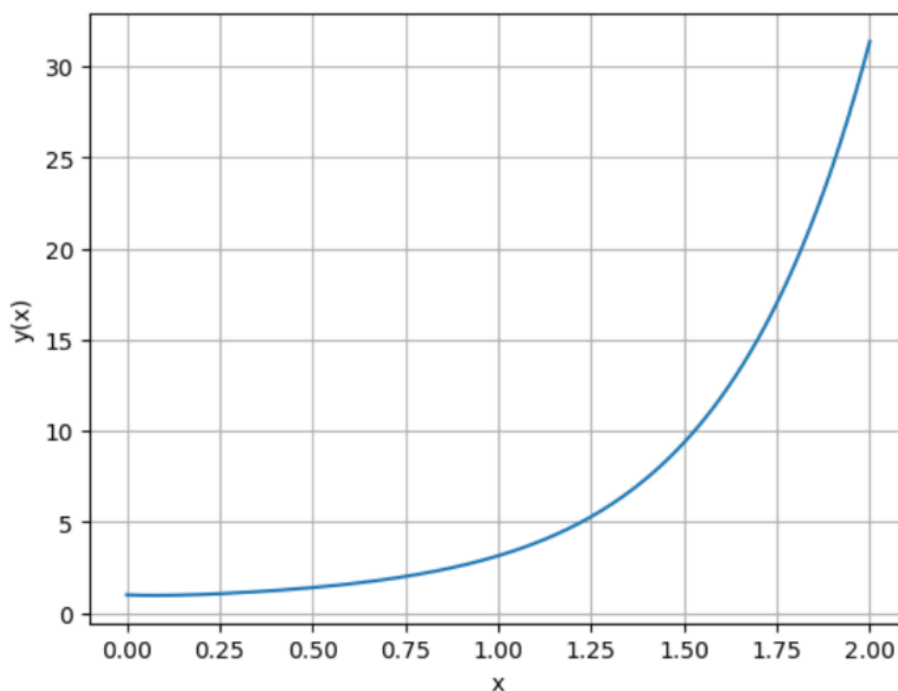
Тоді найстарша похідна $y'' = u'_1$.

Одержимо систему двох звичайних диференціальних рівнянь першого порядку:

$$\begin{cases} u'_0 = u_1; \\ u'_1 = (6x + 1)e^{2x} + 8u_0 - 10u_1. \end{cases}$$

Знаходимо числовий розв'язок задачі Коші для звичайного диференціального рівняння другого порядку:

```
from scipy.integrate import solve_ivp
import numpy as np
import matplotlib.pyplot as plt
# Python-функція для розрахунку виразів
# в правих частинах явних диференціальних рівнянь системи:
def f(x, u):
    return [u[1], (6 * x + 1) * np.exp(2 * x) + 8 * u[0] - 10 * u[1]]
a = 0 # Інтервал інтегрування
b = 2
u0_a = 1 # Значення функцій в початковій точці
u1_a = -1
m = 200 # Кількість точок, в яких обчислюється розв'язок
# Матриця зі значеннями незалежної змінної, для яких обчислюється розв'язок:
X = np.linspace(a, b, m)
# Об'єкт, що містить числовий розв'язок задачі Коші для заданого рівняння:
ivp_result = solve_ivp(f, [a, b], [u0_a, u1_a], t_eval = X)
plt.plot(ivp_result.t, ivp_result.y[0]) # Побудова графіка числового розв'язку
plt.xlabel('x'); plt.ylabel('y(x)') # Назви координатних осей
plt.grid(True) # Сітка
```



Контрольні питання

1. В чому полягає відмінність звичайного диференціального рівняння від диференціального рівняння з частинними похідними?
2. Як визначається частинна похідна функції кількох змінних?
3. Який найменший порядок може мати звичайне диференціальне рівняння?
4. Скільки сталих інтегрування входить до загального розв'язку звичайного диференціального рівняння п'ятого порядку?
5. З якою метою при розв'язанні звичайного диференціального рівняння задають додаткові умови?
6. Які розв'язки отримують в результаті розв'язання задачі Коші та крайової задачі для звичайного диференціального рівняння?
7. В чому полягає відмінність між аналітичним і числовим розв'язками звичайного диференціального рівняння?
8. Від чого залежать похибки числового розв'язку звичайного диференціального рівняння?
9. Що є розв'язком задачі Коші для системи звичайних диференціальних рівнянь першого порядку?
10. Які додаткові умови необхідно задати, щоб розв'язати задачу Коші для системи трьох звичайних диференціальних рівнянь першого порядку?
11. Як знайти значення числового розв'язку звичайного диференціального рівняння в заданій точці з використанням функції `solve_ivp()` модуля `SciPy`?
12. Які основні атрибути містить об'єкт, що повертається функцією `solve_ivp()` модуля `SciPy`?

Лабораторна робота № 8

РОЗВ'ЯЗАННЯ ОПТИМІЗАЦІЙНИХ ЗАДАЧ

Мета роботи: одержання практичних навичок визначення екстремумів функції однієї змінної.

8.1 Визначення екстремумів функції однієї змінної методами класичного математичного аналізу

Оптимізація – це цілеспрямована діяльність, яка полягає у досягненні найкращого результату за заданих умов і обмежень.

Для розв'язання оптимізаційної задачі необхідно мати можливість змінювати значення незалежних параметрів об'єкта оптимізації в певних межах. Співвідношення, що визначають ці межі, мають назву обмежень. Обмеження можуть бути задані у вигляді рівностей, нерівностей або бути відсутніми [23].

Оптимізаційна задача потребує кількісної оцінки якостей об'єкта оптимізації. Таку оцінку задає цільова функція, яку також називають критерієм оптимальності, функцією якості, економічним критерієм тощо. Цільова функція дає змогу порівнювати результати, які досягаються за різних значень параметрів оптимізації, і обирати найкращий варіант.

Оптимізаційна задача полягає в знаходженні (з врахуванням обмежень) таких значень параметрів оптимізації x_i , за яких цільова функція $r(x_i)$ набуває найбільшого або найменшого значення.

Для розв'язання оптимізаційних задач використовують, зокрема, методи лінійного та нелінійного програмування, динамічного програмування, метод множників Лагранжа, принцип максимуму тощо, а також методи класичного математичного аналізу.

Якщо цільова функція залежить лише від одного параметра і має неперервну та однозначну першу похідну, то в точці екстремуму ця похідна дорівнює нулю. Точка x_k , для якої справджується рівність

$$\left. \frac{dr(x)}{dx} \right|_{x=x_k} = 0, \quad (8.1)$$

має назву стаціонарної точки.

Не кожна стаціонарна точка є точкою екстремуму. Щоб з'ясувати, чи є значення цільової функції в x_k мінімальним або максимальним в околі цієї точки, необхідно проаналізувати знаки вищих похідних $r(x)$ в x_k .

Проте на інтервалі, яким зазвичай обмежують значення параметра оптимізації, цільова функція може мати декілька локальних екстремумів. А метою оптимізаційної задачі є знаходження найбільшого або найменшого значення функції на всьому заданому інтервалі. Крім того, екстремальні значення можуть досягатись на межах інтервалу, які не є стаціонарними точками [11].

З огляду на ці особливості, пошук екстремального значення цільової функції на відрізку $[a, b]$ доцільно здійснювати наступним чином: визначити всі стаціонарні точки всередині інтервалу, доповнити їх межами a і b , безпосередньо розрахувати значення цільової функції в кожній із цих точок і з отриманого набору обрати найбільше або найменше значення.

Стаціонарні точки функції однієї змінної в *Python* можливо визначити з використанням функції `stationary_points()` модуля *SymPy*:

`sympy.stationary_points(f, symbol, domain = Reals)`

де f – символічний вираз функції; $symbol$ – символічна змінна, за якою виконується диференціювання; $domain$ – інтервал, на якому визначаються стаціонарні точки [7]. Інтервал можливо задати за допомогою функції

`sympy.Interval(a, b)`

де a і b – межі інтервалу. Якщо параметр $domain$ не задано, стаціонарні точки визначаються на всій множині дійсних чисел.

Функція `stationary_points()` повертає множину (*FiniteSet*) стаціонарних точок, яку можливо перетворити на список з використанням функції `list()`. Отримати відповідну числову матрицю можна за допомогою функції `array()` модуля *NumPy* з параметром $dtype = float$.

Межі інтервалу можна включити до одержаної матриці за допомогою функції `append()` модуля *NumPy*. Для розрахунків значень цільової функції зручно використовувати відповідну *Python*-функцію, яку одержують із символічного виразу за допомогою функції `lambdify()` модуля *SymPy* (див. розділ 6).

Для визначення максимального або мінімального елемента матриці використовують функції `max()` та `min()` модуля *NumPy* відповідно. Індекси цих елементів знаходять за допомогою функцій `argmax()` і `argmin()`.

8.2 Завдання до лабораторної роботи

Знайдіть координати точок, в яких задана цільова функція досягає найменшого і найбільшого значень на вказаному інтервалі. Побудуйте графік цільової функції та позначте на ньому ці точки.

Варіант	Цільова функція	Інтервал
1	$0,1x^5 - 3x^4 - 2x^3 + x^2 + 0,7x + 5,2$	$[-1; 1]$
2	$x^5 + 1,2x^4 - 10x^3 + 6,5x^2 + 4x - 1$	$[-4; 3]$
3	$-0,5x^5 + 8x^4 - 15x^3 + 3x^2 + 7x - 1$	$[-0,5; 1,5]$
4	$0,2x^5 - x^4 - 2x^3 + 10x^2 + 2x + 15$	$[-1; 5]$
5	$0,5x^5 - 4x^4 + 9x^3 - 7x^2 + 2x - 4$	$[-0,5; 2]$
6	$-2x^5 - 10x^4 - 8x^2 + 10x + 3$	$[-5; 2]$
7	$0,5x^5 - 2x^3 + 10x^2 + 2x + 15$	$[-3; 1]$
8	$x^5 - 5x^4 + 5,2x^3 + 0,7x + 3,2$	$[-1; 3,5]$
9	$x^5 - 5,5x^4 + 5x^3 + 7x^2 + 12$	$[-1; 3,5]$
10	$0,8x^5 - 4x^4 - 10x^2 + 12$	$[-2; 5]$
11	$2x^5 - 7,2x^4 - 5x^3 + 2x^2 + 10x + 10$	$[-2; 4]$
12	$2x^5 - 4x^4 - 4x^3 - 3x^2 - 3x$	$[-1,5; 2,5]$
13	$-3x^5 + 12x^4 + 8x^2 - 7x - 3$	$[-1,5; 4]$
14	$4x^5 - 15x^4 + 6x^3 - 2x^2 + 30x + 3$	$[0; 3]$
15	$-1,3x^5 + 10x^4 - 3x^3 - 12x^2 + x + 20$	$[-1; 1,5]$
16	$6,4x^5 - 15x^4 - 30x^3 - 8x^2 + 45,8x + 324$	$[-2; 3,5]$
17	$2,5x^5 - 6x^4 - 6x^3 + 21x^2 - 14x + 30$	$[-1,5; 2,5]$
18	$-2,7x^5 - 3x^4 + 15x^3 + 12x^2 + 8x - 30$	$[-2,5; 2]$
19	$3x^5 - 10x^4 - 10x^2 + 82x + 12$	$[0; 3,5]$
20	$-x^5 - 5x^4 - 3x^3 + 7x^2 - 3,8x - 9$	$[-3,5; 1]$
21	$0,1x^5 + 10x^4 + 10x^3 - 9x^2 - 3x + 3$	$[-1,5; 1]$

22	$-2x^5 + 4,8x^4 - 4x^3 + 45x + 1$	$[-1,5; 2,5]$
23	$2x^5 + 8x^4 - x^3 - 2x^2 + 28x - 7$	$[-3,5; 1]$
24	$-12x^5 - 2x^4 - 4x^3 + 11x^2 + 6x - 2$	$[-0,5; 1]$
25	$x^5 - 3x^4 - 6x^3 - x^2 + 11x - 1$	$[-2; 4]$
26	$-0,3x^5 + 3x^4 - 8x^3 + 4x^2 - 6x + 1$	$[0; 6]$
27	$-1,5x^5 + 7x^4 + x^3 + 12x^2 - 72x + 5$	$[0; 4,5]$
28	$18x^5 - 12x^4 - 22x^3 - 9x^2 + 21x + 12$	$[-1; 1,5]$
29	$-3x^5 + 7x^4 - 11x^3 + 50x^2 - 18x - 16$	$[-0,5; 2,5]$
30	$8x^5 - 7x^4 - 8x^3 - 8x^2 - 6x + 1$	$[-1; 1,5]$

8.3 Приклад розв'язання завдання

Знайдіть координати точок, в яких цільова функція

$$r(x) = x^5 - 5x^4 + 6x^3 + 1$$

досягає найменшого і найбільшого значень на інтервалі $[-1, 3]$. Побудуйте графік цільової функції та позначте на ньому ці точки.

Розв'язання

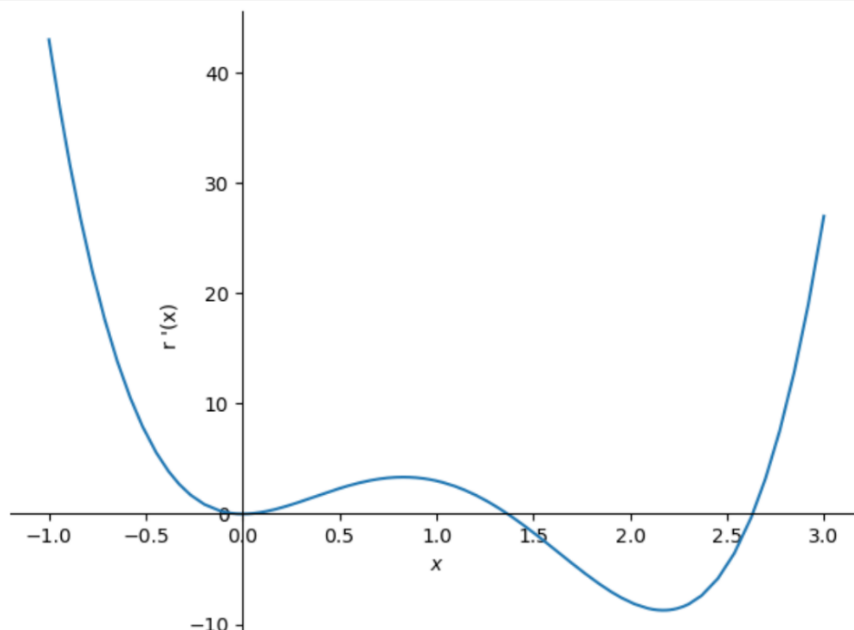
Знаходимо символний вираз першої похідної цільової функції:

```
import sympy as smp
x = smp.Symbol('x') # Параметр оптимізації
# Символьний вираз цільової функції:
r = x ** 5 - 5 * x ** 4 + 6 * x ** 3 + 1
r1 = smp.diff(r, x)
print("r '(x) =")
r1
```

$$r'(x) = 5x^4 - 20x^3 + 18x^2$$

Будуємо графік першої похідної цільової функції:

```
import matplotlib.pyplot as plt
a = -1 # Межі інтервалу, на якому досліджується цільова функція
b = 3
smp.plot(r1, (x, a, b), ylabel = "r '(x)")
```



Визначаємо стаціонарні точки:

```
K = smp.stationary_points(r, x, smp.Interval(a, b)).n(4)
K
```

```
{0, 1.368, 2.632}
```

Формуємо матрицю значень параметра оптимізації, за яких можливі екстремуми цільової функції:

```
import numpy as np
X = np.array(list(K), dtype = float)
X = np.append(X, (a, b)) # Додаємо межі інтервалу
X_T = X.reshape(-1, 1) # Транспонована матриця
print(np.round(X_T, 3))
```

```
[[ 1.368]
 [ 2.632]
 [ 0.    ]
 [-1.    ]
 [ 3.    ]]
```

Розраховуємо значення цільової функції в цих точках:

```
# Python-функція для розрахунку значень цільової функції:
f = smp.lambdify(x, r)
R = f(X) # Матриця значень цільової функції
R_T = R.reshape(-1, 1) # Транспонована матриця
print(np.round(R_T, 3))
```

```
[[ 3.641]
 [-3.241]
 [ 1.    ]
 [-11.   ]
 [ 1.    ]]
```

Знаходимо координати точок, в яких цільова функція досягає найбільшого і найменшого значень:

```

r_max = np.max(R) # Максимальне значення цільової функції
# Значення параметра оптимізації, за якого досягається максимум функції:
x_max = X[np.argmax(R)]
r_min = np.min(R) # Мінімальне значення цільової функції
# Значення параметра оптимізації, за якого досягається мінімум функції:
x_min = X[np.argmin(R)]
print('Максимальне значення цільової функції r_max =',
      round(r_max, 3), 'при x =', round(x_max, 3))
print('Мінімальне значення цільової функції r_min =',
      round(r_min, 3), 'при x =', round(x_min, 3))

```

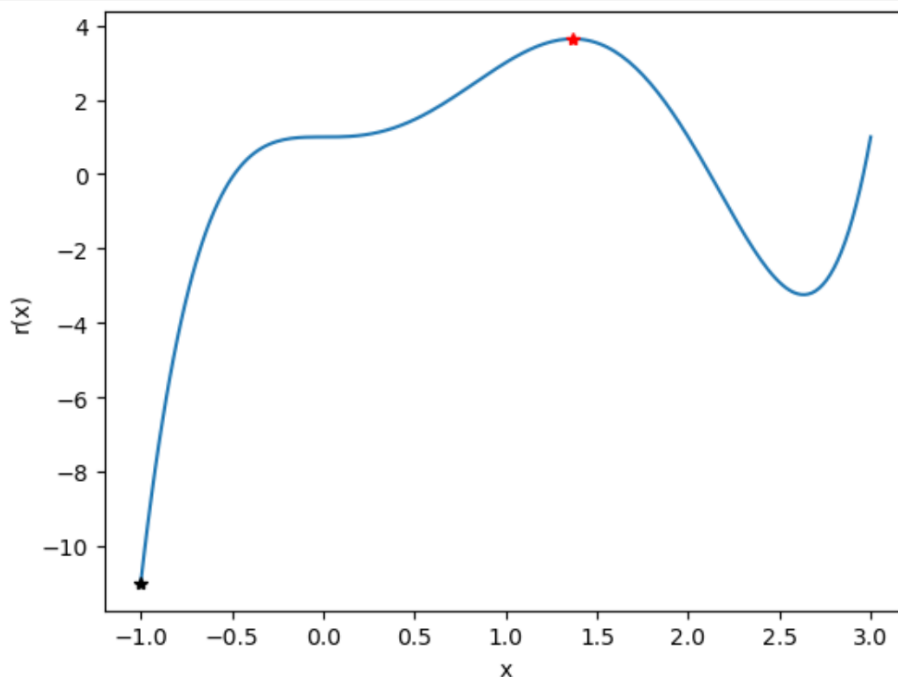
Максимальне значення цільової функції $r_{\max} = 3.641$ при $x = 1.368$
Мінімальне значення цільової функції $r_{\min} = -11.0$ при $x = -1.0$

Будуємо графік цільової функції і позначаємо точки, в яких вона досягає найменшого і найбільшого значень на заданому інтервалі:

```

m = 200 # Кількість точок, за якими будується графік
X_vals = np.linspace(a, b, m) # Матриця значень параметра оптимізації
R_vals = f(X_vals) # Матриця значень цільової функції
plt.plot(X_vals, R_vals, x_max, r_max, 'r*', x_min, r_min, 'k*')
plt.xlabel('x'); plt.ylabel('r(x)')

```



Відповідь:

- максимальне значення цільової функції: $r_{\max} = 3,641$ при $x = 1,368$;
- мінімальне значення цільової функції: $r_{\min} = -11$ при $x = -1$.

Примітка:

Значення параметра оптимізації та цільової функції зручніше відображати у вигляді стовпців. Для цього одновимірні матриці-рядки X і R необхідно перетворити на двовимірні матриці-стовпці. Проте функція *transpose()* модуля *NumPy*, яку розглянуто в другій роботі, призначена для транспонування матриць із двома або більше вимірами.

В цьому випадку для зміни розмірності матриці можна використати метод *reshape()* модуля *NumPy*. Два параметри цього методу визначають кількість рядків і стовпців нової матриці відповідно.

Якщо один із параметрів дорівнює -1, то його значення розраховується автоматично таким чином, щоб збереглась загальна кількість елементів початкової матриці.

Зокрема, якщо кількість стовпців задати рівною 1, то кількість рядків нової матриці відповідатиме кількості елементів початкової матриці.

Контрольні питання

1. В чому полягає задача оптимізації?
2. Що є об'єктом оптимізації?
3. Які обмеження можуть накладатись при розв'язанні оптимізаційної задачі?
4. Яку роль в задачі оптимізації має цільова функція?
5. Що є екстремумом функції?
6. Яка точка є стаціонарною?
7. Як можливо визначити стаціонарні точки, якщо аналітичний вираз цільової функції невідомий, а її значення розраховуються за певним алгоритмом?
8. Які обмеження мають методи класичного математичного аналізу при розв'язанні оптимізаційних задач?
9. Чи може існувати екстремум цільової функції однієї змінної в точці, в якій перша похідна цієї функції має розрив?
10. Як задачу пошуку максимуму функції звести до еквівалентної задачі пошуку мінімуму?
11. В чому полягає відмінність локального і глобального екстремумів функції?
12. Як в *Python* визначити максимальний та мінімальний елементи матриці?

Список використаної літератури

1. *Anaconda Installer*. URL: <https://www.anaconda.com/download> (дата звернення: 03.06.2025)
2. *JupyterLite*. URL: <https://jupyter.org/try-jupyter/lab/> (дата звернення: 03.06.2025)
3. Крєневич А.П. *Python у прикладах і задачах. Частина 1. Структурне програмування : навч. посіб. /* ВПЦ «Київський Університет». Київ, 2017. 206 с.
4. *Math – Mathematical functions*. URL: <https://docs.python.org/3/library/math.html> (дата звернення: 03.06.2025)
5. *NumPy reference*. URL: <https://numpy.org/doc/stable/reference/index.html#reference> (дата звернення: 03.06.2025)
6. *SciPy API*. URL: <https://docs.scipy.org/doc/scipy/reference/index.html> (дата звернення: 03.06.2025)
7. *API Reference SymPy*. URL: <https://docs.sympy.org/latest/reference/index.html#reference> (дата звернення: 03.06.2025)
8. *Matplotlib*. URL: https://matplotlib.org/stable/api/pyplot_summary.html (дата звернення: 03.06.2025)
9. Яковенко А. В. *Основи програмування. Python. Частина 1 : підручник. /* КПІ ім. Ігоря Сікорського. Київ, 2018. 195 с.
10. Івашко В. В. *Конспект лекцій з навчальної дисципліни «Основи програмування».* : Чернівецький національний університет імені Юрія Федьковича. Чернівці, 2021. 177 с.
11. Прокопенко Ю. В., Татарчук Д. Д., Казміренко В. А. *Обчислювальна математика : навч. посіб. /* Видавництво. Київ, 2013. 224 с.
12. Романюк М. І., Батіна О. А. *Конспект лекцій з навчальної дисципліни "Обчислювальна математика" : навч. посіб. /* КПІ ім. Ігоря Сікорського. Київ, 2019. 193 с.
13. Задачин В. М., Конюшенко І. Г. *Чисельні методи : навч. посіб. /* Вид-во ХНЕУ ім. С. Кузнеця. Харків, 2014. 180 с.
14. Кветний Р. Н., Іванчук Я. В., Богач І. В., Софіна О. Ю., Барабан М. В. *Методи та алгоритми комп'ютерних обчислень. Теорія і практика: підручник /* ВНТУ. Вінниця, 2023. 280 с.
15. Дорошенко А. Ю., Погорілий С. Д., Дорогий Я. Ю., Глушко Є. В. *Програмування числових методів мовою Python : навч. посіб. /* за ред. А. В. Анісімова.: ВПЦ «Київський університет». Київ, 2013. 463 с.

16. Сеті Ю. О. Сучасні методи аналітичного і числового програмування в задачах теоретичної фізики : метод. рекомендації / Чернівецький нац. ун-т. Чернівці, 2021. 69 с.
17. Фельдман Л. П., Петренко А. І., Дмитрієва О. А. Чисельні методи в інформатиці: навч. посіб. / ВНУ. Київ, 2006. 480с.
18. Бахвалов Н. С., Жидков Н. П., Кобельков Г. М. Численные методы : учеб. пособ. / Наука. Москва, 1987. 600 с.
19. Березовський В. Є., Ковальов Л. Є., Медведєва М. О. Чисельні методи з прикладами реалізації мовою Python : навч. посіб. / ВПЦ «Візаві». Умань, 2023. 88 с.
20. Степанець О. В. Числові методи: комп'ютерний практикум : навч. посіб. / КПІ ім. Ігоря Сікорського. Київ, 2020. 82 с.
21. Рябушко А. П., Бархатов В. В., Державец В. В., Юреть І. Е. Сборник индивидуальных заданий по высшей математике. В 3 ч. Ч 2 : учеб. пособ. / за ред. А. П. Рябушко. Минск : Выщ. шк., 1991. 352 с.
22. Брановицкая С. В., Медведев Р. Б., Фиалков Ю. Я. Вычислительная математика в химии и химической технологии : учебник / Вища школа. Киев, 1986. 216 с.
23. Бояринов А. И., Кафаров В. В. Методы оптимизации в химической технологии : учебник / Химия. Москва, 1969. 564 с.

Навчально-методичне видання

Шейкус Антон Романович, **Бакеркін** Владислав Романович

ОБЧИСЛЮВАЛЬНА МАТЕМАТИКА ТА ПРОГРАМУВАННЯ

Навчально-методичні рекомендації до лабораторних занять

Електронне видання

Експертний висновок склала к.т.н., доц. Олена ТІТОВА

Зареєстровано НМВ УДУНТ (№ 3.944 від 24.06.2025 р.)

В авторській редакції

Комп'ютерна верстка – Тетяна КІЖЛЮ

Формат 60x84 1/16. Ум. друк. арк. 7,86. Обл.-вид. арк. 7,92.

Зам. № 128

Видавець: Український державний університет науки і технологій
вул. Лазаряна, 2, ауд. 2216, м. Дніпро, 49010.

Свідоцтво суб'єкта видавничої справи ДК № 7709 від 14.12.2022.

Адреса дільниці оперативної поліграфії:
ННІ «Український державний хіміко-технологічний університет»
просп. Науки, 8, м. Дніпро, 49005.